Gobierno de México | Ciencia y Tecnología, Secretaría de Ciencia, Humanidades, Tecnología e Innovación | UNRC Universidad Nacional Rosario Castellanos

UNIVERSIDAD NACIONAL ROSARIO CASTELLANOS

LICENCIATURA EN CIENCIA DE DATOS PARA NEGOCIOS

TORRE DEL CARIBE: CAMPAÑA PUBLICITARIA INTELIGENTE PARA REDISTRIBUIR EL TURISMO HACIA EL SUR DE QUINTANA ROO

Documento técnico: cómo funciona cada modelo, fórmula por fórmula, con ejemplos resueltos con nuestros datos

EQUIPO - 2

Tutor:	Carlos Orozco Rocha
Alumnos	Brandon Uriel García Sánchez Enrique González Ortega Jesús Ramírez Isidro Maribel Mondragón Mercado
Grupo	PHLCDN-503-UST
Unidad Académica	Unidad Santo Tomás
Fecha de Entrega	Octubre de 2026

Contenido

Profesores y materias del proyecto 5
Cómo leer las fórmulas de este documento 5
Tres palabras que se usan en todo el documento 6
Cómo leer los fragmentos de código 6
Cómo está organizado el código 6
1. El problema y nuestra respuesta 8
1.1 La pregunta que nos pide responder el Problema Prototípico 8
1.2 El diagnóstico, con datos 8
1.3 Nuestra respuesta 9
1.4 Tres módulos que no se enciman 9
1.5 Las siete preguntas secundarias y dónde se responden 10
1.6 Cómo trabajamos 10
2. Los datos: de dónde sale cada cifra 11
2.1 La regla: solo fuentes oficiales, descargadas con código 11
2.2 Las fuentes 12
2.3 Lo que no existe (huecos declarados) 14
3. Grandes volúmenes de datos 15
3.1 La arquitectura medallón: Bronze, Silver y Gold 15
3.2 Spark: qué es y qué hace aquí 16
3.3 Reglas de limpieza: un cero no es lo mismo que un hueco 18
3.4 Cuando dos fuentes cuentan lo mismo (conciliación) 20
3.5 El almacén DuckDB y el modelo estrella 20
3.6 Streaming: la Torre en vivo 21
3.7 Privacidad 23
3.8 Las columnas que inferimos: de dónde sale cada una 23
4. Planteamiento: el problema medido 29
4.1 ¿Qué tan concentrado está el turismo? El índice de Herfindahl-Hirschman 29
4.2 ¿Por qué estos cinco lugares? La tabla de criterios 31
4.3 Variables, actores y relaciones 32
5. Minería de datos: encontrar patrones 33
5.1 El índice de presión turística (IPT) 33

5.2 Agrupamiento de los centros turísticos del país (método de Ward) 36
5.3 La forma del año (estacionalidad) 38
5.4 Clima raro con Isolation Forest (detección de anomalías) 40
5.5 Minería de texto de las reseñas 43
5.6 Qué hacer, dónde comer y dónde dormir: clasificación por léxico 46
6. Aprendizaje de máquina: predecir 47
6.1 Clasificación: ¿qué estado tendrá cada lugar el mes siguiente? 47
6.2 Pronóstico de visitantes: series de tiempo 54
7. Modelos estocásticos: la incertidumbre 61
7.1 Cadena de Markov semanal del norte 61
7.2 Tormentas: definición y distancia 64
7.3 Poisson: la probabilidad de tormenta en cada mes 65
7.4 Monte Carlo: escenarios malo, probable y bueno 68
7.5 Sensibilidad: ¿qué variable mueve más? 70
7.6 El año completo como proceso de Poisson 71
7.7 La cadena a largo plazo y la prueba de la propiedad de Markov 71
7.8 ¿Y si la campaña funciona mejor o peor? 72
8. Investigación de Operaciones: repartir el presupuesto 75
8.1 La pregunta 75
8.2 Los parámetros: cuánto rinde un peso en cada canal 75
8.3 El modelo de dos variables, resuelto con el método gráfico 76
8.4 El modelo completo 78
8.5 Cómo lo resuelve la computadora 80
8.6 ¿Cuánto cuesta cada regla? Precios sombra 82
8.7 Sensibilidad: ¿y si los supuestos están mal? 83
8.8 La etapa 2 en la práctica: el motor de la Torre 84
8.9 ¿Valió la pena el modelo estocástico? VSS y EVPI 84
9. Mercadotecnia digital: la campaña 87
9.1 Segmentar: lo que dicen los datos 87
9.2 Elegir: dos viajeras ideales (buyer persona) 88
9.3 Posicionar: la marca 89
9.4 Los mensajes: cada promesa con su dato 89
9.5 Los medios 90

9.6 Cómo se mide: el embudo y los indicadores 90
9.7 La regla anti-colapso 92
10. Integración y propuesta final 93
10.1 La estafeta: la salida de una materia es la entrada de la siguiente 93
10.2 Comparación de escenarios y propuesta 94
10.3 La página y el servidor 94
11. Lo que no se pudo medir y lo que se supuso 97
11.1 Supuestos 97
11.2 Lo que los modelos no pueden afirmar 97
Referencias 98
Anexo A. Dónde está cada cálculo 101
Anexo B. Glosario 103

Profesores y materias del proyecto

Este documento acompaña al Problema Prototípico del 5.º semestre, *Turismo inteligente sustentable para México*. El proyecto integra las seis Unidades de Competencia Académica (UCA) del semestre. La tabla indica qué capítulo corresponde a cada materia y qué criterio de la rúbrica atiende.

Materia (UCA)	Profesor	Capítulo	Criterio de la rúbrica
Herramientas para el Almacenamiento de Grandes Volúmenes de Datos	Víctor Gabriel Cruz Arrona	3	2. Gestión y almacenamiento (12 %)
Minería de Datos	Luis Becerril Espinosa	4 y 5	1. Planteamiento (8 %) y 3. Minería de datos (12 %)
Aprendizaje de Máquina	Raúl Jorge Domínguez Vilchis	6	4. Aprendizaje de máquina (12 %)
Modelos Estocásticos para la Toma de Decisiones	René Chablé García	7	5. Modelos estocásticos (10 %)
Investigación de Operaciones	Carlos Orozco Rocha (tutor del equipo)	8	6. Investigación de Operaciones (12 %)
Mercadotecnia Digital	Víctor Hugo Reyes Anselmo	9	7. Mercadotecnia digital (8 %)

Los criterios 8 (integración, 10 %) y 9 (propuesta e impacto, 8 %) se cubren en el capítulo 10, y el criterio 10 (informe técnico, 4 %) en todo el documento.

Cómo leer las fórmulas de este documento

Cada fórmula aparece siempre con la misma secuencia de cinco pasos, para que se entienda sin abrir el código:

Paso	Qué contiene
1. La idea en palabras	Lo que hace el método, explicado con un ejemplo de la vida diaria y sin símbolos (caja gris).
2. La fórmula	La expresión matemática completa, tal como la calcula el programa.
3. Qué es cada símbolo	Una tabla que traduce cada letra a lo que significa en el proyecto.
4. Paso a paso con nuestros datos	El cálculo resuelto a mano con cifras reales del proyecto; el resultado coincide con el que da el código (caja dorada).
5. Qué significa	Cómo se lee el resultado y a qué decisión de la campaña alimenta.

Además, cada capítulo incluye un recuadro de conexión con la materia:

> **Conexión con la materia.** Qué tema de la UCA se aplica y cómo se relaciona con el incidente crítico que el profesor planteó en el Problema Prototípico.

Tres palabras que se usan en todo el documento

Medido es lo que sale directo de una fuente oficial (INEGI, SECTUR, INAH, Sedetur). **Estimado** es lo que sale de un modelo; en las tablas del proyecto lleva el sufijo `_est` y siempre va acompañado de su error. **Hueco** es un dato que no existe: nunca lo rellenamos con ceros ni con promedios, lo declaramos y decimos qué no se puede afirmar por su culpa. Las cifras se recalcularon el 2 de octubre de 2026 con los datos del proyecto.

Cómo leer los fragmentos de código

Desde el capítulo 3, la mayoría de las secciones termina con un apartado **Cómo lo programamos**: el fragmento de Python que hace el cálculo, tal como corre en el notebook de la materia. No es pseudocódigo: es el código real, recortado a lo esencial. Para leerlo no hace falta saber programar; basta seguir los comentarios en gris (lo que va después de `#`), que dicen en palabras qué hace cada renglón. Debajo de cada fragmento se indica en qué notebook está completo y de qué módulo del proyecto sale.

Cuatro convenciones se repiten. `pd` es pandas, la librería de tablas, y `np` es NumPy, la de cálculo numérico; `F` agrupa las funciones de Spark. Una tabla de pandas se llama DataFrame y una de sus columnas se escribe `tabla.columna`. Las palabras de las tablas están en español y sin acentos (`visitantes`, `anio`, `es_de_los_5`). Y cada notebook cierra cada paso con `coincide(...)`, que compara la cifra recién calculada con la que guardó el proyecto y detiene el notebook si no son iguales: así, lo que se ve en este documento, en los notebooks y en la página es la misma cifra.

Cómo está organizado el código

El proyecto es un paquete de Python llamado `torre`, con una carpeta por módulo y un archivo por tarea. Cada archivo empieza con el mismo encabezado: qué hace, por qué así (la decisión y lo que se descartó), de qué datos sale y a qué decisión de la campaña alimenta. Los siete notebooks de la entrega están en `documento/uca`, uno por materia:

```
backend/torre/
  base/        descarga (ingesta_*), limpieza de Bronze a Silver (silver_*), manifiesto,
               almacén DuckDB y diccionario de datos
  radar/       panel mensual, índice de presión, criterios, clasificador, Markov y Ward
  pronostico/  series, forma del año, cinco modelos, rango conformal, elección,
               Poisson y Monte Carlo, calendario de la página
  envivo/      señales semanales (Isolation Forest), motor de reglas y streaming
  campana/     presupuesto (PuLP), minería de texto, negocios del DENUE, marca,
               fotos comprobadas, idiomas, tarifas y Firebase
  api/         datos de la página (pagina.js) y servidor FastAPI
  documento/   entrega, figuras, PDF y los 7 notebooks por materia (uca/)
frontend/      la página: index.html, app.js, cotizar.js, idiomas, fotos y video
tests/         316 pruebas automáticas (pytest)
```

Estructura del repositorio de trabajo

Los datos viven fuera del código, en tres capas (capítulo 3.1). Cada capa tiene su carpeta y cada fuente su subcarpeta; en Bronze, además, una carpeta por fecha de descarga:

```
datos/
  bronze/   MANIFIESTO.csv · siturq/2026-09-28/ · denue/2026-09-28/ · huracanes/ · smn/ ...
  silver/   siturq/ · inah/ · denue/cve_ent=23/ · clima_diario/anio=2024/ · huracanes/ ...
  gold/     radar_*.parquet · pronostico_*.parquet · presupuesto_*.parquet ·
            envivo_*.parquet · campana.json · torre.duckdb
```

Las tres capas de datos

En qué orden se corre el proyecto

Cada paso lee lo que dejó el anterior, así que el proyecto se reconstruye siempre en el mismo orden: descargar (Bronze), limpiar (Silver), calcular los modelos (Gold), armar la campaña y, al final, generar la página y la entrega. Cada comando es un módulo de `torre` que se corre desde la carpeta `backend`:

```
python -m torre.base.ingesta_abiertas         # 1. descargar las fuentes abiertas (Bronze)
python -m torre.base.silver_huracanes          # 2. limpiar cada fuente (Silver), una por una
python -m torre.base.almacen                   #    almacén DuckDB y modelo estrella
python -m torre.radar.prediccion               # 3. Radar: índice, estados y clasificador (Gold)
python -m torre.pronostico.modelos             # 4. Pronóstico: origen móvil de los 5 modelos
python -m torre.pronostico.escenarios          #    Poisson, Monte Carlo y sensibilidad
python -m torre.campana.presupuesto            # 5. reparto del presupuesto (PuLP)
python -m torre.envivo.torre                   # 6. las 239 semanas en Structured Streaming
python -m torre.campana.marca                  # 7. personas, mensajes e indicadores
python -m torre.api.datos_pagina               # 8. datos de la página (pagina.js)
python -m torre.documento.entregable           # 9. la entrega: ejecuta los 7 notebooks
python -m pytest                               #    las 316 pruebas automáticas
```

Orden de reconstrucción del proyecto

1. El problema y nuestra respuesta

1.1 La pregunta que nos pide responder el Problema Prototípico

> ¿Cómo diseñar una campaña publicitaria basada en Ciencia de Datos que permita promover estratégicamente un destino turístico del estado seleccionado, atraer segmentos de visitantes de manera responsable y contribuir a una distribución más equilibrada de los flujos turísticos, aprovechando la capacidad disponible y reduciendo los impactos económicos, sociales y ambientales asociados con la saturación turística? (UNRC, 2026)

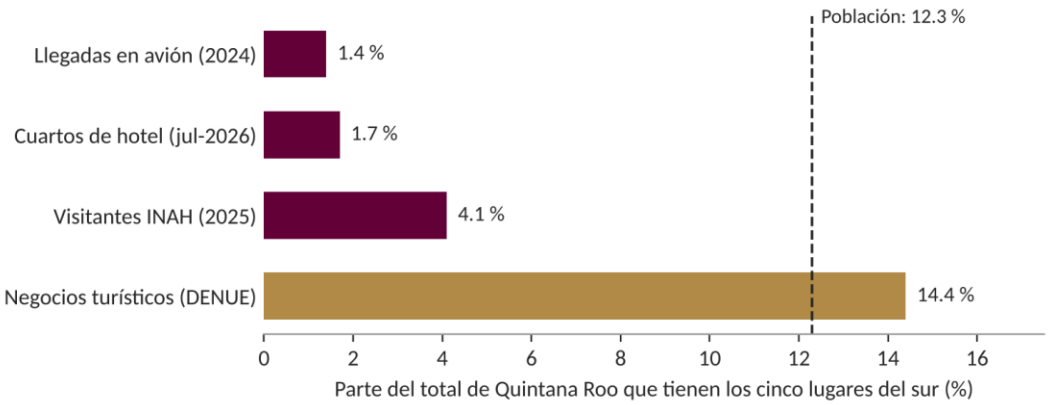
Leímos la pregunta como cuatro exigencias, y cada parte del sistema responde a una:

Exigencia de la pregunta	Qué tenemos que saber	Quién lo responde
Promover estratégicamente	Dónde hay espacio y cuándo conviene ir	Radar y Pronóstico
De manera responsable	No llevar gente a donde ya está lleno ni con mal clima	Reglas del presupuesto y Torre en vivo
Distribución más equilibrada	Cuánto está concentrado el turismo y hacia dónde moverlo	Planteamiento y campaña
Aprovechar la capacidad disponible	Cuánto puede recibir cada lugar sin rebasarse	Capacidad probada, Monte Carlo y optimización

1.2 El diagnóstico, con datos

Elegimos **Quintana Roo** porque concentra el contraste más fuerte del país dentro de un mismo estado. El norte (Cancún, la Riviera Maya y Tulum) recibe casi todo: el aeropuerto de Cancún concentra el 92.5 % de los pasajeros de avión del estado. En 2026 la franja de Tulum a Xcalak tuvo sargazo récord (56 de 140 playas en rojo en agosto) y la zona arqueológica de Tulum perdió el 31.3 % de sus visitas entre enero y julio.

El sur, en cambio, tiene espacio: los cinco lugares de la campaña tienen el **12.3 % de la población** del estado, pero reciben solo el **1.4 % de los pasajeros de avión**.



Parte del total del estado que tienen los cinco lugares del sur. La línea punteada marca su parte de la población: los negocios están donde está la gente, pero los visitantes no.

1.3 Nuestra respuesta

Los cinco lugares que promueve la campaña se eligieron con una tabla de criterios (capítulo 4):

1. **Chetumal**, la capital del estado.
2. **La Bahía de Calderitas y la zona arqueológica de Oxtankah.**
3. **La Ruta arqueológica del sur**: Kohunlich, Dzibanché e Ichkabal.
4. **Maya Ka'an con Kantemó.**
5. **La Laguna Milagros con Xul-Ha.**

Cancún, la Riviera Maya y Tulum aparecen solo como **referencia etiquetada**: sirven para medir desde dónde se redistribuye, pero la campaña no los promueve.

La campaña se llama **"El sur tiene espacio"** y tiene tres reglas que la vuelven responsable: nunca anuncia un lugar en su temporada alta; se pausa sola con mal clima o si llega más gente de la esperada; y a quien iba al norte le ofrece el lugar del sur que tenga más espacio.

1.4 Tres módulos que no se enciman

Una regla de diseño fue que ninguna técnica contestara la pregunta de otro módulo, para que nunca hubiera dos respuestas distintas a lo mismo.

Módulo	Pregunta	Horizonte	Técnicas principales
Radar	¿Dónde hay presión y dónde hay espacio?	Hoy (semanas y meses)	Índice de presión, agrupamiento, clasificador, cadena de Markov
Pronóstico	¿Cuándo conviene ir y cuánto invertir?	1 a 12 meses	Forma del año, cinco modelos de series de tiempo, intervalo conformal, Poisson, Monte Carlo, optimización
Torre en vivo	¿Qué hace la campaña esta semana?	Semana a semana	Spark Structured Streaming, Isolation Forest, motor de reglas
Campaña	¿A quién, con qué mensaje y por qué canal?	Todo el plan	Minería de texto, buyer persona, indicadores

El Radar predice un **estado** (tranquilo, concurrido o saturado) a un mes; el Pronóstico predice un **nivel** (cuántos visitantes) de 1 a 12 meses. La cadena de Markov trabaja en semanas y el Monte Carlo en meses. La Torre no entrena modelos propios: observa la semana real y ejecuta las reglas que el presupuesto dejó previstas.

1.5 Las siete preguntas secundarias y dónde se responden

Pregunta del Problema Prototípico	Se responde en
1. Patrones temporales, espaciales y de comportamiento de la concentración	Cap. 4 (HHI), 5 (forma del año, reseñas) y 9
2. Variables relacionadas con la saturación	Cap. 5 (índice de presión) y 6 (clasificador)
3. Demanda futura con incertidumbre	Cap. 6 (pronóstico) y 7 (Monte Carlo)
4. Escenarios de redistribución sin perder actividad económica	Cap. 8 (presupuesto)
5. Características para recomendar destinos alternativos	Cap. 5 (minería de texto) y 9 (buyer persona)
6. Impacto en comunidades y destinos saturados	Cap. 4 (presión por habitante) y 10 (Torre)
7. Cómo evaluar si la propuesta es viable, sustentable y efectiva	Cap. 8 (escenarios) y 9 (indicadores)

1.6 Cómo trabajamos

Trabajamos en doce fases (de la 0 a la 11), una a la vez: plan y entorno; descarga de 14 fuentes oficiales; limpieza y almacén; elección de los cinco lugares; Radar; Pronóstico; presupuesto; Torre en vivo; campaña; servidor; página final; y cierre. El paquete de código tiene 59 archivos en 7 carpetas, con una regla: **una pieza, un archivo**. Cada archivo hace una sola cosa y empieza con un encabezado que dice qué hace, por qué así, de qué datos sale y a qué decisión alimenta. Así cada pieza se puede probar, explicar y reemplazar sin romper las demás.

Para programar usamos un asistente de programación con inteligencia artificial (Claude Code) como herramienta. Las reglas, las decisiones y la revisión fueron del equipo, y están documentadas en más de 40 registros de decisión, cada uno con las opciones descartadas y su evidencia. Hay 316 pruebas automáticas: unas revisan la forma de los datos (sin duplicados, columnas completas) y otras revisan cifras conocidas (por ejemplo, que Tulum haya tenido 1,031,443 visitantes en 2025). Todo lo aleatorio usa semilla fija, así que volver a correr el proyecto da exactamente las mismas cifras.

2. Los datos: de dónde sale cada cifra

2.1 La regla: solo fuentes oficiales, descargadas con código

Todas las fuentes se descargaron con programas, nunca a mano, y se guardaron sin modificarse en `datos/bronze/<fuente>/<fecha>/`. Cada archivo quedó registrado en un manifiesto con su huella SHA-256, su tamaño, su dirección de origen, la fecha de descarga y su número de registros. El resultado de la fase de descarga fue **353 archivos, 8,134,802 registros y 824 MB**.

No usamos TripAdvisor ni Google Maps, porque sus términos de uso prohíben extraer sus datos; tampoco el semáforo estatal de sargazo, porque solo se publica en Facebook, cuyos términos también lo prohíben. Esta decisión responde directamente al incidente ético de Minería de Datos.

2.2 Las fuentes

Clave	Fuente	Qué trae	Registros	Para qué se usa
D1	SITUR-Q (Sedetur Quintana Roo)	Ocupación, cuartos, Tren Maya, cruceristas, cruces desde Belice, llegadas por avión	6,607	Radar del sur, pronóstico de Chetumal
D2	DataTur, ocupación hotelera (SECTUR)	66 centros del país; 7 de Quintana Roo con 239 semanas (ene-2022 a jul-2026)	166 archivos	Radar del norte, Markov, Torre
D3	DataTur, llegadas por nacionalidad	Extranjeros por aeropuerto, país y sexo, 2012–2026	521,363	Buyer persona
D4	DataTur: INAH, AFAC, cruceros y Compendio	Visitantes por zona arqueológica 2016–2026, pasajeros, cruceros, cuartos por categoría	390,228	Base del pronóstico del sur
D5	Rest-Mex 2025	Reseñas de turistas con calificación de 1 a 5	208,051	Minería de texto
D6	DENUE (INEGI), los 32 estados	Negocios con giro, tamaño y coordenadas	6,138,075	Oferta turística
D7	Censo 2020, ITER (INEGI)	Población de 2,243 localidades de Quintana Roo	2,243	Presión por habitante
D8	Open-Meteo (reanálisis ERA5)	Clima diario desde 1950 y horario desde 2019 en 8 puntos	767,016	Clima de los modelos y anomalías
D9	HURDAT2 (NOAA)	Trayectoria de todas las tormentas del Atlántico, 1851–2025	55,524	Probabilidad de tormenta
D10	FRED (Reserva Federal de St. Louis)	Pesos por dólar diario desde 1993	8,575	Sensibilidad y costos
D11–D15	ENDUTIH 2025, polígonos municipales, benchmarks de anuncios, fotos de Wikimedia Commons	Uso de internet, geografía, costo por clic, imágenes con licencia libre	—	Campaña y página

Clave	Fuente	Qué trae	Registros	Para qué se usa
D18	Normales climatológicas 1991–2020 (SMN-CONAGUA)	Lluvia y temperatura máxima normales de 3 estaciones: Chetumal, Nicolás Bravo y Cancún	36	Clima que ve el viajero

Tres hallazgos de esta etapa marcaron todo el proyecto: SITUR-Q **no publica ocupación hotelera de 2025–2026** para ningún destino (viene en ceros); las llegadas por avión de 2025 vienen en cero en los cuatro aeropuertos; y DataTur corrigió 2,804 cifras entre una publicación y otra (conservamos siempre la más reciente).

Cómo lo programamos: descargar con código

Ningún archivo se bajó a mano. Cada fuente tiene una función que arma la dirección, descarga, guarda el archivo en Bronze con la fecha del día y lo anota en el manifiesto con `registrar`. El clima del reanálisis ERA5 se pide a la API pública de Open-Meteo por punto y por década, en partes pequeñas para respetar su límite gratuito, con una pausa de 3 segundos entre peticiones:

```
def clima():
    tramos = [("diario", f"{d}-01-01", f"{d + 9}-12-31",
               "daily=temperature_2m_max,precipitation_sum,wind_speed_10m_max", f"{d}s")
              for d in range(1950, 2027, 10)]                    # una década por archivo
    for punto, (lat, lon) in PUNTOS_CLIMA.items():               # 8 puntos de Quintana Roo
        for tipo, inicio, fin, variables, etiqueta in tramos:
            url = (f"https://archive-api.open-meteo.com/v1/archive?latitude={lat}"
                   f"&longitude={lon}&start_date={inicio}&end_date={fin}&{variables}")
            ruta = BRONZE / "clima" / HOY / tipo / f"{punto}_{etiqueta}.json"
            datos = json.loads(_bajar_con_espera(url, ruta))     # reintenta si falla
            registrar("D8 Open-Meteo", ruta, url, filas=len(datos["daily"]["time"]))
            time.sleep(3)                                        # pausa entre peticiones
```

torre.base.ingesta_abiertas.clima

Las normales climatológicas del Servicio Meteorológico Nacional son un archivo de texto por estación. Si un archivo no trae el bloque de precipitación, el proceso se detiene en lugar de seguir con un dato a medias (regla de oro 5):

```
ESTACIONES_SMN = {"chetumal": 23032, "kohunlich": 23016, "cancun": 23155}

def normales_smn():
    for punto, estacion in ESTACIONES_SMN.items():
        url = ("https://smn.conagua.gob.mx/tools/RESOURCES/Normales_Climatologicas/"
               f"Normales9120/qroo/nor9120_{estacion}.txt")
        ruta = BRONZE / "smn" / HOY / f"nor9120_{estacion}.txt"
        datos = _bajar(url, ruta).decode("utf-8", errors="replace")
        if "PRECIPITACI" not in datos:
            raise ValueError(f"{url}: no trae la normal de precipitación")
        registrar("D18 SMN normales 1991-2020", ruta, url, filas=12)
```

torre.base.ingesta_abiertas.normales_smn

Cómo lo programamos: la lluvia, del día al mes

La lluvia de un mes es la **suma** de la lluvia de sus días, y la lluvia normal de un mes es el **promedio** de esas sumas entre años. Un detalle técnico importa: en Silver el año viene como categoría, porque la

tabla está partida por año. Si no se convierte a número antes de agrupar, pandas crea combinaciones vacías de punto y año que cuentan como lluvia cero, y la lluvia anual de Chetumal sale en 483 mm en lugar de 1,239. Del archivo del SMN se leen dos bloques, *PRECIPITACIÓN* y *TEMPERATURA MÁXIMA*, cada uno con su renglón *NORMAL*:

```
dia = pd.read_parquet("datos/silver/clima_diario")
dia["anio"], dia["punto"] = dia.anio.astype(int), dia.punto.astype(str)   # categoría → número
d30 = dia[dia.anio.between(1991, 2020)]                       # el periodo de las normales
mensual = d30.groupby(["punto", "anio", "mes"]).lluvia_mm.sum()          # lluvia de cada mes
normal_era5 = mensual.groupby(["punto", "mes"]).mean()                   # Chetumal: 1,239 mm

lineas = [l.strip() for l in texto_smn.splitlines()]
i = next(k for k, l in enumerate(lineas) if l.startswith("PRECIPITACI"))
normal = next(l for l in lineas[i:] if l.startswith("NORMAL"))
lluvia_smn = [float(x) for x in normal.split()[1:13]]                    # Chetumal: 1,355.5 mm
```

Notebook 2, sección 6 · torre.base.silver_clima.normales_smn

Comparadas en 1991–2020, las dos fuentes coinciden en la forma de la lluvia (correlación de 0.96 a 0.99 en Chetumal y Kohunlich; ERA5 da entre 3 % y 9 % menos al año), pero ERA5 marca de 2 a 3 °C menos de temperatura máxima. Por eso el viajero ve la normal de la estación del SMN y los modelos usan ERA5, que tiene la lluvia de cada mes de cada año sin huecos.

2.3 Lo que no existe (huecos declarados)

Hueco	Qué no se puede afirmar por su culpa
Ocupación hotelera del sur 2025–2026	Cuántos cuartos se llenan hoy en Chetumal; por eso el sur se mide con llegadas reales
Estado de origen, edad e ingreso del visitante nacional	Una persona nacional más detallada
Conversión de Facebook para turismo	El rendimiento exacto de Facebook; se prueba con tres valores
Tormentas de 2026	Si hubo tormentas este año; la Torre dice “sin dato”
Reseñas de los cinco lugares	Qué opinan los turistas del sur; las reseñas son de destinos llenos
Derrama económica por lugar	Cuánto dinero deja la campaña; la de SITUR-Q no trae unidad y termina en 2024

3. Grandes volúmenes de datos

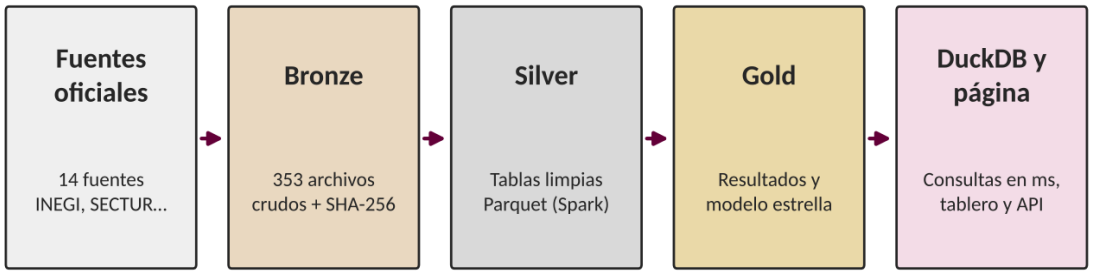
Materia: Herramientas para el Almacenamiento de Grandes Volúmenes de Datos · **Profesor:** Víctor Gabriel Cruz Arrona · **Criterio 2 de la rúbrica (12 %):** integra diversas fuentes; estructura, documenta, transforma y almacena considerando calidad y arquitectura.

Conexión con la materia. El incidente crítico de esta UCA se titula *"Cuando los datos del turismo no caben en una sola computadora"* y plantea cuatro problemas que nos tocaron de verdad: archivos con millones de registros en formatos distintos, fuentes que no coinciden entre sí, la necesidad de datos que llegan de forma continua (baja latencia) y la pregunta ética de si conviene guardar todo solo porque se puede. Este capítulo responde a cada uno: arquitectura medallón (3.1), procesamiento distribuido con Spark (3.2), reglas de calidad (3.3), conciliación de fuentes (3.4), almacén con modelo estrella (3.5), streaming (3.6) y privacidad (3.7).

3.1 La arquitectura medallón: Bronze, Silver y Gold

La idea en palabras

Pensemos en una planta potabilizadora. El agua cruda (Bronze) se guarda tal como llega, para poder volver a ella. Después se filtra y se limpia (Silver). Al final se embotella lista para beber (Gold). Si alguien duda del agua embotellada, se puede revisar cada paso hasta la toma original.



Arquitectura de datos del proyecto, de la fuente oficial a la página.

Capa	Carpeta	Contenido y regla
Bronze	`datos/bronze/`	Los archivos tal como los publica la fuente (zip, Excel, JSON, CSV). Nunca se modifican; cada uno tiene su huella SHA-256.
Silver	`datos/silver/`	Una tabla limpia por fuente, en Parquet, con nombres en español, tipos correctos y banderas de calidad (`_flag`). La escribe Spark.
Gold	`datos/gold/`	Salidas de los modelos y el modelo estrella; es lo que usan la página y los documentos.

Por qué así. Si una cifra de Gold parece rara, se puede repetir el camino desde el archivo crudo, que no se tocó. La alternativa descartada fue limpiar mientras se descarga: Bronze dejaría de ser idéntico a la fuente y ya no habría contra qué comparar.

La huella SHA-256. Es la "huella digital" de un archivo: una cadena de 64 caracteres que se calcula con todo su contenido. Si cambia un solo byte, la huella cambia por completo. Una prueba automática vuelve a calcular las 353 huellas y falla si alguna no coincide con el manifiesto; así se demuestra que Bronze no se alteró.

Cómo lo programamos: comprobar que el crudo no cambió

Comprobar Bronze es volver a calcular la huella SHA-256 de cada archivo y compararla con la que se guardó en el manifiesto el día de la descarga. El archivo se lee por pedazos de 1 MB para no cargar en memoria archivos de cientos de megabytes, como los del DENUE:

```
import hashlib

def sha256(ruta):
    h = hashlib.sha256()
    with open(ruta, "rb") as f:
        for bloque in iter(lambda: f.read(1 << 20), b""):   # de 1 MB en 1 MB
            h.update(bloque)
    return h.hexdigest()                                    # 64 caracteres

manifiesto = pd.read_csv("datos/bronze/MANIFIESTO.csv")     # fuente, archivo, sha256, url
manifiesto["sha256_hoy"] = manifiesto.archivo.apply(sha256)
assert (manifiesto.sha256 == manifiesto.sha256_hoy).all()   # si cambió un byte, se detiene
```

Notebook 1, sección 1 · torre.base.manifiesto

3.2 Spark: qué es y qué hace aquí

> **La idea en palabras**
>
> Limpiar una lista de 6 millones de negocios es como calificar 6 millones de exámenes. Una persona sola tardaría mucho; doce personas, cada una con su montón, terminan mucho antes. Spark reparte la tabla en montones (particiones) y pone a trabajar a la vez a todos los núcleos de la computadora.

Las cinco ideas de Spark que usamos:

1. **Driver y ejecutores.** El *driver* planea el trabajo y los *ejecutores* lo hacen. Aquí Spark corre en modo local (`local[*]`) con los 12 núcleos de la computadora y 4 GB de memoria. Es el mismo programa que correría en un clúster; solo cambia la dirección del "maestro".
2. **DataFrame y particiones.** Un DataFrame de Spark es una tabla repartida en particiones, y cada partición la procesa un núcleo.
3. **Evaluación perezosa.** Las transformaciones (`withColumn`, `filter`) no se ejecutan al escribirlas: Spark arma un plan completo, lo optimiza y lo ejecuta solo cuando se pide una acción (`count`, `write`).
4. **Barajeo (*shuffle*).** Quitar duplicados obliga a juntar en el mismo núcleo todos los renglones con el mismo identificador. Es la parte cara del trabajo y por eso se hace una sola vez.

5. **Parquet particionado.** Spark guarda por columnas y comprime. Además separa la tabla en carpetas por estado (`cve_ent=23/` es Quintana Roo), así que leer un estado no obliga a leer los otros 31.

El trabajo más grande fue el DENUE nacional (6.1 millones de negocios):

```
crudo = (spark.read.option("header", True).option("encoding", "ISO-8859-1")
         .csv("temporal/*.csv"))                  # 33 CSV como UNA sola tabla
silver = (crudo.select(*COLUMNAS)                 # sin datos personales
          .dropDuplicates(["id"])                 # un negocio, una fila (barajeo)
          .withColumn("scian_3", F.substring("codigo_act", 1, 3))
          .withColumn("categoria_turistica", categoria)   # 721 alojamiento, 722 alimentos
          .withColumn("coordenadas_flag",
                      ~(lat.between(14, 33) & lon.between(-119, -86)) | lat.isNull())
          .withColumn("es_qroo", F.col("cve_ent") == "23"))
silver.write.mode("overwrite").partitionBy("cve_ent").parquet("datos/silver/denue")
```

Resultado: 0 duplicados, ninguna coordenada fuera de México, y la razón social, el teléfono y el correo nunca entran a Silver. Tarda cerca de minuto y medio. Del total nacional, el 14.3 % de los negocios son turísticos; en Quintana Roo, el 19.7 %.

Cómo lo programamos: de Bronze a Silver con Spark

La sesión de Spark se crea una sola vez con los parámetros de arriba. El caso que mejor muestra lo que hace Spark es el archivo de huracanes HURDAT2 (NOAA): alterna un renglón de **encabezado** por tormenta (`AL092024, NADINE, 18,`) con sus puntos de trayectoria cada 6 horas, y un punto no dice a qué tormenta pertenece. Con una **función de ventana**, cada renglón toma el último encabezado que apareció antes que él; después, una expresión regular saca la posición y el viento:

```
spark = (SparkSession.builder.appName("torre")
         .master("local[*]")                          # todos los núcleos de la computadora
         .config("spark.driver.memory", "4g")         # memoria del proceso principal
         .config("spark.sql.session.timeZone", "UTC")  # las fechas no cambian entre máquinas
         .getOrCreate())

crudo = (spark.read.text("datos/bronze/huracanes/hurdat2-1851-2025.txt")
              .withColumn("n", F.monotonically_increasing_id()))     # orden del archivo
es_cab = F.col("value").rlike(r"^[A-Z]{2}[0-9]{6},")                 # ¿es encabezado?
hasta_aqui = Window.orderBy("n").rowsBetween(Window.unboundedPreceding, 0)
d = (crudo
     .withColumn("id_cab", F.when(es_cab, F.trim(F.split("value", ",")[0])))
     .withColumn("id_tormenta", F.last("id_cab", ignorenulls=True).over(hasta_aqui))
     .filter(~es_cab)                                                 # solo trayectorias
     .withColumn("anio", F.substring("value", 1, 4).cast("int"))
     .withColumn("lat", F.regexp_extract("value", POS, 1).cast("double"))
     .withColumn("viento_kt", F.regexp_extract("value", POS, 5).cast("int")))
d.write.mode("overwrite").partitionBy("anio").parquet("datos/silver/huracanes")
```

Notebook 1, sección 3 · torre.base.entorno y torre.base.silver_huracanes

El resultado coincide con la Silver del proyecto: 55,524 puntos de 1,988 tormentas, las mismas distancias a Chetumal y los mismos 31 eventos que afectan al sur. Como la tabla queda partida por año, una consulta de 2024 abre una sola de las 175 carpetas; el plan de ejecución de Spark lo muestra como `PartitionFilters`.

Cómo lo programamos: 6.1 millones de negocios en segundos

Con la tabla del DENUE ya en Parquet, contar los negocios turísticos de cada estado es una agrupación. Spark solo lee las dos columnas que usa la consulta y reparte el conteo entre los núcleos; tarda unos 5 segundos. Quintana Roo queda en segundo lugar del país, con 19.7 % de negocios turísticos, después de Nayarit (20.5 %):

```
denue = spark.read.parquet("datos/silver/denue")          # 6,138,075 renglones
(denue.groupBy("entidad")
      .agg(F.count("*").alias("negocios"),
           F.sum(F.col("es_turistico").cast("int")).alias("turisticos"))
      .withColumn("pct_turistico", F.round(100 * F.col("turisticos") / F.col("negocios"), 1))
      .orderBy(F.desc("pct_turistico")))

denue.createOrReplaceTempView("denue")                    # la misma consulta, en SQL
spark.sql("""SELECT municipio, COUNT(*) AS negocios,
                    SUM(CAST(es_turistico AS INT)) AS turisticos
             FROM denue WHERE es_qroo GROUP BY municipio ORDER BY turisticos DESC""")
```

Notebook 1, sección 4

3.3 Reglas de limpieza: un cero no es lo mismo que un hueco

La regla más importante de Silver: **un cero físicamente imposible es un hueco; un cero posible se conserva.**

Caso	Por qué	Regla
Ocupación con 0 cuartos disponibles	No se pueden ocupar cuartos que no existen	Hueco
Los 4 aeropuertos en 0 todo 2025	Cancún no pudo recibir cero pasajeros: la fuente dejó de publicar	Hueco
Cruceristas en 0 en 2020	Real: los puertos cerraron por la pandemia	Se conserva
Zona arqueológica con 0 visitantes	Suele ser un cierre por obras	Se conserva y se marca
Feriado sin tipo de cambio	No hubo cotización ese día	Vacío; no se copia el día anterior

La ocupación se calcula sumando antes de dividir

> **La idea en palabras**
>
> Si en un mes había muchos cuartos y en otro pocos, no es justo que los dos pesen igual. Por eso primero se suman todos los cuartos ocupados del año, luego todos los disponibles, y al final se divide una vez. Así cada cuarto-noche cuenta exactamente una vez.

$$O = \frac{\text{cuartos-noche ocupados en todos los meses}}{\text{cuartos-noche disponibles en todos los meses}} \times 100$$

Símbolo	Qué significa
O	Ocupación del periodo, en porcentaje
cuarto-noche	Un cuarto durante una noche; un hotel de 10 cuartos tiene 300 cuartos-noche en un mes de 30 días

Paso a paso con nuestros datos: ocupación de Chetumal en 2024

Paso 1. Sumamos los cuartos-noche ocupados de los 12 meses: 458,696.

Paso 2. Sumamos los cuartos-noche disponibles de los 12 meses: 791,016.

Paso 3. Dividimos y multiplicamos por 100:

$$O = \frac{458{,}696}{791{,}016} \times 100 = 0.5799 \times 100 = 57.99 \approx 58.0\ \%$$

Qué significa: de cada 10 cuartos, 4 se quedaron vacíos. Es capacidad ociosa, justo lo que la campaña puede aprovechar.

Por qué no promediar porcentajes. Con números redondos, para que se vea el problema: un mes con 50,000 cuartos al 90 % (45,000 ocupados) y otro con 10,000 al 10 % (1,000 ocupados). El promedio simple da (90 + 10) ÷ 2 = 50 %, pero en realidad se ocuparon 46,000 de 60,000 cuartos: 76.7 %. El promedio simple le da al mes chico el mismo peso que al grande.

Un error real que lo demuestra. SITUR-Q publica tres renglones por mes para la ocupación (disponibles, ocupados y porcentaje). La primera versión de nuestro panel los sumaba y la "ocupación" llegó a 2,560,068. Ahora se recalcula ocupados ÷ disponibles, y una prueba automática lo vigila.

Cómo lo programamos: contar huecos y sumar antes de dividir

En Silver, un mes que la fuente no publicó queda con `hueco_flag = True` y valor nulo, nunca en cero. La primera consulta cuenta por separado los huecos y los ceros reales de cada indicador de SITUR-Q: la ocupación hotelera tiene 693 meses sin publicar (35.5 %) y ningún cero real. La segunda muestra la regla de la ocupación: se suman los cuartos y después se divide, para que un hotel chico no pese igual que uno grande:

```
s = spark.read.parquet("datos/silver/siturq")
(s.groupBy("indicador")
  .agg(F.count("*").alias("meses"),
       F.sum(F.col("hueco_flag").cast("int")).alias("huecos"),            # no publicado
       F.sum(F.when(F.col("valor") == 0, 1).otherwise(0)).alias("ceros"))  # cero real
  .orderBy(F.desc("huecos")))

# Ocupación del mes: suma de cuartos ocupados entre suma de cuartos disponibles
m = o.groupby(["lugar", "periodo"])[["cuartos_ocupados", "cuartos_disponibles"]].sum()
m["ocupacion_pct"] = 100 * m.cuartos_ocupados / m.cuartos_disponibles   # nunca promedio de %
```

Notebook 1, sección 5 · torre.base.silver_siturq y torre.radar.panel

3.4 Cuando dos fuentes cuentan lo mismo (conciliación)

El incidente de esta materia pregunta qué hacer cuando "un destino puede aparecer como altamente visitado en una fuente, mientras que otra muestra una menor actividad" (UNRC, 2026). Medimos cuánto difieren:

$$\Delta\% = \left(\frac{S}{D} - 1\right) \times 100$$

Símbolo	Qué significa
S	La cifra de SITUR-Q (fuente estatal)
D	La cifra de DataTur (fuente federal)
$\Delta\%$	Cuánto más (si es positivo) o menos (si es negativo) cuenta la fuente estatal

Paso a paso con nuestros datos: cruceristas en 2025

Cozumel. SITUR-Q cuenta 4,915,242 y DataTur 4,724,255. Primero dividimos: 4,915,242 ÷ 4,724,255 = 1.0404. Le restamos 1: 0.0404. Por 100: **+4.04 %**.

Mahahual. 2,641,695 ÷ 2,379,422 = 1.1102, menos 1 y por 100: **+11.02 %**. En Mahahual SITUR-Q cuenta siempre más: entre 11 % y 35 % según el año.

En la ocupación pasa lo mismo: DataTur marca menos que SITUR-Q (Cancún −1.6 puntos, Cozumel −14.5, Isla Mujeres −19.3), aunque las dos se mueven juntas (correlación entre 0.68 y 0.97). Nuestra regla tiene tres partes:

1. Cada lugar usa **una sola fuente en toda su historia**, para que sus cambios en el tiempo sean reales.
2. La diferencia **se declara** y no se "ajusta".
3. Se revisa en qué sentido va. Aquí el norte se ve menos lleno con DataTur, así que la conclusión "el sur tiene espacio" se sostiene con más fuerza: la diferencia es conservadora.

Cómo lo programamos: la conciliación año por año

Las dos fuentes de cruceristas se juntan por puerto y mes y se comparan **por año**, porque un mes suelto puede diferir mucho por un cambio de registro. En Mahahual, SITUR-Q cuenta entre 11 % (2025) y 35 % (2019) más que DataTur; en Cozumel, en los años sin pandemia, entre −0.4 % y 15 %:

```
conc = pd.read_parquet("datos/gold/reconciliacion_cruceros.parquet")
conc["anio"] = pd.to_datetime(conc.periodo).dt.year
anual = conc.groupby(["puerto", "anio"])[["pasajeros_datatur", "cruceristas_siturq"]].sum()
anual["diferencia_pct"] = (anual.cruceristas_siturq / anual.pasajeros_datatur - 1) * 100
```

Notebook 1, sección 5 · torre.base.silver_cruceros

3.5 El almacén DuckDB y el modelo estrella

DuckDB es una base de datos analítica que vive en un solo archivo (`datos/gold/torre.duckdb`) y lee los Parquet de Silver y Gold sin copiarlos. Responde en milisegundos sin levantar Spark. Tiene una

vista por cada tabla (14 de Silver y 39 de Gold) y un **modelo estrella** (Kimball y Ross, 2013): una tabla de hechos en el centro, con las medidas, rodeada de tablas de dimensiones con las descripciones.

Tabla	Tipo	Contenido
dim_lugar	Dimensión	15 lugares con su papel: 5 promovidos, 3 de referencia y 7 de comparación
dim_tiempo	Dimensión	Un renglón por mes, de 2012 a 2027
hechos_mes	Hechos	3,287 renglones: lugar × mes × variable, con su fuente

El **grano** (qué representa un renglón de hechos) es "una medida de un lugar en un mes". Un hueco no tiene renglón: nunca se guarda un cero inventado. Ejemplo de consulta:

```
SELECT anio, SUM(valor) FROM hechos_mes JOIN dim_tiempo USING (periodo)
WHERE lugar = 'Chetumal' AND variable = 'llegadas_extranjeros_avion'
GROUP BY anio;      -- 2025: 250 extranjeros (Cancún: 9,408,423)
```

El **diccionario de datos** se genera solo desde el almacén: 58 tablas, cada columna con su tipo, su porcentaje de vacíos, un ejemplo real y su significado. Si una columna no tiene significado escrito, el programa se detiene; así el diccionario no puede quedar desactualizado. Descartamos PostgreSQL porque exige instalar y levantar un servidor, y el proyecto debe correr sin internet en cualquier computadora.

Cómo lo programamos: las vistas y una consulta a la estrella

DuckDB lee los Parquet sin copiarlos: cada tabla de Silver y de Gold queda como una **vista**, y solo las tres tablas de la estrella se guardan dentro del archivo. La consulta cruza los hechos con sus dos dimensiones y responde en milisegundos: en 2025, las zonas arqueológicas de los 5 lugares recibieron 77,645 visitantes contra 1.64 millones del resto del estado (4.5 %):

```
import duckdb
db = duckdb.connect("datos/gold/torre.duckdb")
for t in ["inah", "siturq", "denue", "clima_diario"]:            # una vista por tabla
    db.execute(f"""CREATE VIEW silver_{t} AS SELECT * FROM
                   read_parquet('datos/silver/{t}/**/*.parquet', hive_partitioning = true)""")

db.sql("""
    SELECT CASE WHEN l.es_de_los_5 THEN 'los 5 del sur' ELSE 'resto' END AS grupo,
           t.anio, SUM(h.valor) AS visitantes_inah
    FROM hechos_mes h JOIN dim_lugar l USING (lugar) JOIN dim_tiempo t USING (periodo)
    WHERE h.variable = 'visitantes_inah' AND t.anio BETWEEN 2022 AND 2025
    GROUP BY ALL ORDER BY grupo, t.anio""").df()
```

Notebook 1, sección 6 · torre.base.almacen

3.6 Streaming: la Torre en vivo

> **La idea en palabras**
>
> Una banda transportadora: cada semana llega una caja con los datos de esa semana. La Torre abre una caja a la vez, en el orden en que llegaron, decide qué hacer con la campaña y pasa a la siguiente.

Así funciona con Spark Structured Streaming:

1. Cada una de las 239 semanas reales (enero de 2022 a julio de 2026) se escribe como un archivo JSON en una carpeta.
2. Spark escucha la carpeta con `readStream`. La opción `maxFilesPerTrigger=1` hace que cada micro-lote sea exactamente un archivo, es decir, una semana.
3. En cada micro-lote, `foreachBatch` le pasa la semana al motor de reglas, que decide qué anuncio se enciende o se pausa.
4. Un punto de control (*checkpoint*) registra qué archivos ya se procesaron: si el programa se cae, no procesa dos veces la misma semana.
5. Al final se comprueba que llegaron 239 lotes, en orden.

Esta es exactamente la arquitectura que recibiría datos reales cada semana: una carpeta a la que llega un archivo nuevo. Cambiar la fuente por una real no cambia el motor. En la página se presenta como "reproducción de datos históricos reales", porque GitHub Pages no puede transmitir en vivo.

La ventana deslizante de cuatro semanas

> **La idea en palabras**
>
> Una semana rara (un puente, un evento) puede engañar. Por eso la Torre mira el promedio de las últimas cuatro semanas. Cada semana entra la nueva y sale la más vieja: la ventana "se desliza".

$$\bar{o}_w = \frac{o_w + o_{w-1} + o_{w-2} + o_{w-3}}{4}$$

Símbolo	Qué significa
o_w	Ocupación de la semana w (la más reciente)
$o_{w-1}, o_{w-2}, o_{w-3}$	Ocupación de una, dos y tres semanas antes
$\bar{o}_w$	Promedio móvil que usa la Torre para decidir

Es la misma fórmula que en notación de sumatoria se escribe $\bar{o}_w = \frac{1}{4}\sum_{k=0}^{3} o_{w-k}$: "suma las cuatro semanas más recientes y divide entre cuatro". Spark la calcula con una función de ventana sobre cada centro turístico.

Cómo lo programamos: una semana por lote

La reproducción escribe cada semana de señales como un archivo JSON en una carpeta. Spark Structured Streaming «escucha» esa carpeta y procesa **un archivo por lote**; en cada lote, la función `lote` aplica las reglas del motor (capítulo 8.8). El punto de control (*checkpoint*) guarda qué archivos ya se procesaron, para que la Torre pueda reanudarse sin repetir una semana. Las 239 semanas llegaron en 239 lotes, en orden. Abajo, la ventana deslizante que calcula el promedio de cuatro semanas de la ocupación del norte:

```
q = (spark.readStream.schema(esquema)
       .option("maxFilesPerTrigger", 1)                # una semana por lote
       .json("datos/gold/envivo_entrada")
       .writeStream.foreachBatch(lote)                 # lote(): decide esa semana
```

```
       .option("checkpointLocation", "datos/gold/envivo_checkpoint")   # para reanudar
       .trigger(availableNow=True)                      # procesa lo que hay y termina
       .start())
q.awaitTermination()

w = Window.partitionBy("centro").orderBy("semana").rowsBetween(-3, 0)  # semana y 3 antes
norte = norte.withColumn("ocupacion_4sem", F.avg("ocupacion_pct").over(w))
```

Notebook 1, sección 7 · torre.envivo.torre y torre.envivo.senales

3.7 Privacidad

No guardamos ningún dato personal. El DENUE entra sin razón social, teléfono ni correo; las reseñas no tienen identificadores; las nacionalidades y los visitantes vienen agregados por mes. Ninguna tabla permite identificar a una persona. Con esto respondemos a la pregunta del incidente: *"¿Qué riesgos pueden surgir al almacenar información que permita identificar patrones de movilidad o comportamiento de los turistas?"*: guardamos solo lo agregado y lo necesario para una decisión.

Cómo lo programamos: qué deja guardar la base de datos de la página

La página solo guarda dos cosas en Firebase: los corazones de las fotos y las solicitudes de cotización. Las reglas de Firestore deciden qué se puede escribir: una solicitud solo se puede **crear** (nunca leer, editar ni borrar desde la página), con los campos y tamaños revisados y el consentimiento en verdadero. Solo el equipo la ve, desde la consola de Firebase:

```
match /solicitudes/{folio} {
  allow read, update, delete: if false;                    // nadie las lee desde la página
  allow create: if request.auth != null                    // cuenta anónima del visitante
    && folio.matches('TDC-[0-9]{6}-[A-Z0-9]{5}')           // el folio es el documento
    && request.resource.data.consentimiento == true        // aceptó el aviso de privacidad
    && request.resource.data.estado == 'nueva'
    && request.resource.data.nombre.size() <= 80
    && request.resource.data.medio in ['whatsapp', 'correo', 'llamada']
    && request.resource.data.itinerario.size() <= 8000;    // el plan y, si lo pidió, el vuelo
}
```

firebase/firestore.rules (generadas por torre.campana.nube)

3.8 Las columnas que inferimos: de dónde sale cada una

En los datos del proyecto hay tres tipos de columnas. Las **medidas** vienen tal cual de una fuente oficial (los visitantes del INAH, los cuartos ocupados de DataTur). Las **derivadas** las calculamos con una regla fija a partir de las medidas (el índice de presión, la bandera de mes sin visitantes). Y las **estimadas** o **inferidas** las produce un modelo, y por eso pueden equivocarse: llevan el sufijo `_est` (regla de oro 3), para que quien las lea sepa que no son un dato, y cada una tiene su error medido.

Con DuckDB leímos el esquema de todos los archivos de Gold sin cargar los datos: hay **24 columnas inferidas en 9 tablas**. Esta es la lista, con lo que es cada una, de dónde sale y cómo se validó:

Columna inferida	Tabla de Gold	Qué es	De qué sale	Cómo se validó
estado_mes_siguiente_est y prob_*	radar_prediccion	Estado del mes siguiente y su probabilidad	Índice de presión (Tren Maya, SITUR-Q, INAH, DataTur, Censo) → Random Forest + año	130 de 156 en la prueba; Brier 0.214

Columna inferida	Tabla de Gold	Qué es	De qué sale	Cómo se validó
			anterior	
esperado_est, minimo_90_est, maximo_90_est	pronostico_mes	Visitantes esperados y su rango del 90 %	Serie mensual (INAH, SITUR-Q, DataTur) → modelo elegido + cuantil conformal	MAPE de 11 a 22 %; cobertura medida de 80 a 94 %
ocupacion_est	pronostico_calendario	Ocupación hotelera esperada del norte	DataTur → modelo elegido	MAPE 3.8 % en Cancún
malo_p10_est, probable_p50_est, bueno_p90_est	pronostico_escenarios	Escenarios malo, probable y bueno	Pronóstico × errores reales de un año (Monte Carlo, 10,000 futuros)	Desde 5,000 futuros ya no cambia
conversiones_est	presupuesto_plan	Visitantes que trae cada peso	Pesos (PuLP) × conversión ÷ (costo por clic × tipo de cambio)	Parámetros de WordStream, LocaliQ y FRED
campana_p10_est, campana_p50_est, campana_p90_est	presupuesto_montecarlo	Rango de lo que trae la campaña	10,000 sorteos de los supuestos dentro del rango de las fuentes	82 % de probabilidad de traer menos que el plan
semanas_al_anio_est	radar_markov_largo_plazo	Semanas saturadas al año (52 · π)	Distribución estacionaria de la cadena de Markov	π = 9.8 % contra 10 % observado

Cadena 1: de la hoja del INAH al pronóstico con su rango

Seguimos una cifra de punta a punta: los visitantes de la Ruta de las pirámides. Cada captura es un DataFrame real del proyecto; las columnas en rosa son las que **agregamos o inferimos** en ese paso.

```
In [1]:
z = zipfile.ZipFile("datos/bronze/datatur/2026-09-28/inah/BdINAH.zip")
crudo = pd.read_excel(z.open("Bd_INAH.xlsx"))      # Bronze: el archivo tal cual
crudo[crudo["Nombre"].str.contains("Kohunlich")].tail(5)
```

Out[1]:

	Clasificación INAH	Estado	Abreviatura	Nombre	Año	Mes	id_mes	Unidad	Visitantes	Tipo	Fecha
70356	Zona Arqueológica	Quintana Roo	Q. Roo	Z.A. de Kohunlich	2026	Mayo	5	Personas	169	Extranjero	2026-05-01
70638	Zona Arqueológica	Quintana Roo	Q. Roo	Z.A. de Kohunlich	2026	Junio	6	Personas	547	Nacional	2026-06-01
70920	Zona Arqueológica	Quintana Roo	Q. Roo	Z.A. de Kohunlich	2026	Junio	6	Personas	106	Extranjero	2026-06-01
71202	Zona Arqueológica	Quintana Roo	Q. Roo	Z.A. de Kohunlich	2026	Julio	7	Personas	1258	Nacional	2026-07-01
71484	Zona Arqueológica	Quintana Roo	Q. Roo	Z.A. de Kohunlich	2026	Julio	7	Personas	392	Extranjero	2026-07-01

Bronze: el Excel del INAH sin tocar (nombres de columna originales; el mes viene en texto y en número). Nada se infiere todavía.

Paso 1 · Bronze: el Excel del INAH dentro de BdINAH.zip, tal como se descargó.

En Silver cambiamos los nombres a español y snake_case, damos a cada columna su tipo (fecha, entero) y agregamos dos columnas derivadas: la región de la campaña a la que pertenece la zona y la bandera `sin_visitantes_flag`, que distingue un cero real de un mes sin dato.

```
In [2]: s = pd.read_parquet("datos/silver/inah")            # Silver: limpio y con tipos
        s[s.nombre.str.contains("Kohunlich|Dzibanch|Ichkabal") & (s.periodo >= "2026-05")]
```

Out[2]:

	nombre	periodo	tipo_visitante	visitantes	region_campana	sin_visitantes_flag
70166	Z.A de Ichkabal	2026-05-01	extranjero	762	Ruta arqueológica del sur	False
70167	Z.A de Ichkabal	2026-05-01	nacional	1409	Ruta arqueológica del sur	False
70168	Z.A de Ichkabal	2026-06-01	extranjero	652	Ruta arqueológica del sur	False
70169	Z.A de Ichkabal	2026-06-01	nacional	1362	Ruta arqueológica del sur	False
70170	Z.A de Ichkabal	2026-07-01	extranjero	995	Ruta arqueológica del sur	False
70171	Z.A de Ichkabal	2026-07-01	nacional	1592	Ruta arqueológica del sur	False

Silver: tipos correctos, una fila por zona, mes y tipo de visitante. Se AGREGAN dos columnas derivadas: la región de la campaña y la bandera de mes sin visitantes (cero ≠ hueco).

Paso 2 · Silver: tabla limpia y con tipos (torre.base.silver_inah, con Spark).

En Gold juntamos las tres zonas y los dos tipos de visitante en una serie mensual. `entrena_flag` marca los meses que sirven para entrenar: sin cierre, sin mes parcial y fuera de la pandemia.

```
In [3]: t = pd.read_parquet("datos/gold/pronostico_series.parquet")   # la serie mensual
        t[t.lugar == "Ruta arqueológica del sur"].tail(5)
```

Out[3]:

	serie	periodo	valor	zonas_abiertas	entrena_flag	motivo_hueco
449	Ruta arqueológica del sur · …	2026-03-01	4,766.0	3.000	True	None
450	Ruta arqueológica del sur · …	2026-04-01	5,961.0	3.000	True	None
451	Ruta arqueológica del sur · …	2026-05-01	3,842.0	3.000	True	None
452	Ruta arqueológica del sur · …	2026-06-01	3,126.0	3.000	True	None
453	Ruta arqueológica del sur · …	2026-07-01	5,057.0	3.000	True	None

Gold: valor = suma de las 3 zonas (Kohunlich + Dzibanché + Ichkabal) y nacionales + extranjeros. entrena_flag dice si el mes sirve para entrenar (sin cierre, sin mes parcial, sin pandemia).

Paso 3 · Gold: la serie mensual de la Ruta (torre.pronostico.series).

Aquí aparecen las primeras columnas inferidas: el pronóstico del modelo elegido (`esperado_est`) y su rango del 90 % (`minimo_90_est` y `maximo_90_est`), sacado de los errores reales que el modelo cometió al probarlo mes a mes.

```
In [4]: p = pd.read_parquet("datos/gold/pronostico_mes.parquet")   # lo que INFERIMOS
        p[p.lugar == "Ruta arqueológica del sur"].head(6)
```

Out[4]:

	modelo	periodo	horizonte	esperado_est	minimo_90_est	maximo_90_est
48	Regresión con clima	2026-08-01	1	4,857.8	3,207.8	7,356.4
49	Regresión con clima	2026-09-01	2	2,564.1	1,693.2	3,883.0
50	Regresión con clima	2026-10-01	3	3,672.2	2,424.9	5,561.1
51	Regresión con clima	2026-11-01	4	5,520.6	3,508.8	8,686.0
52	Regresión con clima	2026-12-01	5	7,345.3	4,668.5	11,556.9
53	Regresión con clima	2027-01-01	6	8,992.9	5,715.7	14,149.2

Columnas inferidas (_est): el pronóstico del modelo elegido (regresión con clima) y su rango del 90 % (cuantil conformal de los errores reales del origen móvil). MAPE 21.6 %, cobertura 80.3 %.

Paso 4 · Gold: el pronóstico con su rango (torre.pronostico.seleccion).

Del pronóstico salen los escenarios: el Monte Carlo multiplica el pronóstico por un año de errores reales del pasado, 10,000 veces, y guarda los percentiles 10, 50 y 90 y la fracción de futuros que rebasan la capacidad probada.

```
In [5]:
e = pd.read_parquet("datos/gold/pronostico_escenarios.parquet")   # Monte Carlo
e[(e.lugar == "Ruta arqueológica del sur") & (e.golpe_tormenta_supuesto == 0)].head(6)
```

Out[5]:

	periodo	malo_p10_est	probable_p50_est	bueno_p90_est	prob_tormenta	capacidad_probada	riesgo_rebasar_capacidad
84	2026-08-01	3,230.5	4,903.7	6,359.6	0.139	10,465.0	0.000
85	2026-09-01	1,724.3	2,575.1	3,298.9	0.125	10,465.0	0.000
86	2026-10-01	2,438.5	3,688.0	4,724.5	0.095	10,465.0	0.000
87	2026-11-01	3,654.4	5,488.2	6,916.2	0.049	10,465.0	0.000
88	2026-12-01	4,775.6	7,376.7	9,277.0	0.000	10,465.0	0.028
89	2027-01-01	5,993.1	9,031.3	11,384.3	0.000	10,465.0	0.303

Inferidas con 10,000 futuros: pronóstico × e^(error real de un año del pasado). prob_tormenta sale de Poisson; capacidad_probada es el mes más alto ya medido.

Paso 5 · Gold: los escenarios malo, probable y bueno (torre.pronostico.escenarios).

Cadena 2: del panel de medidas al estado del mes siguiente

```
In [6]:
pm = pd.read_parquet("datos/gold/radar_panel_mensual.parquet")   # medidas por mes
pm[pm.lugar == "Chetumal"].tail(5)
```

Out[6]:

	lugar	periodo	llegadas_tren	cruces_belice	cuartos	cuartos_noche_ocupados	poblacion
215	Chetumal	2026-03-01	3,638.0	46,503.0	2,114.0	NaN	169,028.0
216	Chetumal	2026-04-01	4,264.0	44,917.0	2,114.0	NaN	169,028.0
217	Chetumal	2026-05-01	4,120.0	49,526.0	2,114.0	NaN	169,028.0
218	Chetumal	2026-06-01	3,630.0	49,097.0	2,114.0	NaN	169,028.0
219	Chetumal	2026-07-01	3,843.0	NaN	2,114.0	NaN	169,028.0

El panel junta 6 fuentes de Silver por lugar y mes: Tren Maya, Belice (SITUR-Q), INAH, hotelería y población del Censo 2020 por localidad.

Paso 1 · El panel mensual: seis fuentes de Silver por lugar y mes (torre.radar.panel).

Cada medida se pasa a la misma escala, de 0 a 1, y se promedia: es el índice de presión (`ipt_comparable`). Con sus percentiles 50 y 90 se asigna el estado.

```
In [7]:
ic = pd.read_parquet("datos/gold/radar_indice_comparable.parquet")
ic[ic.lugar == "Chetumal"].tail(5)
```

Out[7]:

	lugar	periodo	ipt_comparable	estado	medidas	corte_p50	corte_p90
215	Chetumal	2026-03-01	0.023	tranquilo	llegadas_tren/1000 hab, llegadas/cuarto	0.186	0.756
216	Chetumal	2026-04-01	0.027	tranquilo	llegadas_tren/1000 hab, llegadas/cuarto	0.186	0.756
217	Chetumal	2026-05-01	0.026	tranquilo	llegadas_tren/1000 hab, llegadas/cuarto	0.186	0.756
218	Chetumal	2026-06-01	0.023	tranquilo	llegadas_tren/1000 hab, llegadas/cuarto	0.186	0.756
219	Chetumal	2026-07-01	0.025	tranquilo	llegadas_tren/1000 hab, llegadas/cuarto	0.186	0.756

Inferidas: ipt_comparable = promedio de las medidas en escala 0–1 (z); estado = tranquilo si ipt < p50 (0.186), saturado si ipt > p90 (0.756), concurrido en medio.

Paso 2 · El índice y el estado (torre.radar.prediccion.indice_comparable).

Para predecir el mes siguiente armamos los rasgos: los rezagos del índice, el cambio, el mes como un punto en un círculo y, desde la decisión 34, cuatro rasgos del año anterior buscados por calendario.

```
In [8]:
d = prediccion.tabla_de_aprendizaje(t)   # los rasgos que ve el modelo
d[d.lugar == "Ruta arqueológica del sur"].tail(5)
```

Out[8]:

	periodo	ipt_comparable	ipt_1	cambio	mes_sin	ipt_11	ipt_12	anual_dif	pista_11	estado_hoy	objetivo
443	2026-02-01	0.186	0.227	-0.041	0.866	0.177	0.239	-0.052	0.124	tranquilo	tranquilo
444	2026-03-01	0.121	0.186	-0.065	1.000	0.188	0.177	-0.056	0.132	tranquilo	tranquilo
445	2026-04-01	0.152	0.121	0.030	0.866	0.101	0.188	-0.037	0.065	tranquilo	tranquilo
446	2026-05-01	0.098	0.152	-0.054	0.500	0.084	0.101	-0.004	0.080	tranquilo	tranquilo
447	2026-06-01	0.079	0.098	-0.018	0.000	0.138	0.084	-0.004	0.134	tranquilo	tranquilo

Todas las columnas resaltadas las calculamos: rezagos (shift), el mes como círculo y los rasgos del año anterior buscados por calendario (lugar, mes − 11 y − 12). objetivo = estado del mes siguiente.

Paso 3 · La tabla de aprendizaje; las columnas rosas las calculamos (torre.radar.prediccion.tabla_de_aprendizaje).

```
In [9]:
p = pd.read_parquet("datos/gold/radar_prediccion.parquet")   # lo que INFERIMOS
p[p.es_de_los_5 == 1]
```

Out[9]:

	lugar	periodo	estado_hoy	estado_mes_siguiente_est	prob_tranquilo	prob_concurrido	prob_saturado	modelo
1	Bahía Calderitas–Oxtankah	2026-07-01	tranquilo	tranquilo	1.000	0.000	0.000	Random Forest + año anterior
3	Chetumal	2026-07-01	tranquilo	tranquilo	1.000	0.000	0.000	Random Forest + año anterior
8	Maya Ka'an + Kantemó	2026-07-01	tranquilo	tranquilo	1.000	0.000	0.000	Random Forest + año anterior
11	Ruta arqueológica del sur	2026-07-01	tranquilo	tranquilo	0.937	0.063	0.000	Random Forest + año anterior

Inferidas por el modelo elegido (Random Forest + año anterior): el estado de agosto de 2026 y la probabilidad de cada estado. Validado: 130 de 156 en la prueba, Brier 0.214.

Paso 4 · Lo que inferimos: el estado de agosto de 2026 y su probabilidad (torre.radar.prediccion).

Cadena 3: del reparto del presupuesto al rango de la campaña

```
In [10]:
pl = pd.read_parquet("datos/gold/presupuesto_plan.parquet")   # PuLP / CBC
pl.head(6)
```

Out[10]:

	periodo	lugar	canal	pesos	conversiones_est
0	2026-10-01	Bahía Calderitas–Oxtankah	Google	3,982.9	6.332
1	2026-10-01	Bahía Calderitas–Oxtankah	Facebook	9,293.4	61.414
2	2026-10-01	Chetumal	Google	1,239.9	1.971
3	2026-10-01	Chetumal	Facebook	2,893.1	19.119
4	2026-10-01	Ruta arqueológica del sur	Google	4,553.1	7.238
5	2026-10-01	Ruta arqueológica del sur	Facebook	10,623.8	70.206

pesos = la decisión del modelo de optimización; conversiones_est = pesos × conversión ÷ (costo por clic × pesos por dólar), con los parámetros de WordStream, LocaliQ y FRED.

conversiones_est = pesos × conversión ÷ (costo por clic × pesos por dólar) (torre.campana.presupuesto).

```
In [11]:
mc = pd.read_parquet("datos/gold/presupuesto_montecarlo.parquet")
mc[mc.lugar == "Bahía Calderitas–Oxtankah"].head(5)
```

Out[11]:

	periodo	lugar	capacidad_probada	campana_p10_est	campana_p50_est	campana_p90_est	riesgo_sin_campana	riesgo_con_campana
0	2026-10-01	Bahía Calderitas–Oxtankah	1,639.0	40.984	55.819	71.948	0.002	0.024
3	2026-11-01	Bahía Calderitas–Oxtankah	1,639.0	41.035	55.888	72.037	0.000	0.025
6	2026-12-01	Bahía Calderitas–Oxtankah	1,639.0	0.000	0.000	0.000	0.151	0.151
9	2027-01-01	Bahía Calderitas–Oxtankah	1,639.0	0.000	0.000	0.000	0.041	0.041
12	2027-02-01	Bahía Calderitas–Oxtankah	1,639.0	28.710	39.102	50.400	0.000	0.018

¿Y si la campaña funciona mejor o peor? 10,000 sorteos de costo por clic, conversión y tipo de cambio dentro del rango de las fuentes, más los visitantes que llegan de todos modos.

El Monte Carlo de la campaña: cuánto puede variar lo que trae (torre.campana.presupuesto.montecarlo_campana).

Cómo lo programamos: comprobar las cadenas con código

Una captura muestra, pero no prueba. En el notebook 1 comprobamos las tres cadenas. Con Spark, la serie de la Ruta en Gold es la suma exacta de las tres zonas en Silver en los 127 meses (diferencia 0). El

estado del Radar se recalcula igual en 510 de 510 lugar-meses. Y `conversiones_est ÷ pesos` es una sola constante por canal: 6.61 visitantes por cada $1,000 en Facebook y 1.59 en Google.

```
inah = spark.read.parquet(str(SILVER / "inah"))
ruta = (inah.filter((F.col("estado") == "Quintana Roo")
                    & F.col("nombre").rlike("Kohunlich|Dzibanch|Ichkabal"))
            .groupBy("periodo").agg(F.sum("visitantes").alias("suma_silver"))
            .toPandas())
cadena = serie.merge(ruta, on="periodo")       # serie = Gold pronostico_series, Ruta
assert (cadena.valor == cadena.suma_silver).all()

inferidas = duckdb.sql("""SELECT file_name, name FROM parquet_schema('datos/gold/*.parquet')
                          WHERE name LIKE '%_est'""").df()   # 24 columnas en 9 tablas
```

Notebook 1, sección 8 · torre.documento.uca.volumenes

4. Planteamiento: el problema medido

> **Materia que lo encabeza:** Minería de Datos (Luis Becerril Espinosa), con la tutoría de Carlos Orozco Rocha · **Criterio 1 de la rúbrica (8 %):** delimita el problema e identifica variables, actores y relaciones.

4.1 ¿Qué tan concentrado está el turismo? El índice de Herfindahl-Hirschman

Pregunta. Si el problema es la concentración, hay que medirla. ¿Qué parte de los visitantes, cuartos y negocios del estado tienen los cinco lugares, y qué tan concentrada está cada dimensión?

> **La idea en palabras**
>
> Imaginemos un pastel repartido entre varias personas. Si una se lleva casi todo, el reparto está concentrado. El índice de Herfindahl-Hirschman (HHI) resume en un solo número qué tan desigual es el reparto: vale casi 1 si uno se lleva todo y vale poco si todos reciben parecido. Es el mismo índice que usan las autoridades de competencia para saber si un mercado está dominado por una sola empresa.

El cálculo se hace en tres pasos.

Paso A: la cuota de cada unidad (qué parte del pastel le toca):

$$s_i = \frac{x_i}{x_1 + x_2 + \cdots + x_N}$$

Paso B: el índice (se elevan las cuotas al cuadrado y se suman):

$$\text{HHI} = s_1^2 + s_2^2 + \cdots + s_N^2 = \sum_{i=1}^{N} s_i^2$$

Paso C: el índice normalizado (para que siempre vaya de 0 a 1):

$$\text{HHI}^* = \frac{\text{HHI} - \frac{1}{N}}{1 - \frac{1}{N}}$$

Símbolo	Qué significa
x_i	Lo que tiene la unidad i (pasajeros de un aeropuerto, cuartos de un destino…)
N	Cuántas unidades hay (4 aeropuertos, 11 municipios…)
s_i	Cuota de la unidad i; todas las cuotas suman 1
$\sum$	"Suma de": $\sum_{i=1}^{N} s_i^2$ se lee "suma los cuadrados de las cuotas, desde la unidad 1 hasta la N"
HHI^*	Índice normalizado: 0 = reparto parejo, 1 = todo en una sola unidad

¿Por qué elevar al cuadrado? Para que las unidades grandes pesen mucho más que las chicas. Una cuota de 0.9 aporta $0.9^2 = 0.81$; nueve cuotas de 0.01 juntas apenas aportan $9 \times 0.01^2 = 0.0009$. Así el índice "ve" la dominancia.

¿Por qué normalizar? El valor más bajo posible del HHI depende de cuántas unidades haya. Si todas pesan igual, cada cuota es $1/N$ y el índice vale $N \times (1/N)^2 = 1/N$: con 4 aeropuertos el mínimo es 0.25; con 11 municipios, 0.09. Para comparar dimensiones con distinto N se resta ese mínimo y se divide entre el rango posible.

Paso a paso con nuestros datos: llegadas en avión, 2024

Paso 1. Cuotas. El total es 14,769,302 + 620,384 + 352,067 + 217,524 = 15,959,277 pasajeros. La cuota de Cancún es 14,769,302 ÷ 15,959,277 = 0.92544.

Paso 2. Cuadrados. Cancún: $0.92544^2 = 0.856433$; Tulum: $0.03887^2 = 0.001511$; Cozumel: $0.02206^2 = 0.000487$; Chetumal: $0.01363^2 = 0.000186$.

Paso 3. Suma. HHI = 0.856433 + 0.001511 + 0.000487 + 0.000186 = 0.858617.

Paso 4. Normalizar. Con $N = 4$ aeropuertos, el mínimo es $1/4 = 0.25$:

$$\text{HHI}^* = \frac{0.8586 - 0.25}{1 - 0.25} = \frac{0.6086}{0.75} = 0.811$$

Qué significa: en una escala de 0 a 1, el tráfico aéreo está en 0.81: casi todo entra por una sola puerta.

La razón contra la población

Para saber si los cinco lugares reciben "poco" o "mucho", comparamos su parte de cada cosa con su parte de la gente:

$$\rho = \frac{\text{parte de la dimensión que tienen los 5 lugares}}{\text{parte de la población que tienen los 5 lugares}}$$

Si $\rho = 1$, tienen de visitantes lo mismo que de habitantes; si $\rho < 1$, reciben menos de lo que "les tocaría" por su población. Con avión: $\rho = 1.4/12.3 = 0.11$, la novena parte. Ojo: ρ es una comparación, no una meta; nada dice que el turismo deba repartirse según la población.

Dimensión	HHI*	Parte de los 5 lugares	ρ
Llegadas en avión (2024)	0.811	1.4 %	0.11
Cuartos de hotel (jul-2026)	0.219	1.7 %	0.14
Visitantes INAH (2025)	0.276	4.1 %	0.33
Negocios turísticos (DENUE)	0.133	14.4 %	1.17
Población (2020)	0.225	12.3 %	1.00

La lectura central del proyecto: los negocios están donde está la gente ($\rho = 1.17$), pero los visitantes no (ρ entre 0.11 y 0.33). El sur tiene la infraestructura de una región habitada y recibe una fracción de los visitantes. Eso es **capacidad ociosa**.

Cómo lo programamos: el índice de Herfindahl-Hirschman

La función recibe lo que tiene cada unidad (pasajeros por aeropuerto, cuartos por destino…) y devuelve el índice y su versión normalizada. Con los cuatro aeropuertos de 2024 da el resultado del ejemplo a mano. El notebook 2 la aplica a las cinco dimensiones y compara los cinco resultados con los de la tabla de arriba:

```
def hhi(valores):
    s = valores / valores.sum()              # cuota de cada unidad
    h = (s ** 2).sum()                       # suma de los cuadrados
    n = len(s)
    return h, (h - 1 / n) / (1 - 1 / n)      # HHI y HHI normalizado (0 a 1)

aereo = pd.Series({"Cancún": 14_769_302, "Tulum": 620_384,
                   "Cozumel": 352_067, "Chetumal": 217_524})   # pasajeros 2024, SITUR-Q
hhi(aereo)                                   # (0.8586, 0.811)
rho = parte_de_los_5 / parte_de_la_poblacion  # 1.4 % / 12.3 % = 0.11
```

Notebook 2, sección 3 · torre.radar.planteamiento.cuotas_y_hhi

Cómo lo programamos: contar por localidad

Los cinco lugares se cuentan con sus localidades del Censo (clave de municipio y clave de localidad), no con su municipio. Con esas claves se marca cada negocio del DENUE y cada localidad del Censo que pertenece a los cinco lugares, y de ahí salen su parte de los negocios (14.4 %) y de la población (12.3 %):

```
LOCALIDADES = {"Chetumal": [("004", "0001")], "Bahía Calderitas-Oxtankah": [("004", "0016")],
               "Laguna Milagros-Xul-Ha": [("004", "0114"), ("004", "0037")],
               "Ruta arqueológica del sur": [("004", "0064"), ("004", "0245"), ("004", "0033")],
               "Maya Ka'an + Kantemó": [("002", "0001"), ("002", "0250"), ("002", "0239"),
                                        ("002", "0044"), ("006", "0076")]}
CLAVES = {k for v in LOCALIDADES.values() for k in v}

def en_los_5(df):                             # ¿el renglón cae en una de esas localidades?
    return pd.Series([(m, l) in CLAVES for m, l in zip(df.cve_mun, df.cve_loc)], index=df.index)

parte_negocios = 100 * en_los_5(denue_turisticos).mean()                      # 14.4 %
parte_poblacion = 100 * loc[en_los_5(loc)].poblacion.sum() / poblacion_estado  # 12.3 %
```

Notebook 2, sección 3 · torre.base.silver_iter.REGION_LOCALIDADES

4.2 ¿Por qué estos cinco lugares? La tabla de criterios

Usamos cinco criterios: sin sargazo en rojo, sin cierres recientes, sin saturación, sin fragilidad grave y con datos para medirlos. Tres de ellos se miden con fórmulas sencillas.

Visitantes por residente (presión sobre la comunidad):

$$R_d = \frac{\text{visitantes del lugar } d \text{ en 2025}}{\text{habitantes de las localidades de acceso}}$$

Variación de un sitio (se comparan los mismos meses para que la temporada no distorsione):

$$\Delta\% = \left(\frac{\text{visitantes enero–julio 2026}}{\text{visitantes enero–julio 2025}} - 1\right) \times 100$$

Meses seguidos abierta: se cuenta hacia atrás desde el último mes con dato hasta el último mes en cero. La regla: pasa si son al menos 12 y no hubo cierres en 2026.

Paso a paso con nuestros datos

Tulum, enero a julio. 476,247 ÷ 692,946 = 0.6873; menos 1 = −0.3127; por 100 = **−31.3 %**.

Presión en la Ruta del sur. Visitantes de Kohunlich + Dzibanché + Ichkabal = 21,850 + 6,592 + 38,186 = 66,628. Habitantes de los pueblos de acceso = 3,699 + 1,436 + 963 = 6,098. Entonces $R = 66{,}628/6{,}098 = 10.93$ visitantes por habitante.

Presión en Tulum. $R = 1{,}031{,}443/33{,}374 = 30.91$. La Ruta recibe un tercio de la presión de Tulum: no es poco, y por eso la campaña también le pone tope.

Dzibanché. Su último mes en cero fue enero de 2025; de febrero de 2025 a julio de 2026 son 11 + 7 = 18 meses ≥ 12: pasa. Muyil reabrió en febrero de 2026: solo 6 meses, no pasaría.

Por qué por localidad y no por municipio. Cuatro de los cinco lugares están en el municipio Othón P. Blanco (233,648 habitantes). Con la cifra municipal, Chetumal, Calderitas, la Laguna y la Ruta tendrían el mismo denominador y no se podrían comparar.

Un hallazgo que reportamos aunque no nos conviene: Kohunlich (+13.5 %) y Dzibanché (+69.2 %) crecen en 2026, pero la Ruta completa bajó 8.2 % porque Ichkabal cayó 27.5 %. Nunca destacamos solo las zonas que suben: sería escoger datos a conveniencia.

Cómo lo programamos: los meses seguidos abierta

El criterio de cierres recorre la serie mensual de cada zona arqueológica **de atrás hacia adelante** y cuenta los meses con visitantes hasta encontrar el primer mes en cero. Una región toma el valor de su zona menos abierta, y pasa si suma al menos 12 meses y no tuvo meses cerrados en 2026. Así Dzibanché, que estuvo en cero hasta enero de 2025, suma 18 meses y pasa:

```
minimo = None
for nombre, g in zonas.groupby("nombre"):
    m = g.groupby(["anio", "mes"]).visitantes.sum().sort_index()
    seguidos = 0
    for v in reversed(m.values):              # del último mes publicado hacia atrás
        if v == 0:                            # un mes con 0 visitantes es un mes cerrado
            break
        seguidos += 1
    minimo = seguidos if minimo is None else min(minimo, seguidos)
pasa = minimo >= 12 and cerrados_2026 == 0    # regla del criterio 2
```

Notebook 2, sección 4 · torre.radar.criterios._meses_abierta

4.3 Variables, actores y relaciones

- **17 variables** con su fuente, periodo y porcentaje de huecos: 9 miden presión (llegadas por tren, crucero, frontera y avión; visitantes INAH; ocupación), 3 miden capacidad (cuartos, hoteles, negocios), 2 miden comunidad (población, servicios de vivienda) y 3 son huecos declarados (afluencia y las dos derramas).
- **Actores:** las comunidades de los cinco lugares (229,247 habitantes), los negocios turísticos de sus localidades (1,966), el Tren Maya (58,086 personas bajaron en Chetumal y Maya Ka’an en 2025), la frontera con Belice (653,306 cruces en 2025), el INAH (77,645 visitantes en las cuatro zonas) y las fuentes SITUR-Q y DataTur.
- **Relación central:** la concentración medida con el HHI.

5. Minería de datos: encontrar patrones

Materia: Minería de Datos · **Profesor:** Luis Becerril Espinosa · **Criterio 3 de la rúbrica (12 %):** identifica patrones, tendencias y relaciones con técnicas pertinentes y los interpreta en función del problema.

Conexión con la materia. El incidente crítico de esta UCA, *"Cuando los datos no mienten, pero los patrones sí importan"*, pide descubrir los patrones que explican por qué los turistas eligen destinos saturados, y menciona por nombre las técnicas que usamos: **reglas de asociación**, **análisis de sentimiento** y detección de eventos raros como huracanes. También advierte sobre la capacidad de carga, la interpretabilidad para tomadores de decisiones no técnicos y la privacidad. En este capítulo aplicamos: normalización y construcción de un índice (5.1), agrupamiento jerárquico con validación por silueta (5.2), descomposición estacional (5.3), detección de anomalías (5.4), minería de texto y reglas de asociación con Apriori (5.5) y clasificación por léxico (5.6).

5.1 El índice de presión turística (IPT)

Pregunta. ¿Qué lugar está más presionado, si cada uno se mide con cosas distintas? Chetumal tiene Tren Maya y cruces desde Belice; la Ruta tiene visitantes a zonas arqueológicas; Cancún tiene ocupación hotelera semanal. Hace falta una sola escala.

La idea en palabras

Es como calificar a alumnos de grupos distintos con exámenes distintos. Primero se pasa cada examen a la misma escala, de 0 a 1 (0 = la peor calificación de todo el estado, 1 = la mejor). Después se promedian los exámenes que presentó cada alumno. Al final se dice quién está en la mitad baja, quién en el 10 % más alto y quién en medio.

Los datos. Una tabla de 15 lugares × 55 meses (enero de 2022 a julio de 2026) = 825 renglones, con: personas que bajan del Tren Maya, cruceristas y visitantes INAH por cada mil habitantes, llegadas por cuarto de hotel y ocupación. Los diez destinos del resto del estado dan la escala común; sin ellos, "tranquilo" no significaría nada.

Paso 1. Llevar todo a la misma escala (normalización mín–máx):

$$z = \frac{x - \text{mín}}{\text{máx} - \text{mín}}$$

Paso 2. Promediar lo que cada lugar tiene:

$$\text{IPT} = \frac{z_1 + z_2 + \cdots + z_n}{n}$$

Paso 3. Clasificar con percentiles comunes:

$$\text{estado} = \begin{cases} \text{tranquilo} & \text{si IPT} < u_{50} \\ \text{concurrido} & \text{si } u_{50} \le \text{IPT} \le u_{90} \\ \text{saturado} & \text{si IPT} > u_{90} \end{cases}$$

Símbolo	Qué significa
x	El valor medido de un componente (por ejemplo, 68.78 % de ocupación)
mín, máx	El valor más bajo y más alto de ese componente en todos los lugares y meses juntos
z	El valor ya en escala de 0 a 1
n	Cuántos componentes tiene ese lugar en ese mes (no todos los lugares tienen los mismos)
u_{50}, u_{90}	Percentiles 50 y 90 del índice: la mitad de los casos queda abajo de u_{50} y el 90 % abajo de u_{90}

Por qué mín–máx. La transformación es una línea recta, así que conserva el orden y las proporciones, y el 0 y el 1 tienen significado ("lo más bajo" y "lo más alto registrado en el estado"). Estandarizar ($z = (x - \bar{x})/\sigma$) da números negativos y sin límite, más difíciles de explicar; usar percentiles pierde las proporciones.

Un componente entra al índice solo si lo tienen al menos dos lugares; si no, ese lugar se compararía contra sí mismo. Así salió Belice, que solo tiene Chetumal.

Paso a paso con nuestros datos: Cancún, julio de 2026

Componente	Valor x	mín	máx	z
Tren Maya (por mil habitantes)	25.1081	0	505.2287	0.0497
INAH, zona El Rey (por mil)	0.9608	0	6,451.8787	0.0001
Llegadas por cuarto	0.4709	0	432.0230	0.0011
Ocupación (DataTur)	68.78 %	17.97	88.40	0.7214

Paso 1, Tren Maya: $z = (25.1081 - 0)/(505.2287 - 0) = 0.0497$.

Paso 1, ocupación: $z = (68.78 - 17.97)/(88.40 - 17.97) = 50.81/70.43 = 0.7214$.

Paso 2, promedio de los cuatro:

$$\text{IPT} = \frac{0.0497 + 0.0001 + 0.0011 + 0.7214}{4} = \frac{0.7723}{4} = 0.1931$$

Paso 3: con los cortes del índice comparable ($u_{50} = 0.186$, $u_{90} = 0.756$), 0.193 queda entre los dos: Cancún está **concurrido**.

La prueba de validez que falló, y por qué la contamos

La primera versión (solo con SITUR-Q) calificaba a Cancún como "tranquilo" en julio de 2026, con 0.025. La razón: SITUR-Q dejó de publicar la ocupación en 2025 y, sin ella, Cancún solo tenía componentes casi

en cero; para ese mes DataTur marcaba 68.8 %. Un índice que dice que Cancún está tranquilo en julio no es válido. Comparamos cuatro variantes:

Variante	Cancún	Isla Mujeres	Chetumal	Ruta	Problema
A. Mín–máx, solo SITUR-Q	0.02	—	0.39	0.14	El norte “desaparece”
B. A + ocupación DataTur	0.28	0.45	0.39	0.14	Chetumal arriba de Cancún
C. Percentil + DataTur	0.44	0.25	0.48	0.71	La Ruta arriba de Cancún
D. B sin componentes de un solo lugar (elegida)	0.28	0.45	0.05	0.14	Quiebre de 2025

Dos índices. El quiebre de 2025 (Chetumal “cayó” de 0.54 a 0.05 sin que llegara menos gente) fue un cambio en los datos, no en la realidad, y un modelo entrenado con eso aprendería cambios falsos. Por eso el Radar publica un **índice comparable**: cada lugar usa solo las medidas que tiene hoy, y solo en los meses en que las tiene todas. Con él, en julio de 2026 estaban concurridos Mahahual (0.731), Isla Mujeres (0.510), Bacalar (0.314), Holbox (0.228), Cancún (0.193) y Cozumel (0.191); los cinco lugares de la campaña estaban tranquilos (Ruta 0.129, Chetumal 0.025, Oxtankah 0.020, Maya Ka’an 0.009). La Laguna Milagros no tiene ninguna serie: aparece como “sin dato oficial”, nunca como “tranquila”.

Sensibilidad. Movimos cada peso a 0.5 y a 1.5 (con los demás en 1) y contamos cuántos lugar-mes cambian de estado: entre el 0.6 % y el 6.0 %. La elección de pesos iguales no decide el resultado.

Cómo lo programamos: el índice, los estados y la sensibilidad

Primero cada medida se lleva a la escala de 0 a 1 con el mínimo y el máximo de todos los lugares y meses juntos. Después, el índice de cada lugar y mes es el promedio de las medidas que tiene ese mes: las que faltan no cuentan (no se rellenan). Por último, los percentiles 50 y 90 de todos los valores dan los tres estados. La función `ipt` acepta pesos, y eso permite la prueba de sensibilidad del final:

```
for k in componentes:                                   # mín–máx común a todos
    x = c[k].astype(float)
    z[f"z_{k}"] = (x - x.min()) / (x.max() - x.min())

def ipt(z, pesos=None):
    pesos = pesos or {k: 1.0 for k in componentes}       # pesos iguales por omisión
    Z = z[[f"z_{k}" for k in componentes]].to_numpy()
    W = np.array([pesos[k] for k in componentes])
    hay = ~np.isnan(Z)                                    # medidas que sí tiene el mes
    return np.nansum(np.nan_to_num(Z) * W, axis=1) / (hay * W).sum(axis=1)

u50, u90 = np.nanquantile(z.ipt, [0.50, 0.90])           # cortes comunes del estado
z["estado"] = np.select([z.ipt < u50, z.ipt <= u90], ["tranquilo", "concurrido"], "saturado")

for k in componentes:                                     # sensibilidad: peso × 0.5 y × 1.5
    for factor in (0.5, 1.5):
        nuevo = ipt(z, {j: factor if j == k else 1.0 for j in componentes})
```

Notebook 3, sección 1 · torre.radar.indice

Con cualquier medida a la mitad o al 150 % de su peso, cambia de estado entre el 0.6 % y el 6 % de los lugar-mes; la que más mueve es la ocupación (5.7 % y 6.0 %).

Cómo lo programamos: el índice comparable

Para el aprendizaje de máquina, cada lugar usa solo las medidas que tiene en su último mes con dato, y solo en los meses en que las tiene todas. Así, cuando SITUR-Q dejó de publicar la ocupación del sur, Chetumal no «bajó» de presión por perder una medida: se mide siempre con las mismas dos, llegadas en tren por habitante y llegadas por cuarto, desde diciembre de 2024:

```
for lugar, g in base.groupby("lugar"):
    g = g.sort_values("periodo")
    con = g[g.ipt.notna()]
    actuales = [k for k in componentes if pd.notna(con.iloc[-1][f"z_{k}"])]  # las de hoy
    Zl = g[[f"z_{k}" for k in actuales]]
    completo = Zl.notna().all(axis=1)                    # meses con TODAS esas medidas
    g["ipt_comparable"] = Zl.mean(axis=1).where(completo)
```

Notebook 3, sección 1 · torre.radar.prediccion.indice_comparable

5.2 Agrupamiento de los centros turísticos del país (método de Ward)

Pregunta. ¿A qué se parece el norte de Quintana Roo dentro del país? Si está en el grupo de los destinos más llenos de México, la saturación no es una impresión.

> **La idea en palabras**
>
> Cada centro turístico tiene una “huella” de 12 números: qué tan lleno está cada mes del año. El agrupamiento junta los centros cuyas huellas se parecen. Empieza con cada centro solo y va uniendo, de dos en dos, los grupos que menos “desordenan” el conjunto, hasta formar un árbol.

La huella de cada centro (su ocupación en cada mes del año, sumando los cuartos de todos los años):

$$p_{c,m} = 100 \times \frac{\text{cuartos ocupados del centro } c \text{ en el mes } m\text{, todos los años}}{\text{cuartos disponibles del centro } c \text{ en el mes } m\text{, todos los años}}$$

Qué tan distintos son dos centros (distancia euclidiana, el “teorema de Pitágoras” con 12 dimensiones):

$$d(c, c') = \sqrt{(p_{c,1} - p_{c',1})^2 + (p_{c,2} - p_{c',2})^2 + \cdots + (p_{c,12} - p_{c',12})^2}$$

El costo de unir dos grupos A y B según Ward (1963):

$$\Delta(A,B) = \frac{|A| \cdot |B|}{|A| + |B|} \parallel \bar{p}_A - \bar{p}_B \parallel^2$$

Símbolo	Qué significa
$\|A\|$	Cuántos centros tiene el grupo A
$\bar{p}_A$	La huella promedio del grupo A
$\parallel \bar{p}_A - \bar{p}_B \parallel^2$	La distancia al cuadrado entre las dos huellas promedio
$\Delta(A,B)$	Cuánto aumenta el "desorden" (la varianza dentro de los grupos) si se unen A y B

En cada paso, Ward une los dos grupos con el menor Δ: la unión más "barata". El resultado son grupos compactos.

¿Cuántos grupos? Se mide con la **silueta** de cada centro (Rousseeuw, 1987):

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}}$$

donde $a(i)$ es la distancia promedio del centro a los de su propio grupo y $b(i)$ la distancia promedio al grupo vecino más cercano. Va de −1 (mal asignado) a 1 (perfectamente asignado). Se elige el número de grupos con la mayor silueta promedio:

Número de grupos k	2	3	4	5	6	7	8
Silueta promedio	**0.450**	0.380	0.365	0.300	0.283	0.317	0.306

Paso a paso con nuestros datos: silueta de Cancún

Paso 1. Distancia promedio de Cancún a los otros 22 centros de su grupo: $a = 43.84$.

Paso 2. Distancia promedio a los 32 centros del otro grupo: $b = 116.21$.

Paso 3.

$$s = \frac{116.21 - 43.84}{\max\{43.84,\ 116.21\}} = \frac{72.37}{116.21} = 0.623$$

Qué significa: 0.623 está cerca de 1: Cancún está bien asignado a su grupo.

Resultado ($k = 2$). Grupo 1, playas y destinos muy ocupados: 23 centros con 64.5 % de ocupación promedio (Cancún, Riviera Maya, Playacar, Akumal, Playa del Carmen, Los Cabos). Grupo 2, el resto del país: 32 centros con 41.1 % (incluye a Cozumel e Isla Mujeres). Solo entran centros con sus 55 meses completos: los incompletos no se rellenan, se excluyen.

Segunda opinión. Con k-means y el mismo número de grupos, el índice de Rand ajustado es 0.49: la frontera entre grupos depende del método (k-means pasa al grupo alto 8 centros intermedios, entre ellos Cozumel e Isla Mujeres), pero los 23 centros del grupo alto de Ward quedan también en el grupo alto de k-means. La conclusión que usamos, que el norte de Quintana Roo está en el grupo de ocupación más alta del país, no depende del método.

Cómo lo programamos: Ward, la silueta y una segunda opinión

Cada centro turístico es un renglón con 12 números: su ocupación de cada mes del año. `linkage` arma el árbol de Ward completo; `fcluster` lo corta en k grupos y la silueta mide qué tan bien quedó cada corte. Se elige el k con la silueta más alta (k = 2, con 0.450). Al final, k-means con el mismo k sirve de segunda opinión: el índice de Rand ajustado da 0.49, y los 23 centros del grupo alto de Ward quedan también en el grupo alto de k-means:

```
from scipy.cluster.hierarchy import fcluster, linkage
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.cluster import KMeans

perfil = (100 * s.cuartos_ocupados / s.cuartos_disponibles).unstack("mes")  # centro × 12 meses
Z = linkage(perfil.to_numpy(), method="ward")             # une lo que menos sube la varianza
siluetas = {k: silhouette_score(perfil, fcluster(Z, k, criterion="maxclust"))
            for k in range(2, 9)}
k = max(siluetas, key=siluetas.get)                       # k = 2
grupo = fcluster(Z, k, criterion="maxclust")
km = KMeans(n_clusters=k, n_init=20, random_state=0).fit_predict(perfil)
adjusted_rand_score(grupo, km)                            # 0.49
```

Notebook 3, sección 2 · torre.radar.clustering

5.3 La forma del año (estacionalidad)

Pregunta. ¿Qué meses se llenan y cuáles se vacían en cada lugar? Es la base de la temporada alta, del pronóstico y del calendario de la campaña.

> **La idea en palabras**
>
> Para saber si enero es “alto”, se compara enero con el promedio de su propio año. Si un año tuvo 5,000 visitantes al mes en promedio y enero tuvo 8,000, enero fue 1.6 veces un mes normal. Se repite con varios años y se promedia: lo que queda es la “forma” del año.

Paso 1. La razón de cada mes contra su año:

$$r_{a,m} = \frac{y_{a,m}}{\bar{y}_a}, \qquad \bar{y}_a = \frac{y_{a,1} + y_{a,2} + \cdots + y_{a,12}}{12}$$

Paso 2. El índice del mes (promedio de las razones de ese mes en todos los años A disponibles):

$$\tilde{S}_m = \frac{1}{|A|} \sum_{a \in A} r_{a,m}$$

Paso 3. Reescalar para que los 12 índices promedien exactamente 1: $S_m = \tilde{S}_m / \left(1/12 \sum_{k=1}^{12} \tilde{S}_k\right)$.

Símbolo	Qué significa
$y_{a,m}$	Visitantes del año a en el mes m
$\bar{y}_a$	Promedio mensual del año a
$r_{a,m}$	Cuántas veces un mes normal fue ese mes en ese año
S_m	Índice estacional del mes m: 1.6 = “60 % más que un mes normal”; 0.5 = “la mitad”

¿Multiplicativa o aditiva? Aditiva diría “enero tiene 2,000 visitantes más”; multiplicativa, “enero tiene 61 % más”. Kohunlich reabrió en 2025 con la mitad de su nivel de 2019: “+2,000” ya no vale, pero “+61 %” sí. La temporada crece con el nivel, así que es multiplicativa.

Fuerza de la temporada (Wang et al., 2006). Mide qué parte de los cambios de un mes a otro explica la temporada:

$$F_S = \max\left(0,\ 1 - \frac{\mathrm{Var}(e)}{\mathrm{Var}(s+e)}\right)$$

donde $s + e$ es toda la variación de un mes a otro (en logaritmos) y e es la parte que la temporada no explica. Si la temporada lo explica todo, $\mathrm{Var}(e) = 0$ y $F_S = 1$.

Paso a paso con nuestros datos: Ruta arqueológica del sur

Paso 1, una razón. En 2019, Kohunlich + Dzibanché sumaron 64,139 visitantes: $\bar{y}_{2019} = 64{,}139/12 = 5{,}344.92$. En enero hubo 7,935: $r_{2019,1} = 7{,}935/5{,}344.92 = 1.4846$.

Paso 2, el índice de enero. Las seis razones de enero (2016–2019, 2022 y 2023) son 1.5459, 1.7075, 1.8266, 1.4846, 1.2540 y 1.8173. Suman 9.6359; entre 6: $\tilde{S}_1 = 1.606$.

Paso 3. El promedio de los 12 índices ya es 1.0000, así que $S_1 = 1.606$: en enero llega **61 % más gente** que en un mes promedio. En septiembre, $S_9 = 0.496$: la mitad.

Fuerza: $\mathrm{Var}(e) = 0.03152$ y $\mathrm{Var}(s+e) = 0.15230$:

$$F_S = 1 - \frac{0.03152}{0.15230} = 1 - 0.2070 = 0.793$$

El 79 % de la variación de un mes a otro en la Ruta es temporada.

Serie	Fuerza F	Mes más alto	Mes más bajo
Ruta arqueológica del sur	0.793	enero (1.606)	septiembre (0.496)
Bahía Calderitas–Oxtankah	0.684	diciembre (1.402)	septiembre (0.647)
Chetumal (cruces de Belice)	0.655	diciembre (1.172)	febrero (0.865)
Cancún (referencia)	0.714	marzo (1.084)	septiembre (0.867)

¿Por qué no STL? La descomposición STL necesita una serie continua, y estas tienen huecos (pandemia, cierres por obras). STL solo podría usar el tramo continuo más largo. La usamos como segunda opinión en ese tramo: su índice coincide con el nuestro entre 0.85 y 0.97.

Cómo lo programamos: la forma del año y su fuerza

La función `razones` arma una tabla de años por meses con el valor de cada mes dividido entre el promedio de su año, solo en años completos. El índice de cada mes es el promedio de su columna, reescalado para que los 12 promedien exactamente 1. La fuerza compara cuánto varía lo que el mes no explica contra la variación total, en logaritmos:

```
def razones(s):
    n = s[s.entrena_flag].groupby(s.periodo.dt.year).size()
    x = s[s.periodo.dt.year.isin(n[n == 12].index)].copy()        # solo años completos
    x["anio"], x["mes"] = x.periodo.dt.year, x.periodo.dt.month
```

```
    x["razon"] = x.valor / x.groupby("anio").valor.transform("mean")
    return x.pivot(index="anio", columns="mes", values="razon")

r = razones(serie)
indice = r.mean(axis=0) / r.mean(axis=0).mean()     # los 12 índices promedian 1
s_ = np.log(indice.to_numpy())[None, :]              # la temporada, en logaritmos
e = np.log(r.to_numpy()) - s_                        # lo que el mes no explica
fuerza = max(0, 1 - np.var(e) / np.var(s_ + e))      # Ruta: 0.79
```

Notebook 3, sección 3 · torre.pronostico.forma

Cómo lo programamos: la segunda opinión con STL

STL necesita una serie continua, así que solo se aplica al tramo continuo más largo de cada serie (unos 50 meses). La correlación entre la temporada que encuentra STL y la nuestra va de 0.85 a 0.97: las dos maneras de medir llegan a la misma forma del año:

```
from statsmodels.tsa.seasonal import STL
y = tramo_continuo(serie)                                  # el tramo más largo sin huecos
y.index = pd.DatetimeIndex(y.index, freq="MS")
estacional = STL(np.log(y), period=12, robust=True).fit().seasonal
stl = np.exp(estacional.groupby(estacional.index.month).mean())     # índice de cada mes
np.corrcoef(np.log(stl), np.log(indice))[0, 1]             # Ruta: 0.955
```

torre.pronostico.forma.segunda_opinion_stl

5.4 Clima raro con Isolation Forest (detección de anomalías)

Pregunta. ¿Esta semana tuvo un clima tan raro que conviene pausar el anuncio? Necesitamos detectar “lo raro” sin que alguien lo defina a mano.

> **La idea en palabras**
>
> Imaginemos que separamos puntos en un plano con cortes al azar. Un punto alejado de todos se queda solo con muy pocos cortes; un punto en medio de la multitud necesita muchísimos. Isolation Forest (Liu et al., 2008) hace eso con 300 árboles de cortes al azar y mide cuántos cortes tarda, en promedio, en aislar cada semana. Las semanas raras se aíslan rápido.

$$s(x) = 2^{-E[h(x)]\,/\,c(n)}, \qquad c(n) = 2\,H(n-1) - \frac{2(n-1)}{n}, \qquad H(k) \approx \ln k + 0.5772$$

Símbolo	Qué significa
$h(x)$	Número de cortes que tardó en aislarse la semana x en un árbol
$E[h(x)]$	El promedio de esos cortes en los 300 árboles
n	Semanas que usa cada árbol (hasta 256)
$c(n)$	Los cortes que tarda, en promedio, una semana cualquiera; sirve para comparar
$H(k)$	Número armónico: $1 + 1/2 + 1/3 + \cdots + 1/k$, que se aproxima con $\ln k + 0.5772$
$s(x)$	Puntaje de rareza: cerca de 0.5 es normal; cerca de 1, muy raro

Cómo leer la fórmula. Si una semana tarda lo mismo que cualquier otra ($E[h] = c(n)$), el exponente vale −1 y $s = 2^{-1} = 0.5$: normal. Si se aísla mucho más rápido ($E[h]$ mucho menor que $c(n)$), el exponente se acerca a 0 y s se acerca a $2^0 = 1$: muy rara.

Las variables son la lluvia de la semana, la lluvia del peor día, el viento máximo, la temperatura máxima y la época del año (como seno y coseno de la semana). La época entra para que un aguacero de septiembre no salga raro solo por ser septiembre.

Paso a paso con nuestros datos: Chetumal, semana del 14 de octubre de 2024 (tormenta Nadine)

Llovieron 148.7 mm (65.4 mm el peor día). Había 260 semanas de entrenamiento, así que cada árbol usa $n = 256$.

Paso 1, el número de referencia:

$$c(256) = 2(\ln 255 + 0.5772) - \frac{2 \times 255}{256} = 2(5.5413 + 0.5772) - 1.9922 = 12.237 - 1.992 = 10.245$$

Paso 2. El bosque calcula $s = 0.712$. El corte que elegimos (percentil 95 de las semanas de entrenamiento) es 0.543. Como 0.712 > 0.543, **la semana es rara**.

Paso 3, ¿cuántos cortes tardó? Despejando la fórmula: $E[h] = -c(n) \times \log_2 s = -10.245 \times \log_2 0.712 = -10.245 \times (-0.489) = 5.0$ cortes, contra unos 10 de una semana normal.

Lo que decidimos. El corte automático del método (marcar raro si $s > 0.5$) señalaba el 47 % de las semanas de Chetumal en el primer intento, entrenado solo con 2019–2021 (con la ventana creciente todavía marcaría el 36 %): con pocas semanas el bosque no distinguía, y desde 2022 llueve más y hace más calor que en 2019–2021. Elegimos “más raro que el 95 % de las semanas que el bosque ya conoce”, con un bosque por año entrenado con todos los años anteriores. Resultado: 21 semanas raras en Chetumal y 17 en Kohunlich.

Validación independiente: el bosque marcó como raras las tres semanas en que pasaron tormentas (Lisa en 2022, Nadine y Sara en 2024) sin saber que hubo tormenta. Es evidencia de que "raro" significa algo real.

Comparación con otros detectores. Con la misma ventana creciente y el mismo corte (percentil 95), probamos también la regla de 3 desviaciones, Local Outlier Factor y One-Class SVM. Los cuatro encuentran las tres semanas de tormenta, así que la diferencia está en las falsas alarmas, porque cada semana rara pausa los anuncios del sur: Isolation Forest es el que menos semanas marca (21 en Chetumal y 17 en Kohunlich, de 247), contra 23 y 20 de la regla de 3 desviaciones, 26 y 30 de Local Outlier Factor y 71 y 33 de One-Class SVM. Además mira las seis variables juntas y no supone que la lluvia siga una curva normal.

Cómo lo programamos: un bosque por año, sin ver el futuro

Para cada año desde 2022 se entrena un bosque con todas las semanas anteriores a ese año (ventana creciente) y se calcula el percentil 95 de sus puntajes; una semana del año es rara si lo supera. El signo menos en `-score_samples` voltea la escala de scikit-learn para que un número más alto signifique más raro. El notebook 3 corre además la regla de 3 desviaciones, Local Outlier Factor y One-Class SVM con la misma ventana y el mismo corte, para comparar las falsas alarmas:

```
from sklearn.ensemble import IsolationForest
VARIABLES = ["lluvia_mm", "lluvia_max_dia_mm", "viento_max_kmh", "temp_max_c",
             "epoca_sin", "epoca_cos"]                  # la época del año, como círculo

for anio in range(2022, ultimo_anio + 1):
    entrena = x[(x.semana >= "2019-01-01") & (x.semana < f"{anio}-01-01")]   # solo el pasado
    bosque = IsolationForest(n_estimators=300, random_state=0).fit(entrena[VARIABLES])
    corte = np.quantile(-bosque.score_samples(entrena[VARIABLES]), 0.95)    # percentil 95
    esta = x.anio == anio
    x.loc[esta, "puntaje"] = -bosque.score_samples(x.loc[esta, VARIABLES])
    x.loc[esta, "clima_raro"] = x.loc[esta, "puntaje"] > corte
```

Notebook 3, sección 4 · torre.envivo.senales.anomalias_clima

Cómo lo programamos: los otros tres detectores

Para comparar, los tres detectores alternativos usan la misma ventana creciente y el mismo corte del percentil 95. La regla de 3 desviaciones mira cada variable por separado; Local Outlier Factor y One-Class SVM necesitan estandarizar los datos primero. Los cuatro encuentran las tres semanas de tormenta; el bosque es el que menos semanas marca (21 y 17, contra 71 y 33 de One-Class SVM):

```
def z_score(entrena, prueba):                   # ¿alguna variable a más de 3 desviaciones?
    m, s = entrena.mean(), entrena.std()
    return (np.abs((entrena - m) / s).max(axis=1), np.abs((prueba - m) / s).max(axis=1))

def lof(entrena, prueba):                       # densidad de vecinos contra la de sus vecinos
    m = make_pipeline(StandardScaler(), LocalOutlierFactor(n_neighbors=20, novelty=True))
    m.fit(entrena)
    return -m.score_samples(entrena), -m.score_samples(prueba)

def ocsvm(entrena, prueba):                     # una frontera alrededor de lo normal
    m = make_pipeline(StandardScaler(), OneClassSVM(nu=0.05)).fit(entrena)
    return -m.score_samples(entrena), -m.score_samples(prueba)
```

Notebook 3, sección 4

5.5 Minería de texto de las reseñas

Pregunta. ¿Qué molesta y qué enamora al turista? Es lo que define la promesa de la campaña y sus palabras.

Datos: 85,987 reseñas de Quintana Roo (Rest-Mex 2025, sin duplicados) de Tulum, Isla Mujeres y Bacalar; 3,597 son malas (1–2 estrellas) y 60,082 de 5 estrellas. Ninguna es de los cinco lugares, así que sirven para saber qué molesta en los destinos llenos, no para opinar del sur.

Temas y riesgo relativo

Cada tema es una lista de raíces de palabras que cuentan solo al inicio de una palabra. Por ejemplo, "ruido" incluye *ruido, ruidos, escándalo, música alta, fiesta*; "calma" incluye *tranquil-, paz, relaj-, silencio*. Para saber si un tema "aleja" o "protege", usamos el **riesgo relativo**, la medida que usa la epidemiología:

$$RR = \frac{P(\text{mala} \mid \text{la reseña habla del tema})}{P(\text{mala})}$$

Se lee: "¿cuántas veces más seguido es mala una reseña que habla de este tema, comparada con una reseña cualquiera?". $RR = 1$: el tema no cambia nada; $RR = 2$: es mala el doble de seguido; $RR < 1$: el tema protege.

Paso a paso con nuestros datos: el ruido

Paso 1. Probabilidad de que una reseña cualquiera sea mala: $P(\text{mala}) = 3{,}597/85{,}987 = 0.04183$ (4.2 %).

Paso 2. De las 2,038 reseñas que hablan de ruido, 235 son malas: $P(\text{mala} \mid \text{ruido}) = 235/2{,}038 = 0.11531$ (11.5 %).

Paso 3.

$$RR_{\text{ruido}} = \frac{0.11531}{0.04183} = 2.76$$

Qué significa: una reseña que habla de ruido es mala casi tres veces más seguido que una reseña cualquiera.

Tema	RR	Lectura	Tema	RR	Lectura
Ruido	2.76	Aleja	Comida	0.81	Protege un poco
Suciedad	1.89	Aleja	Naturaleza	0.73	Protege
Precio	1.66	Aleja	Cultura	0.48	Protege
Multitudes	1.52	Aleja	Calma	0.41	Protege
Servicio	1.18	Neutro			

Por qué léxico y no un modelo de temas (LDA). Con LDA cada tema es una lista de palabras que hay que interpretar. Con un léxico, cada tema se define antes y se defiende palabra por palabra. Para una campaña que tiene que explicar por qué promete calma, lo segundo es más defendible.

Reglas de asociación (algoritmo Apriori)

La idea en palabras

Es el análisis de la "canasta del súper": si quien compra pan suele comprar leche, hay una regla pan ⇒ leche. Aquí cada reseña es una canasta con sus temas y su calificación, y buscamos qué temas aparecen juntos con las reseñas malas o con las de 5 estrellas.

$$\text{soporte}(A) = \frac{\text{reseñas que contienen } A}{N}, \qquad \text{confianza}(A \Rightarrow B) = \frac{\text{soporte}(A \text{ y } B)}{\text{soporte}(A)}, \qquad \text{lift} = \frac{\text{confianza}(A \Rightarrow B)}{\text{soporte}(B)}$$

Medida	Pregunta que responde
Soporte	¿Qué tan frecuente es la combinación en todo el conjunto?
Confianza	Cuando aparece A, ¿qué tan seguido aparece también B?
Lift	¿Cuántas veces más probable es B cuando aparece A, comparado con lo normal? Si vale 1, no hay relación

El truco de Apriori (Agrawal y Srikant, 1994): si una combinación es rara, todas las combinaciones más grandes que la contienen también lo son. Así no tiene que revisar todas las combinaciones posibles. Usamos soporte mínimo de 0.1 % y combinaciones de hasta tres elementos.

Paso a paso con nuestros datos: ruido + mal servicio ⇒ reseña mala

1,204 reseñas hablan de ruido y de servicio a la vez; 142 de ellas son malas.

Soporte = 142 ÷ 85,987 = 0.00165. **Confianza** = 142 ÷ 1,204 = 0.1179. **Lift** = 0.1179 ÷ 0.04183 = **2.82**.

Qué significa: una reseña que se queja de ruido y de servicio es mala 2.8 veces más seguido de lo normal. Otras reglas: suciedad + multitudes (lift 2.59), suciedad + precio (2.54). Hacia 5 estrellas: comida + servicio (1.12) y calma + naturaleza (1.10).

Las palabras que distinguen a las reseñas buenas

Contar palabras no basta: una palabra rara que aparece dos veces en las buenas y nunca en las malas daría un cociente enorme por pura casualidad. El método de Monroe et al. (2008) agrega un "colchón" proporcional a lo común que es la palabra y entrega un puntaje z, que se lee como "cuántas desviaciones estándar se aleja la palabra de lo esperado":

$$\delta_w = \ln\frac{y_w^A + \alpha_w}{n^A + \alpha_0 - y_w^A - \alpha_w} - \ln\frac{y_w^B + \alpha_w}{n^B + \alpha_0 - y_w^B - \alpha_w}, \qquad z_w = \frac{\delta_w}{\sqrt{\frac{1}{y_w^A + \alpha_w} + \frac{1}{y_w^B + \alpha_w}}}$$

Símbolo	Qué significa
A, B	Reseñas de 5 estrellas y reseñas malas
y_w^A	Veces que aparece la palabra w en las reseñas de 5 estrellas
n^A	Total de palabras en las reseñas de 5 estrellas
α_w	El colchón: 0.01 × veces que aparece la palabra en todo el corpus
δ_w	Diferencia de "momios" (en logaritmo) entre buenas y malas
z_w	Puntaje final: positivo = palabra de reseña buena; negativo = de reseña mala

Paso a paso con nuestros datos: la palabra "excelente"

Datos: $y^A = 18{,}952$, $y^B = 144$, $\alpha = 190.96$, $n^A = 1{,}879{,}868$, $n^B = 162{,}035$, $\alpha_0 = 20{,}419.0$.

Paso 1, momios en las buenas: $(18{,}952 + 190.96)/(1{,}879{,}868 + 20{,}419 - 18{,}952 - 190.96) = 19{,}142.96/1{,}881{,}144.04$; su logaritmo es −4.5878.

Paso 2, momios en las malas: $334.96/182{,}119.04$; su logaritmo es −6.2985.

Paso 3, diferencia: $\delta = -4.5878 - (-6.2985) = 1.7107$.

Paso 4, puntaje:

$$z = \frac{1.7107}{\sqrt{1/19{,}142.96 + 1/334.96}} = \frac{1.7107}{0.05512} = 31.04$$

"Excelente" es de las palabras que más distinguen a las reseñas de 5 estrellas. En el otro extremo, "sucia" tiene $z = -18.76$.

Palabras de 5 estrellas: *excelente, increíble, hermoso, deliciosa, amable, ruinas, ambiente*. Palabras de reseñas malas: *peor, horrible, dinero, caro, sucia, grosero*. Con ellas se escribieron los mensajes de la campaña (capítulo 9).

Cómo lo programamos: temas, reglas y palabras

Cada tema es una lista de raíces; `\b` exige que la raíz esté al inicio de una palabra. El riesgo relativo divide la proporción de reseñas malas entre las que tocan el tema y la proporción general. Apriori busca combinaciones de hasta tres elementos que aparezcan en al menos el 0.1 % de las reseñas (unas 86) y deja las reglas con lift mayor que 1:

```
for tema, raices in ASPECTOS.items():                  # "multitudes": ["gente", "lleno", ...]
    patron = r"\b(?:" + "|".join(re.escape(x) for x in raices) + ")"
    r[tema] = r.texto.str.contains(patron, regex=True)
r["mala"] = r.calificacion <= 2
riesgo = {t: r.mala[r[t]].mean() / r.mala.mean() for t in ASPECTOS}   # ruido: 2.76
```

```
from mlxtend.frequent_patterns import apriori, association_rules
canasta = r[list(ASPECTOS)].assign(**{"reseña mala (1–2)": r.mala, "reseña de 5": r.cinco})
frec = apriori(canasta.astype(bool), min_support=0.001, use_colnames=True, max_len=3)
reglas = association_rules(frec, metric="lift", min_threshold=1.0)
```

Notebook 3, sección 5 · torre.campana.texto

Las palabras que distinguen a las reseñas de 5 estrellas salen del log-odds con prior informativo: el prior de cada palabra es el 1 % de su frecuencia total, para que una palabra que aparece tres veces no parezca importante por azar, y solo cuentan las palabras con al menos 50 apariciones:

```
alfa = {w: 0.01 * c for w, c in todo.items()}          # prior: 1 % de su frecuencia
for w, c in todo.items():
    if c < 50:
        continue
    ya, yb, al = A[w], B[w], alfa[w]                     # en reseñas de 5 y de 1–2
    delta = (np.log((ya + al) / (na + a0 - ya - al))
             - np.log((yb + al) / (nb + a0 - yb - al)))
    z = delta / np.sqrt(1 / (ya + al) + 1 / (yb + al))     # puntaje z de la palabra
```

Notebook 3, sección 5 · torre.campana.texto.palabras_que_distinguen

5.6 Qué hacer, dónde comer y dónde dormir: clasificación por léxico

De los negocios del DENUE, ¿cuáles recomendar en cada lugar? Usamos un clasificador por léxico que lee el nombre del negocio y, si el nombre no dice nada, su giro oficial (SCIAN). No entrenamos un modelo estadístico porque no hay etiquetas con qué entrenarlo; un léxico es lo honesto y se explica regla por regla. Las reglas salieron de errores reales: “MICHELADAS” contiene “HELAD” y salía como heladería, y “BARBACOA” contiene “BAR”; por eso la raíz cuenta solo al inicio de una palabra. **Precisión medida:** revisamos a mano dos muestras al azar: 38 de 40 y, tras corregir, 39 de 40 en una muestra nueva.

Cómo lo programamos: la raíz al inicio de la palabra

La función `tiene` decide si una raíz aparece en el nombre de un negocio. Sin la condición de inicio de palabra, «MICHELADAS» contendría «HELAD» (helados) y «BARBACOA» contendría «BAR». Algunas raíces cortas tienen que ser la palabra completa:

```
PALABRA_COMPLETA = {"BAR", "PUB", "CAFE", "CARNE", "PASTA", "CHINA", "CHINO", "VILLA"}

def tiene(texto, raices):
    """¿Alguna raíz aparece al INICIO de una palabra del nombre?"""
    return any(re.search(r"(?<![A-Z0-9])" + re.escape(r)
                         + (r"(?![A-Z])" if r in PALABRA_COMPLETA else ""), texto)
               for r in raices)

tiene("MICHELADAS EL GUERO", ["HELAD"])   # False
tiene("BAR LA PALAPA", ["BAR"])           # True
```

Notebook 3, sección 6 · torre.campana.lugares.tiene

6. Aprendizaje de máquina: predecir

Materia: Aprendizaje de Máquina · **Profesor:** Raúl Jorge Domínguez Vilchis · **Criterio 4 de la rúbrica (12 %):** construye, valida, compara e interpreta modelos con métricas adecuadas y justifica la selección.

Conexión con la materia. El incidente crítico de esta UCA pide desarrollar "modelos de predicción, clasificación o agrupamiento" para detectar patrones de saturación e identificar temporadas, pero advierte que "un modelo con alta precisión no necesariamente produce una solución sustentable" y que los modelos pueden "favorecer a los lugares más conocidos". Por eso este capítulo tiene las dos tareas clásicas del curso (**clasificación** y **regresión**), compara cada modelo contra una línea base, valida sin dejar que el modelo "vea el futuro" y reporta el sesgo hacia los lugares con más datos.

El proyecto tiene dos problemas de aprendizaje de máquina distintos, y conviene no confundirlos:

	Clasificación (Radar)	Regresión de series de tiempo (Pronóstico)
Qué predice	Un estado: tranquilo, concurrido o saturado	Un número: cuántos visitantes
Horizonte	El mes siguiente	De 1 a 12 meses
Modelos	Logística, Random Forest y Gradient Boosting, con y sin el año anterior	Línea base, Holt-Winters (2), regresión con clima, Gradient Boosting
Contra qué se compara	"Igual que este mes"	"El mismo mes del año pasado"
Métricas	Aciertos, cambios anticipados, F1, kappa, Brier, log-loss, McNemar	MAE, MAPE, cobertura del rango

6.1 Clasificación: ¿qué estado tendrá cada lugar el mes siguiente?

Unidad de análisis: un lugar en un mes. **Variable objetivo (y):** el estado del índice comparable el mes siguiente. **Variables de entrada (x, lo que el modelo ve):**

1. El índice de este mes y de los dos anteriores (IPT_t, IPT_{t-1}, IPT_{t-2}) y el cambio ($\text{IPT}_t - \text{IPT}_{t-1}$).
2. El mes del año como un punto en un círculo: $\sin\left(\frac{2\pi m}{12}\right)$ y $\cos\left(\frac{2\pi m}{12}\right)$.
3. Si es uno de los cinco lugares de la campaña (1 o 0).

Cuatro rasgos del año anterior (decisión 34): el índice del mes objetivo un año antes (ipt_11), el de este mes un año antes (ipt_12), cuánto va arriba o abajo del año pasado (anual_dif) y la «pista» ipt_11 + anual_dif. Se buscan por calendario; si no hay dato, quedan vacíos y no se rellenan.

¿Por qué el mes en un círculo? Si el mes entrara como el número 1 a 12, el modelo creería que diciembre (12) está lejos de enero (1), cuando son vecinos. Al ponerlo en un círculo, como las horas de un reloj, diciembre y enero quedan juntos.

Datos: 468 casos para el modelo final: 234 tranquilos, 185 concurridos y 49 saturados.

Los modelos

Línea base (persistencia): $\hat{y}_{t+1} = y_t$, es decir, "el mes que entra será igual que este". Un modelo que no le gane a esto no sirve.

Regresión logística multiclase.

> **La idea en palabras**
>
> El modelo le pone un puntaje a cada estado (tranquilo, concurrido, saturado) sumando las variables, cada una multiplicada por su peso. Después convierte los tres puntajes en probabilidades que suman 100 %, con una función llamada *softmax*: el estado con más puntaje se lleva la mayor probabilidad.

Paso 1, el puntaje de cada clase:

$$z_k = b_k + \beta_{k,1}x_1 + \beta_{k,2}x_2 + \cdots + \beta_{k,7}x_7$$

Paso 2, convertir puntajes en probabilidades (softmax):

$$P(y = k \mid x) = \frac{e^{z_k}}{e^{z_{\text{tranq}}} + e^{z_{\text{conc}}} + e^{z_{\text{sat}}}}$$

Símbolo	Qué significa
k	Una de las tres clases: tranquilo, concurrido o saturado
$x_1, \ldots, x_7$	Las siete variables de entrada, ya estandarizadas
$\beta_{k,j}$	Peso de la variable j para la clase k: cuánto la empuja hacia esa clase
b_k	Término independiente (el puntaje "de partida" de la clase)
e^z	El número $e \approx 2.718$ elevado al puntaje; siempre es positivo

Por qué funciona. Como e^z siempre es positivo y se divide entre la suma de los tres, las probabilidades quedan entre 0 y 1 y suman exactamente 1. Los pesos β se estiman por **máxima verosimilitud**: se buscan los que hacen más probables los estados que de verdad ocurrieron. Dos detalles: las variables se estandarizan $((x - \bar{x})/\sigma)$ para que los pesos sean comparables entre sí, y se usan **pesos de clase balanceados**, porque "saturado" es raro (49 de 468) y sin ellos el modelo casi nunca lo predeciría.

> **Paso a paso con nuestros datos: Mahahual, predicción para agosto de 2026**
>
> **Paso 1, estandarizar.** El índice de julio fue 0.731; la media de entrenamiento es 0.2762 y la desviación estándar 0.2799:

$$x_{\text{std}} = \frac{0.731 - 0.2762}{0.2799} = 1.625$$

> Así con las siete variables: (1.625, 1.362, 1.623, 0.871, −0.803, −1.158, −0.657).
>
> **Paso 2, puntaje de "saturado".** Pesos β = (1.462, 1.291, 1.560, 0.567, −0.158, 1.195, −0.016) y $b = -3.255$. Multiplicamos cada variable por su peso y sumamos:

$$z_{\text{sat}} = -3.255 + 2.376 + 1.758 + 2.532 + 0.494 + 0.127 - 1.384 + 0.011 = 2.659$$

De la misma manera: $z_{\text{conc}} = 3.068$ y $z_{\text{tranq}} = -5.728$.

Paso 3, softmax. Para que la calculadora no se desborde, a todos los puntajes se les resta el mayor (3.068); el resultado no cambia:

$$e^{0} = 1, \qquad e^{2.659-3.068} = e^{-0.409} = 0.664, \qquad e^{-5.728-3.068} = e^{-8.796} = 0.00015$$

La suma es 1 + 0.664 + 0.00015 = 1.664. Entonces:

$$P(\text{concurrido}) = \frac{1}{1.664} = 0.601, \qquad P(\text{saturado}) = \frac{0.664}{1.664} = 0.399, \qquad P(\text{tranquilo}) \approx 0$$

Qué significa: con la regresión logística, Mahahual seguiría concurrido, con 40 % de probabilidad de saturarse. El modelo elegido (Random Forest + año anterior, abajo) da 30 % de probabilidad de saturarse, y eso es lo que muestra la página.

Qué variable pesa más (promedio del tamaño de β en las tres clases): el índice del mes (1.07), el de dos meses antes (1.04), el del mes anterior (0.95), la temporada como coseno (0.80), el cambio (0.40) y ser uno de los cinco lugares (0.35).

Random Forest (Breiman, 2001): 400 árboles de decisión; cada uno se entrena con una muestra al azar de los casos y mira un subconjunto al azar de variables en cada corte. La predicción promedia lo que dicen los 400. Promediar muchos árboles distintos reduce los errores de un árbol solo, como cuando se pregunta a muchas personas en lugar de a una.

Gradient Boosting (Friedman, 2001): suma árboles pequeños uno tras otro; cada árbol nuevo se dedica a corregir los errores que dejaron los anteriores. Usamos 100 árboles con tasa de aprendizaje 0.1: $F_M(x) = \nu h_1(x) + \nu h_2(x) + \cdots + \nu h_M(x)$.

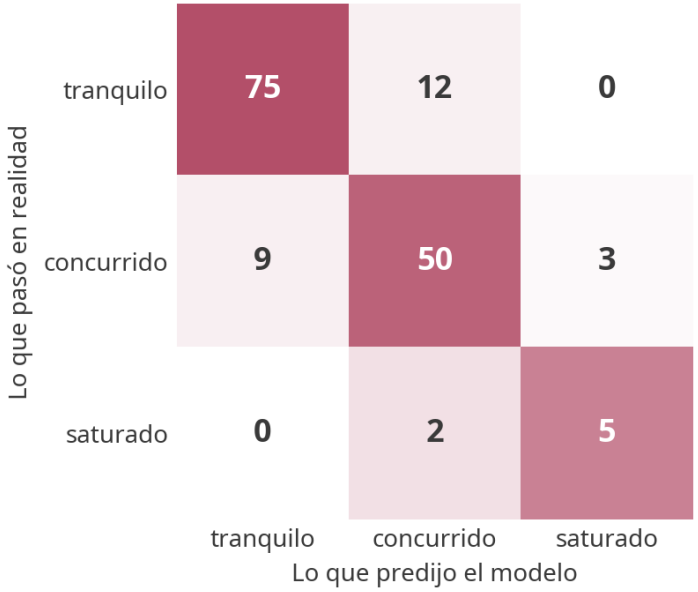
Random Forest + año anterior y HistGradientBoosting + año anterior (decisión 34): los mismos tipos de modelo, pero viendo también los cuatro rasgos del año anterior. Los dos aceptan huecos en esos rasgos, así que se prueban con los mismos casos que los demás. HistGradientBoosting agrupa los valores en histogramas para elegir los cortes; lo usamos con profundidad 3, tasa 0.05 y 200 árboles, valores conservadores para unos 300 casos de entrenamiento.

Cómo se validó: origen móvil

Por qué no revolver al azar. La forma escolar de validar (80 % para entrenar y 20 % al azar para probar) aquí sería trampa: el modelo podría entrenar con agosto y probarse con julio, es decir, ver el futuro. En series de tiempo se valida como se usaría en la vida real.

Origen móvil: para cada mes se reentrena cada modelo solo con lo anterior y se predice ese mes, en dos años distintos. Con el año de validación (agosto de 2024 a julio de 2025, 136 casos) se elige el modelo; con el año de prueba (agosto de 2025 a julio de 2026: 156 predicciones y 30 cambios reales de estado) solo se mide. Elegir con los mismos meses con que se mide premiaría al modelo que tuvo suerte en esos meses.

Las métricas



Matriz de confusión de Random Forest + año anterior, el modelo elegido, en las 156 predicciones de prueba.

$$\text{Precisión} = \frac{VP}{VP + FP}, \qquad \text{Sensibilidad} = \frac{VP}{VP + FN}, \qquad F1 = \frac{2 \times \text{Precisión} \times \text{Sensibilidad}}{\text{Precisión} + \text{Sensibilidad}}$$

Término	Qué significa (con la clase “saturado” como ejemplo)
VP, verdaderos positivos	Dijo “saturado” y sí era saturado
FP, falsos positivos	Dijo “saturado” y no lo era (falsa alarma)
FN, falsos negativos	Era saturado y no lo dijo (se le escapó)
Precisión	De lo que el modelo dijo que era esta clase, ¿cuánto sí lo era?
Sensibilidad (*recall*)	De lo que de verdad era esta clase, ¿cuánto encontró?
F1	Media armónica de las dos: un solo número que castiga si cualquiera es baja
F1 macro	El promedio del F1 de las tres clases, con el mismo peso aunque “saturado” sea rara

Paso a paso con nuestros datos: F1 del modelo elegido

En la matriz, cada renglón es lo que pasó y cada columna lo que predijo el modelo.

Tranquilo. Precisión: el modelo dijo «tranquilo» 75 + 9 + 0 = 84 veces y acertó 75: 75 ÷ 84 = 0.893. Sensibilidad: hubo 75 + 12 + 0 = 87 meses tranquilos y encontró 75: 75 ÷ 87 = 0.862. F1 = 2 · 0.893 · 0.862 ÷ (0.893 + 0.862) = 0.877.

Concurrido. Precisión 50 ÷ 64 = 0.781; sensibilidad 50 ÷ 62 = 0.806; F1 = 0.794. Saturado. Precisión 5 ÷ 8 = 0.625; sensibilidad 5 ÷ 7 = 0.714; F1 = 0.667.

F1 macro = (0.877 + 0.794 + 0.667) ÷ 3 = 0.779. Aciertos totales: 75 + 50 + 5 = 130 de 156 (83 %).

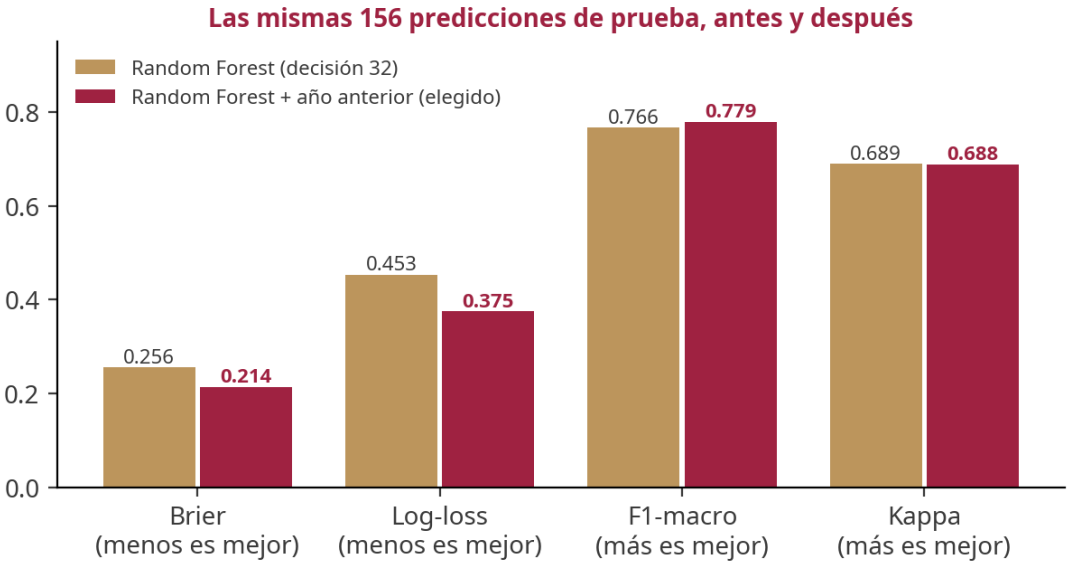
Modelo	Aciertos en la validación (de 136)	Aciertos en la prueba (de 156)	Cambios anticipados (de 30)	Falsas alarmas	F1 macro
Persistencia (línea base)	119	126	0	0	0.774
Regresión logística	112	130	8	4	0.793
Random Forest (elegido hasta la decisión 32)	122	130	6	2	0.766
Gradient Boosting	111	128	7	5	0.731
Random Forest + año anterior (elegido)	**124**	**130**	**8**	**4**	**0.779**
HistGradientBoosting + año anterior	122	131	9	4	0.835

Lo que decidimos. El criterio es «el modelo que más acierta y, a igualdad, el que más cambios anticipa», porque la regla anti-colapso necesita anticipar. Lo aplicamos en el año de validación, y el de prueba solo mide. En la validación gana **Random Forest + año anterior** (124 de 136; la persistencia 119; el mismo bosque sin el año anterior 122). En la prueba acierta 130 de 156, anticipa 8 de los 30 cambios (antes 6) con 4 falsas alarmas, y sus probabilidades mejoran 16 % (Brier 0.214 contra 0.256). HistGradientBoosting + año anterior acierta uno más en la prueba (131), pero en la validación quedó en 122: elegirlo después de ver la prueba sería elegir con la prueba, lo que corrigió la decisión 32.

Todas las métricas, antes y después

Cada métrica responde una pregunta distinta. Las calculamos en las mismas 156 predicciones de prueba para la persistencia, para el modelo elegido hasta la decisión 32 y para el elegido ahora:

Métrica	Qué pregunta responde	Persistencia	Antes	Ahora
Exactitud	¿Cuántas veces acierta?	0.808	0.833	0.833
Exactitud balanceada	Promedio de las sensibilidades: no premia acertar solo la clase común	0.774	0.793	0.794
Kappa de Cohen	Acierto por encima del azar (0 = azar, 1 = perfecto)	0.636	0.689	0.688
F1-macro	Las tres clases pesan igual	0.774	0.766	0.779
Brier (menos es mejor)	¿Qué tan buenas son las probabilidades?	0.385	0.256	0.214
Log-loss (menos es mejor)	Igual, castigando decir «casi imposible» y fallar	— (climatología 0.870)	0.453	0.375
McNemar contra la persistencia (p)	¿La ventaja es más que suerte?	—	0.14	0.19
Intervalo del 90 % de la exactitud	Bootstrap de 5,000 remuestreos	—	78.2 % a 88.5 %	78.2 % a 88.5 %



Las métricas del modelo anterior y del elegido en las mismas 156 predicciones de prueba.

Cómo se lee. La mejora principal está en las **probabilidades**: el Brier baja de 0.256 a 0.214 (16 % menos) y el log-loss de 0.453 a 0.375, contra 0.870 de la climatología (dar a cada estado su frecuencia sin ver nada del lugar). Por eso la página puede decir «tranquilo, con 94 % de probabilidad» y que ese número signifique algo. La exactitud y el kappa quedan iguales.

Lo que hay que decir con honestidad. En los 12 casos donde uno acierta y el otro no, el modelo gana 8 y la persistencia 4. Con tan pocos casos eso no es significativo (McNemar, p = 0.19), y el intervalo de la ventaja va de -1.3 a +5.8 puntos. El modelo no se justifica por ganarle en exactitud a la regla sencilla, sino porque anticipa cambios (la persistencia no anticipa ninguno) y da probabilidades útiles.

Un error que encontramos. La función log_loss de scikit-learn supone que las columnas de probabilidad vienen en orden alfabético (concurrido, saturado, tranquilo); las nuestras van en el orden de los estados. El primer cálculo dio 8.0, un número sin sentido. Ahora lo calculamos con la fórmula, a mano, y una prueba automática lo vigila.

> **Paso a paso con nuestros datos: el puntaje de Brier de Mahahual**
> El modelo le da a Mahahual, para agosto de 2026: tranquilo 0.02, concurrido 0.68 y saturado 0.30.
> Si queda concurrido: $(0.02 - 0)^2 + (0.68 - 1)^2 + (0.30 - 0)^2 = 0.0004 + 0.1008 + 0.0889 = 0.190$.
> Si se satura: 0.0004 + 0.4658 + 0.4926 = 0.959.
> La persistencia diría «concurrido» con 100 %: 0 si acierta y 2 si falla. El Brier promedio premia las probabilidades honestas: decir 30 % cuando hay riesgo real cuesta poco si pasa lo otro.

Métricas de impacto: lo que le importa a la campaña

La pregunta de la materia no es solo «¿qué tan exacto?», sino «¿sirve para un turismo más sustentable?». En el año de prueba hubo 7 meses saturados y el modelo detectó 5; solo habría pausado 3 anuncios sin necesidad en 156 lugar-meses (antes, 4). Pero las 2 saturaciones nuevas (un lugar que pasa a saturado desde otro estado) no las anticipó. Esa es la limitación más importante para la regla anti-colapso, y por eso la regla no depende solo de este modelo: la Torre en vivo vigila cada semana las llegadas reales y pausa el anuncio en cuanto un lugar rebasa lo esperado.

¿Qué rasgos usa el modelo? Importancia por permutación

Revolvemos un rasgo a la vez en los casos de prueba y medimos cuánto baja la exactitud (30 revueltas por rasgo). Es más honesta que la importancia por impureza de los árboles, que favorece a las variables continuas:

Rasgo	Qué es	Baja la exactitud (puntos)
ipt_11	El mes objetivo, un año antes	1.71
pista_11	El año pasado corrido al nivel de hoy	1.67
ipt_comparable	El índice de este mes	1.32
ipt_1	El índice del mes anterior	0.71
es_de_los_5	Si es uno de los 5 lugares	0.43
mes_cos	La temporada (coseno)	0.38

Los dos rasgos que más pesan son del **año anterior**: julio de este año se parece más a julio del año pasado del mismo lugar que a «un julio cualquiera». La temporada como círculo casi no aporta, porque el año anterior ya la trae.

Cómo lo programamos: los rasgos, los modelos y los dos años

Para cada lugar se calculan los rasgos con `shift`: `shift(1)` trae el valor del mes anterior y `shift(-1)` el del mes siguiente, que es lo que se quiere predecir. Los tres modelos se crean de nuevo en cada reentrenamiento, con los hiperparámetros fijos:

```
for _, g in t.groupby("lugar"):
    g["ipt_1"], g["ipt_2"] = g.ipt_comparable.shift(1), g.ipt_comparable.shift(2)
    g["cambio"] = g.ipt_comparable - g.ipt_1
    g["objetivo"] = g.estado.shift(-1)                    # el estado del mes siguiente
d["mes_sin"] = np.sin(2 * np.pi * d.periodo.dt.month / 12)   # diciembre junto a enero
d["mes_cos"] = np.cos(2 * np.pi * d.periodo.dt.month / 12)

def modelos():
    return {"Regresión logística": make_pipeline(StandardScaler(),
                LogisticRegression(max_iter=2000, class_weight="balanced")),
            "Random Forest": RandomForestClassifier(n_estimators=400, min_samples_leaf=3,
                class_weight="balanced", random_state=0),
            "Gradient Boosting": GradientBoostingClassifier(random_state=0)}
```

Notebook 4, sección 1 · torre.radar.prediccion

El origen móvil reentrena con todo lo anterior a cada mes y predice solo ese mes. La misma función se usa en el año de validación (donde se elige) y en el de prueba (donde solo se mide); la elección ordena por aciertos y, a igualdad, por cambios anticipados:

```
def origen_movil(desde, hasta):
    for mes in meses_entre(desde, hasta):
        ent = d[d.periodo_objetivo < mes]                 # solo el pasado
        pru = d[d.periodo_objetivo == mes]                # el mes que se predice
        for nombre, m in modelos().items():
            m.fit(ent[RASGOS], ent.objetivo)
            predicciones[nombre].append(m.predict(pru[RASGOS]))
    ...                                                   # aciertos, cambios, falsas alarmas

val = origen_movil("2024-08-01", "2025-08-01")            # validación: aquí se ELIGE
```

```
pru = origen_movil("2025-08-01", "2026-08-01")            # prueba: aquí solo se MIDE
mejor = val.sort_values(["aciertos", "cambios_acertados"], ascending=False).index[0]
```

Notebook 4, sección 2 · torre.radar.prediccion.origen_movil

Cómo lo programamos: la matriz de confusión

`confusion_matrix` cruza lo que pasó (renglones) con lo que predijo el modelo (columnas) en los 156 casos de prueba; `f1_score` con `average=None` da el F1 de cada estado, y su promedio es el F1-macro:

```
from sklearn.metrics import confusion_matrix, f1_score
ESTADOS = ["tranquilo", "concurrido", "saturado"]
mc = confusion_matrix(y_prueba, prediccion_rf, labels=ESTADOS)   # [[75 12 0] [9 50 3] [0 2 5]]
f1_por_estado = f1_score(y_prueba, prediccion_rf, labels=ESTADOS, average=None)
f1_por_estado.round(3)                                           # [0.877 0.794 0.667]
f1_por_estado.mean()                                             # F1-macro: 0.779
```

Notebook 4, sección 2

Cómo lo programamos: los rasgos del año anterior y el log-loss

Los rasgos del año anterior se buscan por calendario (el mismo lugar, 11 y 12 meses antes), no por la posición del renglón: si a un lugar le falta un mes, la posición se correría. El log-loss se calcula con la fórmula, con las columnas en el orden de los estados:

```
def agregar_anual(x, t):
    ipt = t.set_index(["lugar", "periodo"]).ipt_comparable
    for k in (11, 12):                                  # el mismo lugar, k meses antes
        atras = (x.periodo.dt.to_period("M") - k).dt.to_timestamp()
        x[f"ipt_{k}"] = ipt.reindex(pd.MultiIndex.from_arrays([x.lugar, atras])).to_numpy()
    x["anual_dif"] = x.ipt_comparable - x.ipt_12       # arriba o abajo del año pasado
    x["pista_11"] = x.ipt_11 + x.anual_dif             # el mes objetivo del año pasado, hoy
    return x

def _logloss(y, P):        # -promedio de ln(probabilidad dada a lo que pasó)
    p = np.array([P[i, ESTADOS.index(v)] for i, v in enumerate(y)])
    return float(-np.log(np.clip(p, 1e-6, 1)).mean())
```

Notebook 4, secciones 1 y 2.2 · torre.radar.prediccion

6.2 Pronóstico de visitantes: series de tiempo

Qué series se pronostican

Serie	Fuente	Meses	Meses que entrenan
Visitantes a Oxtankah (Bahía)	DataTur, base del INAH	127	90
Kohunlich + Dzibanché + Ichkabal (Ruta)	DataTur, base del INAH	127	91
Cruces desde Belice (Chetumal)	API de SITUR-Q	90	62
Ocupación de Cancún (referencia)	DataTur semanal	55	55

El sur no tiene ocupación oficial en 2025–2026, así que se pronostica solo lo medido que llega a 2026. Maya Ka'an y la Laguna no tienen serie mensual propia: hueco declarado.

Meses que no entrenan. Un mes cerrado no es "cero demanda". Si el modelo aprendiera de esos ceros, creería que en abril puede haber 0 visitantes por temporada. Quedan fuera del entrenamiento (pero se conservan): los cierres (la zona reportó 0), los meses parciales (justo antes de cerrar o después de reabrir) y la pandemia (INAH de marzo de 2020 a diciembre de 2021; Belice de marzo de 2020 a junio de 2022). Cada reapertura inicia un **tramo** con su propio nivel.

Los cinco modelos

Notación común: y_t = valor del mes t; o = el último mes conocido (el "origen"); h = cuántos meses hacia adelante; S_m = forma del año del capítulo 5, calculada solo con años anteriores al origen.

1. Línea base (ingenuo estacional): $\hat{y}_{o+h}$ = el último valor del mismo mes del año. "Este diciembre será como el diciembre pasado."

2 y 3. Holt-Winters con la forma del año fija.

> **La idea en palabras**
>
> Primero se le quita la temporada a la serie (cada mes se divide entre su índice), y queda el "nivel de fondo". Ese nivel se actualiza cada mes mezclando el dato nuevo con lo que ya se creía. Para pronosticar, al nivel se le vuelve a poner la temporada.

$$z_t = \frac{y_t}{S_{m(t)}}, \qquad \ell_t = \alpha\, z_t + (1-\alpha)\,\ell_{t-1}, \qquad \hat{y}_{o+h} = \ell_o \times S_{m(o+h)}$$

Símbolo	Qué significa
z_t	El dato del mes sin temporada ("desestacionalizado")
ℓ_t	El nivel de fondo después de ver el mes t
α	Cuánto se cree en el dato nuevo (entre 0 y 1): cerca de 1, el nivel sigue cada dato; cerca de 0, cambia lento
$\hat{y}_{o+h}$	El pronóstico: el último nivel por el índice del mes que se quiere pronosticar

La fórmula del nivel es un **promedio ponderado**: si $\alpha = 0.12$, el nivel nuevo es 12 % el dato nuevo y 88 % lo que ya se creía. α se estima minimizando el error de pronóstico a un paso. La variante con tendencia agrega una pendiente que se va apagando. Usamos la forma del año fija porque el Holt-Winters clásico necesita al menos 24 meses seguidos para estimar la temporada, y desde las reaperturas solo hay de 17 a 20.

> **Paso a paso con nuestros datos: Ruta, origen julio de 2026**
>
> El programa estimó $\alpha = 0.1183$.
>
> **Paso 1, quitar la temporada.** Julio tuvo 5,057 visitantes y su índice es 0.953: $z = 5{,}057/0.953 = 5{,}306.6$.
>
> **Paso 2, actualizar el nivel.** El nivel de junio era 5,332.5:

$$\ell_{\text{jul}} = 0.1183 \times 5{,}306.6 + 0.8817 \times 5{,}332.5 = 627.8 + 4{,}701.7 = 5{,}329.4$$

Paso 3, pronosticar enero de 2027. Se vuelve a poner la temporada de enero:

$$\hat{y} = 5{,}329.4 \times 1.606 = 8{,}559 \text{ visitantes}$$

El código da 8,559.

4. Regresión con clima (mínimos cuadrados sobre el logaritmo):

$$\ln y_t = \tau_{k(t)} + \mu_{m(t)} + \beta \cdot \frac{L_t - \bar{L}_{m(t)}}{100} + \gamma\, T_t + \varepsilon_t$$

Símbolo	Qué significa
$\ln y_t$	Logaritmo natural de los visitantes; con logaritmos los efectos se leen como porcentajes
τ_k	Nivel del tramo k (antes de 2020, después de la pandemia, desde la reapertura)
μ_m	Efecto del mes m (enero es la referencia: $\mu_1 = 0$)
$L_t - \bar{L}_m$	Lluvia del mes menos la lluvia normal de ese mes, en cientos de milímetros
T_t	1 si empezó una tormenta que afecta al sur; 0 si no
β, γ	Efecto de la lluvia y de la tormenta
ε_t	Error: lo que el modelo no explica

Los coeficientes se estiman por **mínimos cuadrados ordinarios**: se buscan los que hacen más pequeña la suma de los errores al cuadrado. En forma matricial, $\hat{\theta} = (X^{\top}X)^{-1}X^{\top}y$. Para el futuro el clima no se conoce, así que se usa el normal: lluvia anómala = 0 y tormenta = su frecuencia histórica en ese mes.

Paso a paso con nuestros datos: Bahía, diciembre de 2026

Paso 1. El tramo actual (desde la reapertura de 2024) tiene $\hat{\tau} = 7.0382$; es decir, un enero típico tiene $e^{7.0382} = 1{,}139$ visitantes.

Paso 2. El efecto de diciembre es $\hat{\mu}_{12} = 0.1404$: diciembre es $e^{0.1404} = 1.151$ veces un enero.

Paso 3. Nunca hubo tormenta en diciembre ($\hat{p} = 0$) y la lluvia se supone normal. Se suman los logaritmos y se regresa con la exponencial:

$$\hat{y} = e^{7.0382 + 0.1404} = e^{7.1786} = 1{,}311 \text{ visitantes}$$

El código da 1,311.13. Además, el coeficiente de la lluvia, $\hat{\beta} = -0.115$, dice que cada 100 mm de lluvia sobre lo normal se asocian a $e^{-0.115} - 1 = -10.9\ \%$ de visitantes (valor $p = 0.027$).

5. Gradient Boosting con rezagos: 150 árboles que aprenden de todos los pares pasados (origen, destino) con: meses hacia adelante, mes del año, logaritmo del último valor, del promedio de los últimos 3 meses y del mismo mes del año anterior.

Cómo se midió el error

Para cada mes del pasado (desde enero de 2019 en el INAH) se pronostican los 12 siguientes **usando solo lo que se sabía entonces** y se compara con lo que pasó. Solo cuentan los pares que los cinco modelos pudieron pronosticar (366 en la Bahía, 378 en la Ruta, 366 en Belice y 438 en Cancún), para que la comparación sea justa.

$$\text{MAE} = \frac{1}{n}\sum|y - \hat{y}|, \qquad \text{MAPE} = \frac{100}{n}\sum\left|\frac{y - \hat{y}}{y}\right|, \qquad \text{error relativo} = \frac{\text{MAE del modelo}}{\text{MAE de la línea base}}$$

Medida	Cómo se lee
MAE	En promedio, ¿por cuántos visitantes se equivoca?
MAPE	En promedio, ¿por qué porcentaje se equivoca?
Error relativo	Menor que 1: el modelo le gana a “el mismo mes del año pasado”

El rango del 90 %: intervalo conformal

> **La idea en palabras**
>
> Revisamos los errores que el modelo cometió en el pasado en pronósticos parecidos. Si 9 de cada 10 fueron menores que cierto tamaño, lo razonable es decir que el próximo tampoco se equivocará más que eso, con 90 % de confianza. No suponemos que los errores sigan una campana normal: usamos los errores reales (Angelopoulos y Bates, 2023).

$$k = \lceil (n+1) \times 0.9 \rceil, \qquad \hat{q} = \text{el } k\text{-ésimo error más chico}, \qquad \text{rango} = \left[\hat{y}\, e^{-\hat{q}},\ \hat{y}\, e^{\hat{q}}\right]$$

Símbolo	Qué significa
n	Cuántos errores pasados hay del mismo modelo y del mismo plazo (1–3, 4–6 o 7–12 meses)
$\lceil\ \rceil$	Redondear hacia arriba
$\hat{q}$	El tamaño de error que cubre al 90 % de los casos (en logaritmo)
$e^{\pm\hat{q}}$	Convierte ese error en un factor: el rango va de “tantas veces menos” a “tantas veces más”

La garantía: si los errores futuros se parecen a los pasados, el dato real cae dentro del rango al menos $k/(n+1)$ de las veces. El “+1” y el redondeo hacia arriba son la corrección que vuelve la garantía exacta con muestras chicas. Trabajamos en logaritmos porque los errores son proporcionales al nivel (equivocarse por 100 no es lo mismo con 1,000 visitantes que con 10,000).

> **Paso a paso con nuestros datos: Bahía, diciembre de 2026 (5 meses adelante)**
>
> **Paso 1.** Hay $n = 105$ errores pasados: $k = \lceil 106 \times 0.9 \rceil = \lceil 95.4 \rceil = 96$.
>
> **Paso 2.** Se ordenan los 105 errores de menor a mayor; el número 96 es $\hat{q} = 0.3132$. Garantía: 96/106 = 90.6 %.
>
> **Paso 3.** Con el pronóstico de 1,311.13:

$$\text{mínimo} = 1{,}311.13 \times e^{-0.3132} = 1{,}311.13 \times 0.7311 = 959$$

$$\text{máximo} = 1{,}311.13 \times e^{0.3132} = 1{,}311.13 \times 1.3678 = 1{,}793$$

Comprobación: diciembre de 2025 tuvo 1,224 visitantes, dentro del rango.

La elección y los resultados

Regla: por serie, el modelo de menor error entre los que tienen un rango que se cumple al menos 80 de cada 100 veces. Descartamos "el menor error sin importar el rango" (en la Ruta elegiría un modelo cuyo rango acierta 72 de cada 100) y "un solo modelo para todo" (en Cancún la regresión comete 58 % más error que la línea base).

Serie	Modelo	MAPE	Error vs. base	Cobertura	Rango típico
Bahía	**Regresión con clima**	15.7 %	0.898	91.3 %	±43 %
	Holt-Winters sin tendencia	19.4 %	1.078	96.5 %	±63 %
	Holt-Winters con tendencia	27.6 %	1.524	87.8 %	±108 %
	Gradient Boosting	28.7 %	1.405	85.8 %	±83 %
	Línea base	19.8 %	1.000	85.4 %	±77 %
Ruta	**Regresión con clima**	21.6 %	0.737	80.3 %	±51 %
	Holt-Winters sin tendencia	20.1 %	0.639	72.3 %	±45 %
	Línea base	27.9 %	1.000	74.3 %	±63 %
Belice	**Regresión con clima**	11.0 %	0.820	94.1 %	±26 %
Cancún	**Línea base**	3.8 %	1.000	89.3 %	±10 %

El valor comercial que pide el incidente. El incidente de esta UCA pide pensar cómo la solución predictiva puede volverse un producto de negocio. El Radar y el Pronóstico ya funcionan como un servicio: cualquier hotelero o autoridad puede consultar el estado de un lugar el mes siguiente y su rango de visitantes a 12 meses. Y la predicción también decide **cuándo activar la campaña**: solo en los meses que el pronóstico marca con espacio.

Cómo lo programamos: los meses que no entrenan

Antes de cualquier modelo, cada mes de cada serie lleva su marca. Un mes en cero es un **cierre**; el mes de antes y el de después de un cierre son **meses parciales**; y los meses de la pandemia se marcan aparte. El valor no se borra: solo se marca, y `entrena_flag` dice si el mes sirve para aprender:

```
def _motivos_zona(v):                                  # serie mensual de una zona
    cerrado = v == 0
    parcial = ~cerrado & (cerrado.shift(1, fill_value=False)    # justo después de cerrar
                          | cerrado.shift(-1, fill_value=False))  # o justo antes
```

```
    return (pd.Series(None, index=v.index, dtype="object")
              .mask(parcial, "mes parcial").mask(cerrado, "cierre"))

PANDEMIA_INAH = ("2020-03-01", "2021-12-01")
s.loc[s.motivo_hueco.isna() & s.periodo.between(*PANDEMIA_INAH), "motivo_hueco"] = "pandemia"
t["entrena_flag"] = t.valor.notna() & t.motivo_hueco.isna()   # Ruta: 91 de 127 meses
```

Notebook 4, sección 3 · torre.pronostico.series

Cómo lo programamos: dos de los cinco modelos

Holt-Winters con forma fija divide el tramo actual entre la forma del año (calculada solo con años anteriores al origen), sigue el nivel con suavizamiento exponencial y vuelve a multiplicar por el índice de cada mes. La regresión con clima es un ajuste de mínimos cuadrados sobre el logaritmo, con un nivel por tramo, once columnas de mes, la lluvia sobre lo normal y la tormenta:

```
def holt_sin_tendencia(s, origen, destinos):
    S, y = forma_hasta(s, origen), tramo_actual(s, origen)    # nada posterior al origen
    z = (y / y.index.month.map(S).values).to_numpy()          # la serie sin temporada
    nivel = ExponentialSmoothing(z, initialization_method="estimated").fit().forecast(1)[0]
    return np.array([nivel * S[d.month] for d in destinos])   # se vuelve a poner la temporada

def regresion_con_clima(s, origen, destinos):
    X = pd.DataFrame(index=tramo.index)
    for k in sorted(tramo.unique()):
        X[f"tramo_{k}"] = (tramo == k).astype(float)          # un nivel por tramo
    for m in range(2, 13):
        X[f"mes_{m}"] = (X.index.month == m).astype(float)    # enero es la referencia
    X["lluvia_anom"] = (cl.lluvia_mm - cl.mes.map(normal)).to_numpy() / 100
    X["tormenta"] = cl.tormenta.to_numpy()
    beta, *_ = np.linalg.lstsq(X.to_numpy(), np.log(y), rcond=None)   # mínimos cuadrados
```

Notebook 4, sección 3 · torre.pronostico.modelos

El rango conformal toma, para cada pronóstico, los errores de pronósticos anteriores cuyo mes ya había pasado, del mismo modelo y del mismo tramo de horizonte, y usa su cuantil ⌈(n+1)·0.9⌉/n. Con menos de 20 errores no se da rango. La elección toma el menor error entre los modelos con cobertura de al menos 80 %:

```
def cuantil_conformal(errores, nivel=0.90):
    n = len(errores)
    return np.sort(errores)[min(math.ceil((n + 1) * nivel), n) - 1]

b["error_log"] = np.abs(np.log(b.pronostico / b.real))     # el error como "± %"
for _, g in b.groupby(["serie", "modelo", "tramo_h"]):
    for i, o in zip(g.index, g.origen):
        cal = g.error_log[g.destino <= o]                  # errores ya conocidos en o
        if len(cal) >= 20:
            b.loc[i, "q_log"] = cuantil_conformal(cal.to_numpy())
# el rango es [pronóstico × e^(−q), pronóstico × e^(q)]
elegido = tabla[tabla.cobertura_pct >= 80].groupby("serie").mae.idxmin()
```

Notebook 4, secciones 4 y 5 · torre.pronostico.intervalos y torre.pronostico.seleccion

Cómo lo programamos: la línea base, el Gradient Boosting y el error

La línea base es la respuesta sencilla que hay que vencer: cada mes vale lo mismo que ese mismo mes la última vez que hubo dato. El Gradient Boosting aprende de todos los pares pasados (mes de origen

→ mes destino) con cinco rasgos. El error se mide con el MAE y el MAPE, y la columna *vs base* divide el MAE de cada modelo entre el de la línea base:

```
def ingenuo_estacional(s, origen, destinos):
    x = s[(s.periodo <= origen) & s.entrena_flag]
    ultimo = x.sort_values("periodo").groupby(x.periodo.dt.month).valor.last()
    return np.array([ultimo.get(d.month, np.nan) for d in destinos])  # marzo 2026 = marzo 2025

def _rasgos(x, o, d):                                    # horizonte, mes, último valor,
    antes = x[x.index <= o]                              # promedio de 3 y mismo mes anterior
    mismo = antes[antes.index.month == d.month]
    return [(d.year - o.year) * 12 + d.month - o.month, d.month, np.log(antes.iloc[-1]),
            np.log(antes.iloc[-3:].mean()), np.log(mismo.iloc[-1]) if len(mismo) else np.nan]
m = HistGradientBoostingRegressor(max_iter=150, learning_rate=0.05, min_samples_leaf=20,
                                  random_state=0)         # aprende despacio: hay pocos datos

b["error_abs"] = (b.pronostico - b.real).abs()
mae = b.groupby(["serie", "modelo"]).error_abs.mean()
mae_vs_base = mae / mae.xs("Línea base (ingenuo estacional)", level="modelo")  # Ruta: 0.737
```

Notebook 4, sección 3 · torre.pronostico.modelos

¿Y si combinamos los modelos?

Una forma clásica de mejorar un pronóstico es combinar varios modelos (Bates y Granger, 1969): sus errores se cancelan en parte. Lo probamos con los mismos pronósticos del origen móvil, en los casos donde los cinco modelos tienen pronóstico. MAPE de cada opción:

Serie	Modelo elegido	Mediana de los 5	Promedio de 3 sencillos
Bahía Calderitas–Oxtankah	Regresión con clima · 15.65 %	15.12 %	15.58 %
Chetumal	Regresión con clima · 10.99 %	10.92 %	11.15 %
Ruta arqueológica del sur	Regresión con clima · 21.62 %	21.03 %	20.86 %

Combinar mejora a lo más 0.6 puntos de MAPE. Cambiar de modelo por eso obligaría a rehacer el pronóstico, el Monte Carlo, el presupuesto y la página por casi nada, así que la elección se queda. El resultado sirve de otra forma: es la prueba de que el modelo elegido ya está cerca de lo mejor que permiten estos datos (torre.pronostico.seleccion.combinaciones).

7. Modelos estocásticos: la incertidumbre

Materia: Modelos Estocásticos para la Toma de Decisiones · **Profesor:** René Chablé García · **Criterio 5 de la rúbrica (10 %):** incorpora la incertidumbre, genera escenarios pertinentes y analiza sus implicaciones para decidir.

Conexión con la materia. El incidente crítico de esta UCA pregunta: *"¿Qué pasaría si la campaña funciona mejor o peor de lo esperado?"*. Pide intervalos de confianza para la demanda, escenarios optimista, moderado y pesimista con su probabilidad, simulación de **Monte Carlo** para el riesgo de rebasar la capacidad de carga, análisis de sensibilidad y comunicar la incertidumbre a personas no técnicas. Aplicamos tres procesos estocásticos del curso: una **cadena de Markov** (cómo cambia de estado la ocupación del norte semana a semana), un **proceso de Poisson** (cuántas tormentas llegan cada mes) y una **simulación de Monte Carlo** (cuántos visitantes podrían llegar en los próximos 12 meses).

Un **proceso estocástico** es algo que evoluciona en el tiempo con una parte de azar. No sabemos exactamente qué va a pasar, pero sí con qué probabilidad puede pasar cada cosa.

7.1 Cadena de Markov semanal del norte

Pregunta. Si Cancún está concurrido esta semana, ¿qué probabilidad hay de que se sature en las próximas semanas? Cuando el norte se va a saturar es el momento de ofrecerle el sur a quien iba allá.

La idea en palabras

Una cadena de Markov es como un juego de tablero en el que la siguiente casilla depende solo de la casilla donde estás, no de cómo llegaste. Aquí las casillas son tranquilo, concurrido y saturado, y las probabilidades de moverse de una a otra se cuentan con lo que pasó en 239 semanas reales.

Datos. 7 centros de DataTur (Cancún, Riviera Maya, Playa del Carmen, Playacar, Akumal, Cozumel e Isla Mujeres) × 239 semanas. Cada semana se clasifica con los percentiles de la ocupación de todas las semanas de los 7 centros: tranquilo por debajo de 71.16 %, saturado por encima de 85.92 %.

La propiedad de Markov ("solo importa el presente"):

$$P(X_{t+1} = j \mid X_t = i,\ X_{t-1},\ \ldots) = P(X_{t+1} = j \mid X_t = i) = p_{ij}$$

Cómo se estima cada probabilidad (contar y dividir):

$$\hat{p}_{ij} = \frac{n_{ij}}{n_{i,\text{tranq}} + n_{i,\text{conc}} + n_{i,\text{sat}}}$$

Símbolo	Qué significa
X_t	El estado del centro turístico en la semana t
p_{ij}	Probabilidad de pasar del estado i (esta semana) al estado j (la siguiente)
n_{ij}	Cuántas veces, en los datos, un centro pasó de i a j entre dos semanas seguidas
P	La matriz de transición: una tabla de 3 × 3 con todas las p_{ij}; cada renglón suma 1

Por qué "contar y dividir" es lo correcto. Es el estimador de máxima verosimilitud: de todas las matrices posibles, es la que hace más probable haber observado exactamente las transiciones que vimos, con la condición de que cada renglón sume 1.

Paso a paso con nuestros datos: la matriz de transición

Esta semana	Conteo → tranq.	→ conc.	→ sat.	Prob. → tranq.	→ conc.	→ sat.
Tranquilo	765	65	0	0.922	0.078	0.000
Concurrido	67	555	46	0.100	0.831	0.069
Saturado	2	44	122	0.012	0.262	0.726

Un renglón. Desde "tranquilo" hubo 765 + 65 + 0 = 830 transiciones; 765 se quedaron en tranquilo: 765 ÷ 830 = 0.922.

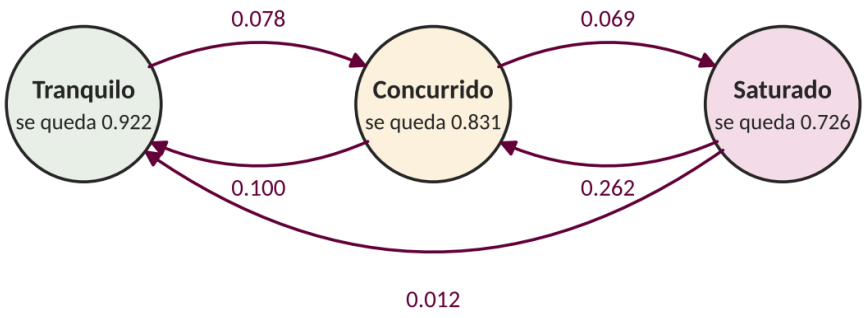


Diagrama de la cadena: cada flecha es la probabilidad de pasar de un estado a otro en una semana.

Varias semanas adelante (Chapman-Kolmogorov)

Para saber qué pasa en dos semanas, hay que considerar **todos los caminos** posibles por la semana intermedia y sumarlos. Eso es exactamente multiplicar la matriz por sí misma: la matriz a k semanas es P^k.

$$p^{(2)}_{\text{conc, sat}} = p_{\text{conc,tranq}}\, p_{\text{tranq,sat}} + p_{\text{conc,conc}}\, p_{\text{conc,sat}} + p_{\text{conc,sat}}\, p_{\text{sat,sat}}$$

Paso a paso con nuestros datos: de concurrido a saturado en dos semanas

Hay tres caminos por la semana intermedia:

Camino 1, concurrido → tranquilo → saturado: $0.100 \times 0.000 = 0$.

Camino 2, concurrido → concurrido → saturado: $0.831 \times 0.069 = 0.057$.

Camino 3, concurrido → saturado → saturado: $0.069 \times 0.726 = 0.050$.

Suma: $0 + 0.057 + 0.050 = 0.107$. Un centro concurrido tiene **10.7 %** de probabilidad de estar saturado dentro de dos semanas. Desde tranquilo, a 8 semanas (P^8), es 5.4 %.

A largo plazo: la distribución estacionaria

Si se deja correr la cadena muchas semanas, la proporción de tiempo en cada estado se estabiliza en un vector π que ya no cambia al multiplicarlo por la matriz:

$$\pi = \pi P, \qquad \pi_{\text{tranq}} + \pi_{\text{conc}} + \pi_{\text{sat}} = 1$$

Paso a paso con nuestros datos: comprobación de la estacionaria

Resolviendo el sistema se obtiene $\pi = (0.513,\ 0.389,\ 0.098)$. Comprobamos la primera componente de πP:

$$0.513 \times 0.922 + 0.389 \times 0.100 + 0.098 \times 0.012 = 0.473 + 0.039 + 0.001 = 0.513 ✓$$

Qué significa: a la larga, el norte pasa el **9.8 % de las semanas saturado**.

Cuánto dura una racha. El número de semanas seguidas en un estado sigue una distribución geométrica con media $1/(1 - p_{ii})$. Una racha saturada dura en promedio $1/(1 - 0.726) = 1/0.274 = 3.65$ semanas.

¿Sirve? El puntaje de Brier

Para evaluar probabilidades (no solo aciertos) se usa el puntaje de Brier (1950): el error al cuadrado entre la probabilidad que se dio y lo que pasó.

$$B = \frac{1}{N}\sum_{n=1}^{N}\sum_{j}\left(p_{n,j} - o_{n,j}\right)^2$$

donde $p_{n,j}$ es la probabilidad que el modelo le dio al estado j en el caso n, y $o_{n,j}$ vale 1 si ese fue el estado real y 0 si no. Cero es perfecto; más bajo es mejor. Por ejemplo, si un centro está concurrido, la cadena da (0.100, 0.831, 0.069); si la semana siguiente resultó saturada, el puntaje de ese caso es $(0.100 - 0)^2 + (0.831 - 0)^2 + (0.069 - 1)^2 = 0.010 + 0.691 + 0.867 = 1.567$, un error grande porque le dio poca probabilidad a lo que pasó.

Lo probamos en las últimas 52 semanas, estimando la matriz cada semana solo con las transiciones anteriores:

Horizonte	Brier, Markov	Brier, persistencia	Brier, frecuencia simple
1 semana	**0.200**	0.225	0.544
4 semanas	**0.389**	0.484	0.549
8 semanas	**0.490**	0.676	0.555

La cadena da mejores probabilidades en los tres horizontes. Su valor está en la probabilidad (“14 % de riesgo de saturarse”), no en una etiqueta.

Cómo lo programamos: estados, matriz y Brier

Cada semana se clasifica con los percentiles 50 y 90 de toda la ocupación semanal. Las transiciones se cuentan solo entre semanas consecutivas del mismo centro, y la matriz es contar y dividir cada renglón entre su total. Para varias semanas adelante se eleva la matriz a la potencia k:

```
u50, u90 = np.quantile(o.ocupacion_pct, (0.50, 0.90))       # 71.2 % y 85.9 %
o["estado"] = np.select([o.ocupacion_pct < u50, o.ocupacion_pct <= u90],
                        ["tranquilo", "concurrido"], "saturado")
sig = o.groupby("centro").shift(-1)                          # la semana siguiente
consecutiva = (sig.semana - o.semana) == pd.Timedelta(days=7)
n = pd.crosstab(o.estado[consecutiva], sig.estado[consecutiva])   # conteos n_ij
P = n.div(n.sum(axis=1), axis=0)                             # p_ij = n_ij / Σ_j n_ij

def a_k_semanas(estado, k):
    pi = np.array([e == estado for e in ESTADOS], dtype=float)
    return pi @ np.linalg.matrix_power(P.to_numpy(), k)        # Chapman-Kolmogorov

brier = ((q - real) ** 2).sum()                              # por predicción; 0 es perfecto
```

Notebook 5, sección 3 · torre.radar.markov

Cómo lo programamos: la estacionaria y la prueba de 52 semanas

La distribución estacionaria es el vector que no cambia al multiplicarlo por la matriz: el vector propio de valor propio 1, normalizado para que sume 1. La prueba recorre las últimas 52 semanas; en cada una, la matriz se estima solo con las transiciones que ya se conocían y se compara con la persistencia y la frecuencia simple:

```
val, vec = np.linalg.eig(P.to_numpy().T)                # π = π P
pi = np.real(vec[:, np.argmin(np.abs(val - 1))])
pi = pi / pi.sum()                     # tranquilo 0.513, concurrido 0.389, saturado 0.098

for semana in ultimas_52_semanas:
    pasado = tr[tr.semana + pd.Timedelta(days=7) <= semana - pd.Timedelta(weeks=k - 1)]
    Pk = np.linalg.matrix_power(matriz(pasado).to_numpy(), k)  # solo lo que ya se sabía
    pred = {"Markov": hoy @ Pk, "Persistencia": hoy, "Frecuencia simple": frecuencias}
    for nombre, q in pred.items():
        brier[nombre].append(((q - real) ** 2).sum())  # k = 1: Markov 0.200, persistencia 0.225
```

Notebook 5, sección 3 · torre.radar.markov.backtest

7.2 Tormentas: definición y distancia

Una tormenta “afecta al sur” si un punto de su trayectoria pasa a 200 km o menos de Chetumal, con viento de al menos 34 nudos (tormenta tropical o huracán), de 1966 en adelante (era satelital).

Probamos alternativas: 100 km daba 13 eventos (muy pocos para estimar una tasa por mes) y 300 km daba 59 (incluye tormentas que no traen viento fuerte al sur).

La distancia sobre la Tierra se calcula con la **fórmula de haversine**, que mide la distancia por la superficie de una esfera (no en línea recta a través de ella):

$$d = 2R\arcsin\sqrt{\sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1\cos\varphi_2\sin^2\left(\frac{\Delta\lambda}{2}\right)}$$

donde $R = 6{,}371$ km es el radio de la Tierra, φ la latitud y λ la longitud, en radianes. Ejemplo: el huracán Carmen pasó el 2 de septiembre de 1974 por (18.6°, −88.2°); Chetumal está en (18.5°, −88.3°). Con $\Delta\varphi = \Delta\lambda = 0.1°$ la fórmula da **15.3 km**: con 125 nudos, es evento. En total hubo **31 eventos en 60 temporadas** (1966–2025).

Cómo lo programamos: la distancia y la definición de evento

La función `eventos` aplica la definición completa y deja una fila por tormenta, con el mes de su primer punto dentro del radio. Cambiar el radio es cambiar un número, y así se comparan 100, 200 y 300 km (13, 31 y 59 eventos):

```
def km_haversine(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, (lat1, lon1, lat2, lon2))
    a = (np.sin((lat2 - lat1) / 2) ** 2
         + np.cos(lat1) * np.cos(lat2) * np.sin((lon2 - lon1) / 2) ** 2)
    return 2 * 6371 * np.arcsin(np.sqrt(a))                    # radio de la Tierra: 6,371 km

h["km"] = km_haversine(h.lat, h.lon, 18.50, -88.30)            # distancia a Chetumal

def eventos(radio=200, viento=34, desde=1966):
    cumple = (h.km <= radio) & (h.viento_kt >= viento) & (h.anio >= desde)
    x = h[cumple].sort_values(["fecha", "hora_utc"])
    return x.groupby("id_tormenta").agg(anio=("anio", "first"), mes=("mes", "first"))
```

Notebook 5, sección 1 · torre.base.silver_huracanes

7.3 Poisson: la probabilidad de tormenta en cada mes

> **La idea en palabras**
>
> Las tormentas son eventos raros que llegan sin aviso, y una no depende de la otra. La distribución de Poisson es la que describe cuántos eventos así ocurren en un periodo, y solo necesita un número: cuántos llegan en promedio.

$$P(N_m = k) = \frac{e^{-\lambda_m}\lambda_m^k}{k!}, \qquad \hat{\lambda}_m = \frac{\text{eventos que empezaron en el mes } m}{60 \text{ temporadas}}, \qquad P(N_m \geq 1) = 1 - e^{-\hat{\lambda}_m}$$

Símbolo	Qué significa
N_m	Número de tormentas en el mes m (variable aleatoria)
λ_m	Promedio de tormentas en el mes m por temporada
$k!$	Factorial: $3! = 3 \times 2 \times 1 = 6$
$P(N_m \geq 1)$	Probabilidad de que haya al menos una tormenta en el mes

Por qué "1 menos ninguna". Es más fácil calcular la probabilidad de que no haya ninguna tormenta: con $k = 0$, la fórmula da $e^{-\lambda}\lambda^0/0! = e^{-\lambda}$. "Al menos una" es todo lo demás: $1 - e^{-\lambda}$.

Por qué Poisson. Si un mes se divide en muchísimos instantes muy cortos y en cada uno puede empezar una tormenta con una probabilidad muy pequeña e independiente, el total sigue una binomial que, en el límite, es una Poisson (Ross, 2019).

¿Se vale usar Poisson? Lo comprobamos con los datos. En una Poisson la varianza es igual al promedio. En los 60 años el promedio es 0.517 tormentas por año y la varianza 0.559: el índice de dispersión es 1.08 y la prueba con la ji cuadrada (63.8 con 59 grados de libertad) da p = 0.31, así que no hay evidencia de que la varianza supere al promedio. Los años con 0, 1, 2 y 3 tormentas fueron 36, 19, 3 y 2; Poisson esperaría 35.8, 18.5, 4.8 y 0.8. Por eso no hizo falta una binomial negativa.

¿Y el cambio climático? En 1966–1995 hubo 10 eventos y en 1996–2025, 21. Si la tasa fuera la misma, cada evento caería en cualquiera de las dos mitades con 50 % de probabilidad; la prueba binomial da p = 0.071, que no es significativa al 5 %. Nos quedamos con los 60 años y lo declaramos como límite: con solo los últimos 30, la probabilidad de agosto subiría de 13.9 % a 23.4 %.

¿Qué tan precisa es cada probabilidad? Con pocos eventos por mes, cada promedio tiene incertidumbre. Su intervalo exacto al 90 % (método de Garwood, con la ji cuadrada) da para agosto entre 7.5 % y 23.0 %. En los meses sin ningún evento el modelo da 0 %, pero no haber visto tormentas en 60 años no prueba que nunca haya: el límite superior es 3/60 (la regla del 3), hasta 4.9 %. El cálculo completo está en el notebook 5 y en el Excel 5_Poisson_tormentas.xlsx, con fórmulas.

Paso a paso con nuestros datos: agosto y el año completo

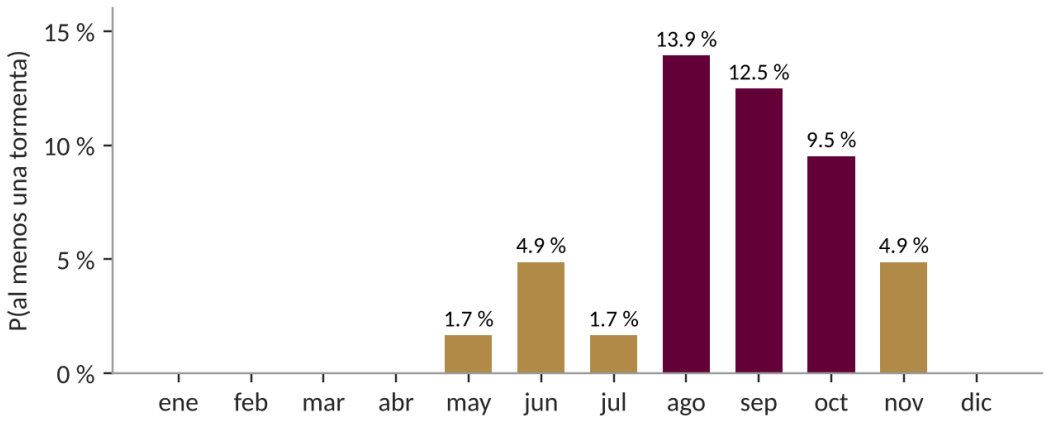
Paso 1. Agosto tuvo 9 eventos en 60 temporadas: $\hat{\lambda}_8 = 9/60 = 0.15$ tormentas por agosto.

Paso 2. Probabilidad de ninguna: $e^{-0.15} = 0.8607$.

Paso 3. Al menos una: $1 - 0.8607 = 0.139$, es decir, **13.9 %**.

Septiembre: $8/60 = 0.1333$; $1 - e^{-0.1333} = 12.5\ \%$.

Todo el año: la suma de procesos de Poisson independientes también es Poisson, así que se suman los promedios: $\lambda = 31/60 = 0.5167$ y $1 - e^{-0.5167} = 1 - 0.5965 = 40.3\ \%$.



Probabilidad de al menos una tormenta que afecte al sur, por mes.

Cómo lo programamos: Poisson y sus tres pruebas

La tasa de cada mes es contar y dividir entre 60 años. Las tres pruebas usan `scipy.stats`: la de dispersión compara el estadístico D con una ji cuadrada; la de tendencia pregunta qué tan raro es ver 21 de 31 eventos en la segunda mitad si cada evento cayera en cualquiera de las dos con 50 %; y el intervalo de Garwood sale de la ji cuadrada:

```
from scipy import stats
p["eventos"] = p.mes.map(ev.mes.value_counts()).fillna(0)
p["lambda"] = p.eventos / 60                                  # 60 años: 1966–2025
p["prob_tormenta"] = 1 - np.exp(-p["lambda"])                 # agosto: 0.139

por_anio = ev.groupby("anio").size().reindex(range(1966, 2026), fill_value=0)
D = (len(por_anio) - 1) * por_anio.var() / por_anio.mean()    # 63.8
stats.chi2.sf(D, len(por_anio) - 1)                           # p = 0.31: sí es Poisson
stats.binomtest(21, 31, 0.5).pvalue                           # p = 0.071: no significativa

k = p.eventos.to_numpy()
inf = np.where(k > 0, stats.chi2.ppf(0.05, 2 * k) / 2, 0) / 60  # Garwood, 90 %
sup = stats.chi2.ppf(0.95, 2 * k + 2) / 2 / 60                # meses sin eventos: 3/60
```

Notebook 5, sección 2 y Excel 5_Poisson_tormentas.xlsx · torre.pronostico.escenarios.poisson_tormentas

Las mismas cuentas en Excel

El archivo `5_Poisson_tormentas.xlsx` repite el cálculo con fórmulas vivas, para revisarlo celda por celda. La hoja *Eventos* trae los 31 casos; *Poisson por mes* cuenta y calcula; *Prueba de dispersión* y *Tendencia* repiten las dos pruebas. Junto a cada fórmula va el valor de Python, y abriéndolo en Excel dan lo mismo:

```
Eventos      =COUNTIF(Eventos!D:D, A2)                  agosto: 9
λ            =C2/D2                                     9/60 = 0.15
P(≥1)        =1-EXP(-E2)                                0.139
P(0), P(1)   =POISSON.DIST(0, E2, FALSE), =POISSON.DIST(1, E2, FALSE)
λ inferior   =IF(C2=0, 0, CHISQ.INV(0.05, 2*C2)/2/D2)    Garwood, 90 %
λ superior   =CHISQ.INV(0.95, 2*C2+2)/2/D2
Dispersión   =(COUNT(B2:B61)-1)*VAR.S(B2:B61)/AVERAGE(B2:B61)      63.8
p-valor      =CHISQ.DIST.RT(E5, COUNT(B2:B61)-1)                   0.31
Tendencia    =MIN(1, 2*(1-BINOM.DIST(B3-1, B4, 0.5, TRUE)))        0.071
```

Excel 5_Poisson_tormentas.xlsx (lo escribe el notebook 5)

7.4 Monte Carlo: escenarios malo, probable y bueno

Pregunta del incidente: ¿y si el año sale mejor o peor de lo pronosticado? ¿Qué tan probable es rebasar lo que un lugar ha podido recibir?

> **La idea en palabras**
>
> En lugar de un solo pronóstico, la computadora imagina 10,000 años posibles. En cada uno toma el pronóstico, le aplica un año de errores reales del pasado y le sortea tormentas. Con esos 10,000 años se ve qué tan malo puede ser un año malo y qué tan probable es pasarse de la capacidad.

Para cada futuro simulado $r = 1, \ldots, 10{,}000$ y cada mes $j = 1, \ldots, 12$:

$$D_j^{(r)} = \hat{y}_j \times e^{\varepsilon_j^{(r)}} \times \left(1 - \delta \cdot \mathbf{1}\left[N_j^{(r)} \geq 1\right]\right)$$

Símbolo	Qué significa
$D_j^{(r)}$	Visitantes del mes j en el futuro simulado número r
$\hat{y}_j$	El pronóstico del modelo elegido (capítulo 6)
$\varepsilon_j^{(r)}$	Un error real del pasado, $\ln(\text{real}/\text{pronóstico})$, sorteado al azar
$N_j^{(r)}$	Tormentas sorteadas con la Poisson del mes
$\mathbf{1}[\cdot]$	Vale 1 si lo de adentro es cierto (hubo tormenta) y 0 si no
δ	Cuánto quita una tormenta: 0, 25 % o 50 % (supuesto que se prueba)

Los tres factores, en palabras: el pronóstico, por un "golpe de suerte" del pasado, por un posible golpe de tormenta.

Errores en bloque. Se copian los 12 errores de un mismo origen del pasado juntos, no mes por mes. Los meses flojos vienen juntos (si un año sale bajo, sale bajo varios meses); sortear cada mes por separado haría los años demasiado parejos y subestimaría el riesgo.

El golpe de tormenta (δ) es un supuesto. Solo hubo 3 meses con tormenta entre los meses que entrenan (Earl 2016, Franklin 2017, Lisa 2022), muy pocos para medirlo. Por eso se prueban tres valores.

Escenarios: malo, probable y bueno son los percentiles 10, 50 y 90 de los 10,000 valores de cada mes. **Riesgo de capacidad:** la proporción de futuros en los que el mes rebasa la **capacidad probada** C (el mes más alto que el lugar ya recibió):

$$\hat{p}_j = \frac{\text{futuros en los que } D_j > C}{10{,}000}$$

¿Por qué 10,000 simulaciones? El error de un riesgo estimado con R simulaciones es $\sqrt{p(1-p)/R}$. Con el riesgo de la Ruta en enero ($p = 0.3035$): $\sqrt{0.3035 \times 0.6965/10{,}000} = 0.0046$, menos de

medio punto porcentual. Más simulaciones no cambiarían ninguna decisión. Lo comprobamos con 1,000, 5,000, 10,000 y 20,000 futuros y diez semillas distintas: desde 5,000 el escenario malo del año ya no cambia. Lo que limita la precisión no es el número de futuros sino los años de errores reales que tenemos (48 orígenes en la Ruta), y así lo declaramos.

> **Paso a paso con nuestros datos: dos futuros de la Bahía en agosto de 2026**
>
> El pronóstico es 879 visitantes.
>
> **Futuro A:** sorteó un error de −0.20 y una tormenta con $\delta = 0.5$: $879 \times e^{-0.20} \times (1 - 0.5) = 879 \times 0.8187 \times 0.5 = 360$ visitantes.
>
> **Futuro B:** sorteó un error de +0.10 y no hubo tormenta: $879 \times e^{0.10} \times 1 = 879 \times 1.105 = 971$ visitantes.
>
> **Con los 10,000 futuros**, los percentiles 10, 50 y 90 son **684 / 930 / 1,202**: el escenario malo, el probable y el bueno.
>
> **Por qué el probable (930) no es el pronóstico (879):** los errores reales de la Bahía tienen sesgo (el modelo se quedó corto 6 % en la mediana) y el Monte Carlo lo hereda. Es más honesto que suponer errores centrados en cero.

Lugar (12 meses)	Malo / probable / bueno, sin golpe	Con golpe de 50 %	Riesgo de capacidad
Ruta del sur	55,811 / 62,212 / 73,884	54,751 / 61,440 / 72,628	30.9 % (cap. 10,465 al mes)
Bahía	10,106 / 11,217 / 13,085	9,915 / 11,019 / 12,777	24.2 % (cap. 1,639 al mes)
Chetumal (Belice)	551,986 / 620,304 / 671,956	535,346 / 606,138 / 660,162	38.3 % (cap. 65,792 al mes)

Lectura para decidir. Primero, las tormentas pesan en el mes, no en el año: con el golpe más duro el escenario probable del año baja solo 1–2 %, así que la reserva para tormentas es por mes (agosto a octubre), no un recorte anual. Segundo, el riesgo de capacidad está en los meses pico: diciembre de 2026 en Chetumal (29 %) y en la Bahía (14 %), y enero de 2027 en la Ruta (30 %). Empujar la campaña en esos meses podría llevar a esos lugares a recibir más de lo que nunca han recibido. Por eso el modelo de presupuesto (capítulo 8) prohíbe anunciar ahí.

Cómo lo programamos: un año de errores por futuro

Cada fila de la matriz E es un futuro y cada columna un mes. Para cada futuro se elige al azar un origen del pasado y se copian sus 12 errores juntos; si a ese origen le falta un horizonte, ese mes se sortea entre los errores del mismo tramo. La tormenta se sortea con la Poisson del mes y, si hay, quita el golpe supuesto. La semilla 2026 hace que cualquiera obtenga los mismos números:

```
rng = np.random.default_rng(2026)
def simular(pr, golpe, rng, R=10_000):
    b["e"] = np.log(b.real / b.pronostico)                          # errores reales
    por_origen = {o: g.set_index("horizonte").e.reindex(range(1, 13)).to_numpy()
                  for o, g in b.groupby("origen")}
    origenes = list(por_origen)
    E = np.array([por_origen[origenes[i]] for i in rng.integers(len(origenes), size=R)])
    for j, tramo in enumerate(tramos_de(pr.horizonte)):
        vacios = np.isnan(E[:, j])                                  # horizonte que faltaba
```

```
        E[vacios, j] = rng.choice(por_tramo[tramo], size=vacios.sum())
    D = pr.esperado_est.to_numpy()[None, :] * np.exp(E)            # pronóstico × e^error
    tormenta = rng.poisson(lam[None, :], size=D.shape) > 0         # ¿hubo tormenta?
    return np.where(tormenta, D * (1 - golpe), D)

q10, q50, q90 = np.percentile(simular(pr, 0.0, rng), [10, 50, 90], axis=0)
riesgo = (D[:, j] > capacidad_probada).mean()                      # rebasar lo ya recibido
```

Notebook 5, sección 4 · torre.pronostico.escenarios.simular

Cómo lo programamos: ¿bastan 10,000 futuros?

La prueba repite la simulación de la Ruta con 1,000, 5,000, 10,000 y 20,000 futuros y diez semillas distintas, y mide cuánto se mueve el escenario malo del año. Desde 5,000 ya no se mueve: cada futuro copia el año de errores de uno de los 48 orígenes, así que lo que limita la precisión es cuántos años de errores reales hay, no el número de futuros:

```
for R in (1_000, 5_000, 10_000, 20_000):
    p10 = [np.percentile(simular(pr, 0.0, np.random.default_rng(semilla), R=R).sum(axis=1), 10)
           for semilla in range(10)]                               # escenario malo del año, 10 semillas
    variacion = np.std(p10) / np.mean(p10) * 100                   # % del p10; desde 5,000: 0 %
```

Notebook 5, sección 4

7.5 Sensibilidad: ¿qué variable mueve más?

Se agrega una variable v a la misma regresión del capítulo 6, $\ln y_t = \tau_{k(t)} + \mu_{m(t)} + \theta v_t$. El efecto en porcentaje es $e^{\hat{\theta}} - 1$, y el **valor** p dice qué tan probable sería ver un efecto así de grande si en realidad no hubiera ninguno (por debajo de 0.05 se considera que hay evidencia).

Lugar	+100 mm de lluvia sobre lo normal	+1 peso por dólar
Bahía	−9.7 % ($p = 0.039$)	−0.2 % ($p = 0.92$)
Ruta	−0.6 % ($p = 0.92$)	+6.8 % ($p = 0.011$)
Chetumal (Belice)	−4.7 % ($p = 0.031$)	+1.2 % ($p = 0.36$)

Ejemplo, lluvia en la Bahía: $\hat{\theta} = -0.1017$, entonces $e^{-0.1017} - 1 = 0.9033 - 1 = -9.7$ % de visitantes por cada 100 mm sobre lo normal. Son asociaciones, no causas probadas. Por ejemplo, la hipótesis de que un peso barato atrae más cruces desde Belice no se sostiene con estos datos ($p = 0.36$).

Cómo lo programamos: la regresión de sensibilidad

Es la misma regresión del pronóstico, con una variable agregada. `statsmodels` da además el p-valor de cada coeficiente, que dice si el efecto se distingue del azar:

```
import statsmodels.api as sm
X = columnas_de_tramo_y_mes(serie)                                 # niveles por tramo + 11 meses
X["v"] = variable                                                  # lluvia sobre lo normal (cientos de
                                                                   # mm) o pesos por dólar
r = sm.OLS(np.log(visitantes), X).fit()
efecto_pct = (np.exp(r.params["v"]) - 1) * 100                     # Bahía, lluvia: -9.7 %
r.pvalues["v"]                                                     # 0.04
```

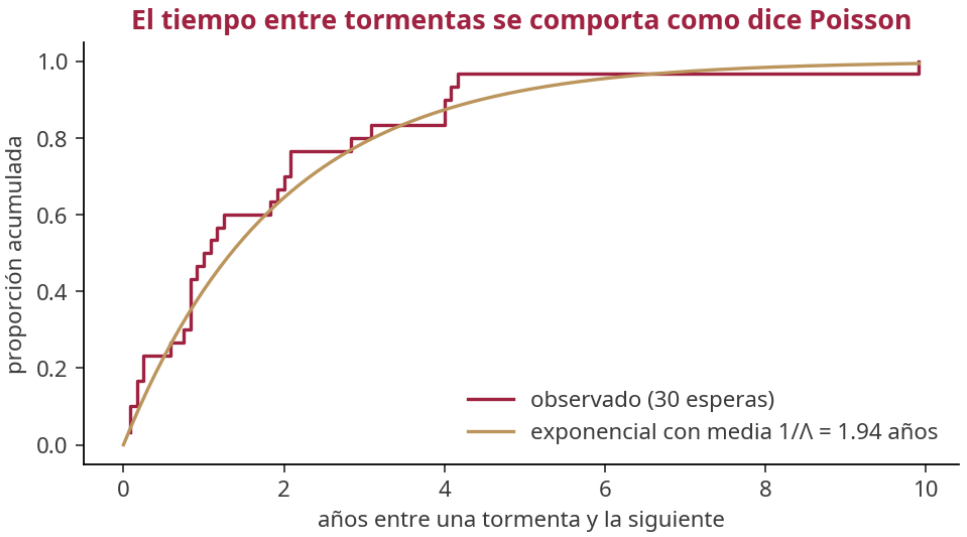
Notebook 5, sección 5 · torre.pronostico.escenarios.sensibilidad

7.6 El año completo como proceso de Poisson

Si las tormentas de cada mes son Poisson e independientes, la suma de un año también es Poisson, con tasa $\Lambda = \Sigma\, \lambda_m$ = 31 eventos ÷ 60 años = 0.517 por año. Eso permite responder preguntas sobre el año entero y comparar cada respuesta con lo que pasó en los 60 años:

Pregunta	Fórmula	Modelo	Observado (1966–2025)
¿Al menos una tormenta en el año?	1 − e^(−Λ)	40.3 %	40.0 %
¿Dos o más?	1 − e^(−Λ)(1 + Λ)	9.5 %	8.3 %
¿Cada cuánto, en promedio?	1 / Λ	1.94 años	1.77 años (espera media)
¿Tres años seguidos sin tormenta?	e^(−3Λ)	21.2 %	—

Y una propiedad que no habíamos probado: en un proceso de Poisson, el **tiempo entre una tormenta y la siguiente** sigue una distribución exponencial con media 1/Λ. Lo probamos con la prueba de Kolmogórov-Smirnov sobre los 30 intervalos entre las 31 tormentas: estadístico 0.124, p = 0.70. No se distingue de una exponencial: el supuesto de Poisson se sostiene también aquí.



El tiempo entre tormentas (escalones) contra la exponencial que predice Poisson.

Para la campaña: en un año cualquiera hay 4 de 10 probabilidades de que una tormenta obligue a pausar anuncios en algún mes; por eso el presupuesto de contingencia no es opcional (torre.pronostico.escenarios.temporada_tormentas).

7.7 La cadena a largo plazo y la prueba de la propiedad de Markov

Con la matriz P se responden tres preguntas más, todas resolviendo ecuaciones lineales: la **distribución estacionaria** $\pi = \pi P$ (la fracción de semanas en cada estado a largo plazo), la **duración media de una racha** en el estado i, $1 / (1 - p_{ii})$, porque el número de semanas seguidas es geométrico, y el **tiempo**

medio hasta la primera saturación, que resuelve (I − Q)·m = 1, con Q la matriz sin el renglón ni la columna de saturado.

Estado	π (modelo)	Frecuencia observada	Semanas al año	Racha media (semanas)	Semanas hasta saturar
tranquilo	0.513	0.500	26.7	12.77	45.9
concurrido	0.389	0.400	20.2	5.91	33.1
saturado	0.098	0.100	5.1	3.65	—

> **Paso a paso con nuestros datos: cuánto dura una saturación**
> De la matriz, un centro saturado sigue saturado la semana siguiente con probabilidad p = 0.726.
> La racha termina con probabilidad 1 − p = 0.274 cada semana; el número de semanas es geométrico, con media 1 ÷ 0.274 = 3.65 semanas.
> Para la campaña: cuando un centro del norte se satura, el mensaje «¿Ibas al norte?» tiene casi un mes de ventana.

La cadena dice que el norte pasa 9.8 % de las semanas saturado; en los datos fue 10 %: la matriz es coherente con lo observado.

¿Basta con la semana actual? Toda la cadena descansa en un supuesto: el estado de la semana siguiente depende solo del de esta semana. Contamos las ternas de semanas seguidas h → i → j y lo probamos con la razón de verosimilitud de Anderson y Goodman (1957), $G = 2\,\Sigma\, n_{hij} \cdot \ln(\hat{p}_{hij} / \hat{p}_{ij})$, que bajo el orden 1 sigue una χ^2. Resultado: G = 51.2 con 6 grados de libertad, p < 0.001. **La prueba rechaza el orden 1**: la semana anterior sí aporta algo.

Entonces probamos una cadena de **orden 2** (los estados son pares de semanas, 9 en total) en el mismo backtest de 52 semanas:

Horizonte	Brier, orden 1	Brier, orden 2	Mejora
1 semana	0.201	0.196	2.5 %
4 semanas	0.391	0.380	2.8 %
8 semanas	0.493	0.484	1.8 %

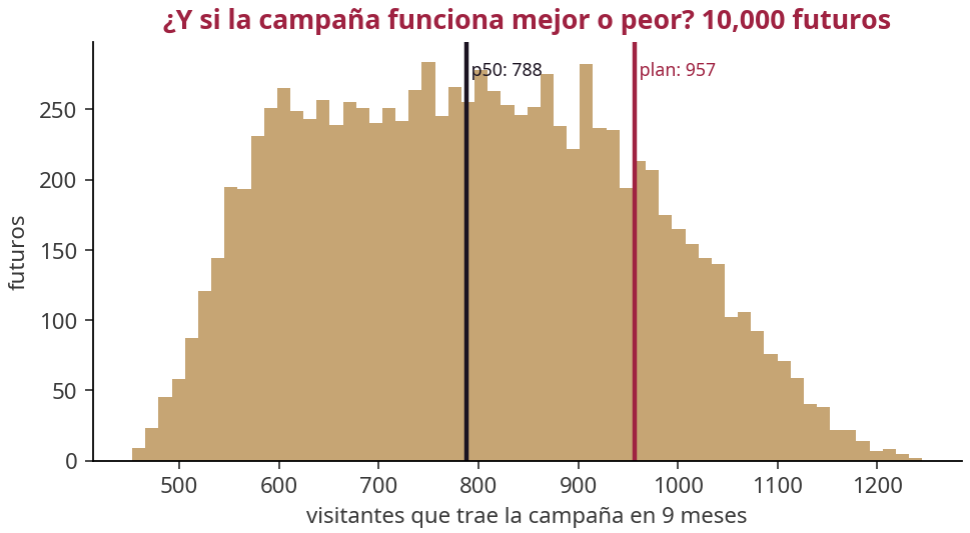
Decisión (34): nos quedamos con la cadena de orden 1. Es una matriz de 3 × 3 que se calcula a mano y se explica en el coloquio, y la de orden 2 mejora el pronóstico solo 2 a 3 %. Lo que cambia es cómo la presentamos: no decimos «el norte no tiene memoria», sino «una semana de memoria basta para pronosticar casi igual que dos» (torre.radar.markov.prueba_orden y comparar_orden2).

7.8 ¿Y si la campaña funciona mejor o peor?

Es la pregunta del incidente crítico de esta materia. El plan del capítulo 8 dice que la campaña trae 957 visitantes, pero ese número sale de un costo por clic y una conversión **fijos**. Dejamos fijo el plan de pesos y sorteamos 10,000 veces lo incierto, **cada supuesto dentro del rango que publican las fuentes**, y a cada futuro le sumamos los visitantes que llegan de todos modos (el Monte Carlo de la sección 7.4):

Supuesto	Rango sorteado (uniforme)	De dónde sale
Costo por clic de Google (Travel)	2.12 a 2.14 dólares	WordStream 2025 y LocaliQ 2026

Supuesto	Rango sorteado (uniforme)	De dónde sale
Conversión de Google (Travel)	5.75 % a 5.83 %	WordStream 2025 y LocaliQ 2026
Costo por clic de Facebook (Travel)	0.42 a 0.51 dólares	LocaliQ 2026 y WordStream 2025
Conversión de Facebook	3 % a 6.38 %	No se publica para turismo: del supuesto bajo a la mediana de todas las industrias
Pesos por dólar	un mes al azar de los últimos 24 (17.06 a 20.54)	FRED (Reserva Federal)



Visitantes que trae la campaña en 10,000 futuros, contra el plan.

Resultado	Valor
Plan (supuestos fijos)	957
Escenario malo (p10)	579
Lo más probable (p50)	788
Escenario bueno (p90)	1,016
Probabilidad de traer menos que el plan	82 %

El plan era optimista. Lo más probable son 788 visitantes, y en 8 de cada 10 futuros quedan entre 579 y 1,016. La razón: el plan usa para Facebook la conversión de Google (5.75 %), que está en la parte alta del rango, y el tipo de cambio más reciente (17.06), el más bajo de los últimos 24 meses. La variable que más mueve el resultado es la **conversión de Facebook**, porque Facebook recibe 70 % del dinero: es lo primero que hay que medir cuando la campaña empiece.

El riesgo de rebasar la capacidad casi no cambia, salvo en la Bahía de Calderitas–Oxtankah, el lugar más chico (1,639 visitantes en su mejor mes): en octubre, noviembre y febrero pasa de casi 0 a cerca de 2.5 %. La campaña no satura el sur, pero en la Bahía la Torre en vivo debe vigilar. Por eso presentamos la meta como un **rango** y no como un número fijo.

```
s = {k: rng.uniform(lo, hi, size=R) for k, (lo, hi) in rango.items()}   # cpc y conversión
s["tc"] = rng.choice(tc_ultimos_24, size=R)                                  # pesos por dólar
camp = (pesos_google[:, None] * s["cvr_g"] / (s["cpc_g"] * s["tc"])
        + pesos_facebook[:, None] * s["cvr_f"] / (s["cpc_f"] * s["tc"]))   # celdas × futuros
con = base + camp[i]                        # base = futuros del Pronóstico (escenarios.simular)
riesgo_con, riesgo_sin = (con > cap).mean(), (base > cap).mean()
```

Notebook 5, sección 4 · torre.campana.presupuesto.montecarlo_campana

8. Investigación de Operaciones: repartir el presupuesto

Materia: Investigación de Operaciones · **Profesor:** Carlos Orozco Rocha (tutor del equipo) · **Criterio 6 de la rúbrica (12 %):** formula variables de decisión, función objetivo y restricciones; la solución es justificable.

Conexión con la materia. En clase formulamos y resolvimos problemas de **programación lineal** con el **método gráfico**, a mano y en Excel: definir variables de decisión, escribir la función objetivo y las restricciones, dibujar la región factible y evaluar sus vértices. El incidente crítico de esta UCA, *"Optimización de los flujos turísticos para un desarrollo sustentable"*, pregunta qué significa que una distribución sea óptima, qué restricciones son indispensables, qué tan sensible es la solución y, en concreto, cómo asignar el presupuesto de la campaña "sin rebasar la capacidad de carga de ningún destino y sin generar desigualdades entre comunidades". Por eso este capítulo empieza con la versión de dos variables del modelo, resuelta con el método gráfico (8.3), y después la extiende al modelo completo (8.4).

8.1 La pregunta

¿Cuánto dinero de la campaña va a cada lugar del sur, en qué mes y por qué canal, para traer el mayor número de visitantes **sin anunciar donde ya está lleno** y **sin rebasar la capacidad de nadie**?

8.2 Los parámetros: cuánto rinde un peso en cada canal

La idea en palabras

Un anuncio funciona como un embudo: muchas personas lo ven, algunas le dan clic y, de esas, algunas planean el viaje (conversión). Se paga por cada clic. Lo que importa para decidir es cuántos visitantes trae cada peso.

$$\text{clics} = \frac{\text{pesos}}{\text{costo por clic}}, \qquad \text{visitantes} = \text{clics} \times \text{tasa de conversión}$$

Juntando las dos, el rendimiento de un peso en el canal c es:

$$r_c = \frac{\text{tasa de conversión}_c}{\text{costo por clic}_c \text{ (dólares)} \times \text{pesos por dólar}}$$

Datos (categoría viajes, WordStream by LocaliQ, 2025; promedios de anunciantes de Estados Unidos, usados como supuesto etiquetado): Google, clic de 2.12 dólares y conversión de 5.75 %; Facebook, clic de 0.51 dólares. El dólar es el promedio de agosto de 2026 en FRED: 17.0609 pesos.

Paso a paso con nuestros datos: visitantes por peso

Google. Un clic cuesta $2.12 \times 17.0609 = 36.17$ pesos.

$$r_G = \frac{0.0575}{36.169} = 0.001590 \text{ visitantes por peso} = 1.59 \text{ por cada mil pesos}$$

Facebook. Un clic cuesta $0.51 \times 17.0609 = 8.70$ pesos.

$$r_F = \frac{0.0575}{8.701} = 0.006608 \text{ visitantes por peso} = 6.61 \text{ por cada mil pesos}$$

Qué significa: Facebook rinde cuatro veces más por peso porque su clic es cuatro veces más barato (con la misma conversión supuesta).

Hueco declarado: la conversión de Facebook para turismo no está publicada. El caso base usa la de Google viajes (5.75 %) y la sensibilidad prueba 3 % y 6.38 %. Además suponemos que cada conversión es un visitante; es una cota alta: si llegan menos, la capacidad queda aún más protegida.

Cómo lo programamos: cuánto rinde un peso

Los costos por clic y la conversión se leen de la tabla de WordStream guardada en Bronze, y el tipo de cambio del último mes completo de FRED:

```
t = bm[bm.categoria == "Travel"].set_index(["clave_pagina", "metrica"]).valor
cpc_google = t["wordstream_google_2025", "Average CPC"]          # 2.12 dólares
cvr_google = t["wordstream_google_2025", "Average CVR"] / 100    # 5.75 %
cpc_facebook = t["wordstream_facebook_2025", "Average CPC"]      # 0.51 dólares
TC = fx[~fx.mes_incompleto_flag].sort_values("periodo").pesos_por_dolar.iloc[-1]  # 17.0609

r = {"Google": cvr_google / (cpc_google * TC),                   # 0.001590 por peso
     "Facebook": cvr_google / (cpc_facebook * TC)}               # 0.006608 (supuesto: misma
                                                                 # conversión que Google)
```

Notebook 6, sección 1 · torre.campana.presupuesto.canales

8.3 El modelo de dos variables, resuelto con el método gráfico

Antes del modelo completo, conviene ver el corazón del problema con solo dos variables, como en clase.

Variables de decisión (en miles de pesos):

- x_1 = dinero en Facebook / Instagram.
- x_2 = dinero en Google.

Función objetivo (maximizar visitantes):

$$\max Z = 6.608\, x_1 + 1.590\, x_2$$

Restricciones:

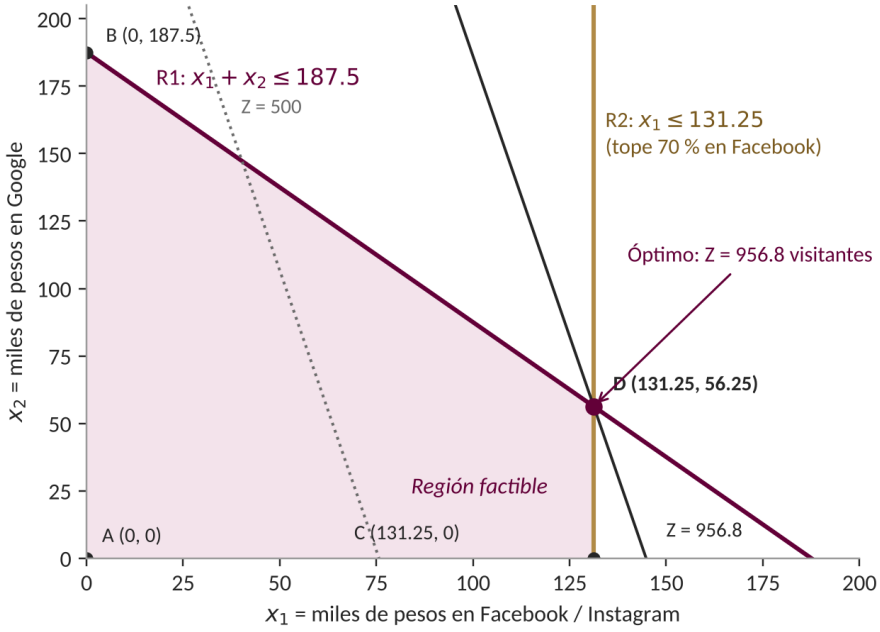
R1, presupuesto: $x_1 + x_2 \leq 187.5$

R2, tope de 70 % en Facebook: $x_1 \leq 131.25$

No negatividad: $x_1 \geq 0, x_2 \geq 0$

Elemento	De dónde sale
6.608 y 1.590	Visitantes por cada mil pesos en Facebook y en Google (8.2)
187.5	El presupuesto de 250 mil pesos al año, prorrateado a los 9 meses del plan: $250 \times 9/12 = 187.5$
131.25	El 70 % del presupuesto: $0.70 \times 187.5 = 131.25$. Es un tope para no depender de una sola plataforma

Paso 1. Graficar las restricciones. R1 es la recta $x_1 + x_2 = 187.5$, que corta los ejes en (187.5, 0) y (0, 187.5); la región que la cumple está debajo de la recta. R2 es la recta vertical $x_1 = 131.25$; la región está a su izquierda. Con la no negatividad, la **región factible** es el polígono sombreado de la figura.



Método gráfico: la región factible es el área sombreada; la recta negra continua es la función objetivo en su valor máximo, que toca la región solo en el vértice D.

Paso 2. Encontrar los vértices. El óptimo de un problema lineal siempre está en un vértice de la región factible. El vértice D es el cruce de R1 y R2: si $x_1 = 131.25$, entonces $x_2 = 187.5 - 131.25 = 56.25$.

Paso 3. Evaluar la función objetivo en cada vértice:

Vértice	x_1 (Facebook)	x_2 (Google)	$Z = 6.608\,x_1 + 1.590\,x_2$
A	0	0	0
B	0	187.5	1.590 × 187.5 = 298.1
C	131.25	0	6.608 × 131.25 = 867.3
D	**131.25**	**56.25**	867.3 + 89.4 = **956.8**

Paso 4. Conclusión. El óptimo es el vértice D: **131,250 pesos en Facebook y 56,250 en Google, que traen 956.8 visitantes** (unos 196 pesos por visitante). En la gráfica se ve también con las rectas de nivel:

la recta $Z = 500$ (punteada) cruza la región; al moverla hacia arriba y a la derecha sin cambiar su inclinación, el último punto que toca es D.

La lógica, sin gráfica: Facebook rinde más por peso, así que se lleva todo lo que el tope le permite (70 %); lo que sobra va a Google, porque gastarlo rinde más que dejarlo sin usar.

> **Este mismo resultado lo da el programa.** El modelo completo de 8.4 tiene 216 variables y no se puede dibujar, pero su respuesta total es la misma: 956.8 visitantes. Lo que agrega el modelo completo es **cómo repartir** ese dinero entre 3 lugares y 9 meses sin rebasar ninguna capacidad. El modelo chico se puede resolver también en Excel (con la herramienta Solver), como en las prácticas de la materia.

Cómo lo programamos: los vértices

El método gráfico en código es evaluar la función objetivo en los cuatro vértices de la región factible. El mismo modelo está en el Excel 6_Metodo_grafico.xlsx, listo para resolverse con Solver:

```
B, TOPE = 250 * 9 / 12, 0.70 * 250 * 9 / 12                # 187.5 y 131.25 (miles)
rF, rG = r["Facebook"] * 1000, r["Google"] * 1000           # 6.608 y 1.590
vertices = pd.DataFrame({"x1": [0, 0, TOPE, TOPE], "x2": [0, B, 0, B - TOPE]},
                        index=["A", "B", "C", "D"])
vertices["Z"] = rF * vertices.x1 + rG * vertices.x2         # D: 956.8 visitantes
```

Notebook 6, sección 2

8.4 El modelo completo

Conjuntos

- **Meses** m**:** de octubre de 2026 a junio de 2027, los 9 que tienen calendario y escenarios para los tres lugares. Después no hay pronóstico, y no se extrapola.
- **Lugares** l**:** Chetumal, Bahía, Ruta.
- **Canales** c**:** Google, Facebook.
- **Escenarios** s**:** malo, probable y bueno (percentiles 10, 50 y 90 del Monte Carlo), con probabilidades p_s = 0.3, 0.4 y 0.3 (regla de Swanson para resumir una distribución con tres percentiles).

Variables de decisión

Variable	Tipo	Qué decide	Cuándo se decide
$x_{m,l,c} \geq 0$	Continua	Pesos en el mes m, lugar l, canal c	Hoy (“etapa 1”)
$y_{s,m,l} \in \{0,1\}$	Binaria	Si el anuncio sigue encendido (1) o se pausa (0)	Cuando se ve el escenario (“etapa 2”)
$w_{s,m,l} \geq 0$	Continua	Visitantes que sí llegan por la campaña	Cuando se ve el escenario

Visitantes que trae la campaña en cada mes y lugar: $v_{m,l} = r_G\, x_{m,l,G} + r_F\, x_{m,l,F}$.

Función objetivo

$$\max \sum_{s} p_s \sum_{m,l} w_{s,m,l}$$

En palabras: **maximizar los visitantes esperados**, es decir, el promedio (ponderado por la probabilidad de cada escenario) de los visitantes que sí llegan.

Restricciones

#	Restricción	En palabras
R1	$\sum_{m,l,c} x_{m,l,c} \leq 187{,}500$	No gastar más del presupuesto
R2	$x_{m,l,c} = 0$ si (m, l) es temporada alta	Cero anuncio en temporada alta
R3	$D_{m,l}^{p50} + v_{m,l} \leq C_l$	En el escenario probable, los visitantes esperados más los de la campaña no pasan de la capacidad probada
R4	$\sum_{m,c} x_{m,l,c} \geq 0.15 \times 187{,}500$ para cada lugar	Piso de equidad: cada lugar recibe al menos 15 %
R5	$\sum_{m,l} x_{m,l,c} \leq 0.70 \times 187{,}500$ para cada canal	Tope por canal: ninguno pasa de 70 %
R6	$D_{s,m,l} + v_{m,l} \leq C_l + M_{s,m,l}\,(1 - y_{s,m,l})$	Si en un escenario se rebasaría la capacidad, el anuncio se pausa
R7	$w_{s,m,l} \leq v_{m,l}$ y $w_{s,m,l} \leq U\, y_{s,m,l}$	Solo cuentan los visitantes de anuncios encendidos

Cómo funciona la “M grande” (R6 y R7)

La restricción R6 tiene dos modos, como un interruptor:

- **Si el anuncio sigue ($y = 1$):** el término $M(1 - y)$ vale 0 y queda $D + v \leq C$. La capacidad debe respetarse en ese escenario.
- **Si se pausa ($y = 0$):** el término suma M, un número suficientemente grande para que la restricción ya no obligue. Pero entonces R7 dice $w \leq U \times 0 = 0$: esos visitantes se pierden.

El valor de M es el más chico que logra eso, $M_{s,m,l} = \max\left(D_{s,m,l} + U - C_l,\ 0\right)$, porque una M demasiado grande vuelve lento al solucionador. $U = \max_c r_c \times 187{,}500$ es lo más que podría traer una celda.

Por qué se llama “estocástico de dos etapas”. La decisión de hoy (x) se toma sin saber qué escenario va a ocurrir; la reacción (y) depende del escenario. Al maximizar el promedio sobre escenarios, el modelo aprende, desde hoy, a no gastar en meses en los que un año bueno llenaría el lugar: ese dinero se perdería con probabilidad $p_{\text{bueno}} = 0.3$. Aquí se juntan las dos materias: la incertidumbre de Modelos Estocásticos entra a la optimización de Investigación de Operaciones.

8.5 Cómo lo resuelve la computadora

El modelo se escribe con PuLP y lo resuelve el solucionador CBC. Tiene 54 variables x, 81 binarias y y 81 variables w, y tarda 0.3 segundos.

- **Simplex.** Si todas las variables fueran continuas, las restricciones formarían un poliedro (la versión en muchas dimensiones del polígono del método gráfico) y el óptimo estaría en un vértice. El método simplex salta de vértice en vértice, mejorando cada vez, hasta que ningún vecino mejora. Es exactamente la idea del Paso 3 del método gráfico, pero sin tener que revisar todos los vértices.
- **Ramificación y acotamiento.** Como y solo puede valer 0 o 1, CBC resuelve primero la versión con y continua (una cota optimista). Si alguna y sale fraccionaria, crea dos subproblemas ($y = 0$ y $y = 1$) y descarta las ramas que ya no pueden superar la mejor solución entera encontrada.

Desempate en dos pasos

Como todos los lugares convierten igual (no hay dato de respuesta por lugar), hay muchísimas soluciones con los mismos 956.8 visitantes. El solucionador entregaba una cualquiera: todo el dinero de cada lugar en un solo mes. Decidimos desempatar en dos pasos:

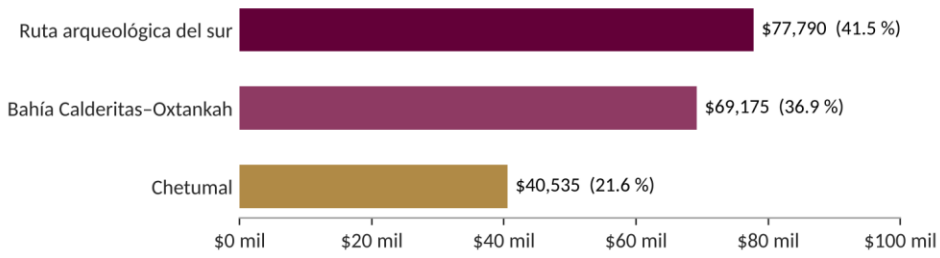
Paso 1: encontrar el máximo de visitantes, $z^* = 956.8$.

Paso 2: exigir al menos z^* y, entre esas soluciones, quedarse con la que reparte cada canal **en proporción al espacio libre** de cada mes y lugar:

$$\text{libre}_{m,l} = 1 - \frac{D_{m,l}^{p50}}{C_l}, \qquad \pi_{m,l} = \frac{\text{libre}_{m,l}}{\text{suma de los espacios libres de las celdas permitidas}}$$

y se minimiza la distancia entre lo que recibe cada celda y su meta $\pi_{m,l} \times$ (dinero del canal). El valor absoluto se vuelve lineal con un truco clásico: se agrega $d \geq 0$ con $d \geq x -$ meta y $d \geq$ meta $- x$, y se minimiza d.

> **Paso a paso con nuestros datos: la Ruta en octubre de 2026**
>
> **Paso 1, espacio libre.** Se esperan 3,688 visitantes y la capacidad probada es 10,465: libre $= 1 - 3{,}688/10{,}465 = 1 - 0.3524 = 0.6476$.
>
> **Paso 2, su proporción.** Hay 20 combinaciones de mes y lugar permitidas (las otras 7 son temporada alta) y sus espacios libres suman 8.0004: $\pi = 0.6476/8.0004 = 0.0809$.
>
> **Paso 3, el dinero.** Facebook: $0.0809 \times 131{,}250 = 10{,}624$ pesos. Google: $0.0809 \times 56{,}250 = 4{,}553$ pesos. El código da lo mismo.



Reparto final del presupuesto de nueve meses por lugar.

Reparto final: Ruta 41.5 % ($77,790), Bahía 36.9 % ($69,175) y Chetumal 21.6 % ($40,535). Diciembre queda sin anuncio en los tres lugares porque es temporada alta.

Cómo lo programamos: el modelo en PuLP

Cada variable de decisión es un objeto de PuLP; las restricciones se agregan con += y llevan nombre para poder pedir después su precio sombra. Este es el corazón del modelo, con las siete restricciones del capítulo 8.4:

```
m = pulp.LpProblem("presupuesto", pulp.LpMaximize)
celdas = list(enumerate(idx))                                    # (mes, lugar) numerados
x = {(p, l, c): pulp.LpVariable(f"x_{i}_{c}", lowBound=0) for i, (p, l) in celdas for c in r}
y = {(e, p, l): pulp.LpVariable(f"y_{e}_{i}", cat="Binary")
     for e in ESC for i, (p, l) in celdas}
w = {(e, p, l): pulp.LpVariable(f"w_{e}_{i}", lowBound=0) for e in ESC for i, (p, l) in celdas}
v = {(p, l): pulp.lpSum(r[c] * x[p, l, c] for c in r) for (p, l) in idx}    # visitantes
m += pulp.lpSum(x.values()) <= B, "presupuesto"                              # R1
for (p, l), f in filas.items():
    if f.nivel == "alta":
        for c in r: x[p, l, c].upBound = 0                                   # R2
    m += v[p, l] <= f.capacidad_probada - f.probable_p50_est                 # R3
    for e, (col, _) in ESC.items():
        M = max(getattr(f, col) + U - f.capacidad_probada, 0)                # la M más chica
        m += getattr(f, col) + v[p, l] <= f.capacidad_probada + M * (1 - y[e, p, l])  # R6
        m += w[e, p, l] <= v[p, l]                                           # R7
        m += w[e, p, l] <= U * y[e, p, l]
for l in SUR:
    m += pulp.lpSum(x[p, l2, c] for (p, l2, c) in x if l2 == l) >= 0.15 * B  # R4
for c in r:
    m += pulp.lpSum(x[p, l, c2] for (p, l, c2) in x if c2 == c) <= 0.70 * B  # R5
m.setObjective(pulp.lpSum(ESC[e][1] * w[e, p, l] for (e, p, l) in w))      # esperado
m.solve(pulp.PULP_CBC_CMD(msg=False))                                        # 956.8
```

Notebook 6, sección 3 · torre.campana.presupuesto.resolver

El desempate agrega una restricción que exige al menos el máximo encontrado y cambia el objetivo: minimizar la distancia entre lo que recibe cada celda y su meta proporcional al espacio libre. El valor absoluto se vuelve lineal con dos desigualdades:

```
m += esperado >= z1 * (1 - 1e-7), "lexicografico"          # no perder visitantes
for (p, l, c) in x:
    meta = libre[p, l] / total_libre * dinero_del_canal[c]
    m += d[p, l, c] >= x[p, l, c] - meta                    # |x − meta| ≤ d
    m += d[p, l, c] >= meta - x[p, l, c]
m.setObjective(-pulp.lpSum(d.values()))                   # maximizar −Σd = minimizar Σd
m.solve(pulp.PULP_CBC_CMD(msg=False))
```

Notebook 6, sección 3 · torre.campana.presupuesto.resolver

8.6 ¿Cuánto cuesta cada regla? Precios sombra

El costo de cada regla se mide resolviendo el modelo sin ella y comparando:

Regla	Visitantes que cuesta	%
Cero anuncio en temporada alta	0	0 %
Tope de capacidad probada	0	0 %
Piso de equidad de 15 %	0	0 %
Tope de 70 % por canal	282	29.5 %

Precio sombra. Para cada restricción, cuánto mejoraría el objetivo si se aflojara una unidad: $\lambda_i = \partial z^* / \partial b_i$. En la gráfica de 8.3 se ve directo: si el presupuesto sube un peso, la recta R1 se mueve un poquito hacia afuera y el vértice D se desliza hacia arriba (más Google), porque Facebook ya está en su tope.

- **Presupuesto:** 0.00159 visitantes por peso (1.59 por cada mil pesos), justo el rendimiento de Google, porque un peso extra iría a Google.
- **Tope de Facebook:** $0.006608 - 0.001590 = 0.005019$ por peso. Cada peso que pasa de Google a Facebook suma 0.005 visitantes. Quitar el tope suma $0.30 \times 187{,}500 \times 0.005019 = 282.3$: exactamente el costo de la tabla.
- **Equidad y capacidad: precio sombra 0.** Por la **holgura complementaria**, una restricción que no "aprieta" no tiene precio: aflojarla no trae ningún visitante. Con este presupuesto, la regla ambiental (no rebasar la capacidad) no le cuesta nada a la campaña.

Frontera de Pareto: visitantes contra espacio libre

Cuando hay dos objetivos que chocan (más visitantes contra menos presión), no hay un solo óptimo, sino una frontera de soluciones en la que no se puede mejorar uno sin empeorar el otro. Se traza con el método de la ε-restricción: se pide que ningún mes con anuncio pase de ε veces su capacidad en el escenario probable ($D^{p50} + v \le \varepsilon\, C$) y se maximizan los visitantes para varios valores de ε.

Tope ε	100 %	90 %	80 %	70 %	60 %	50 %	45 %	40 %	35 %	30 %
Visitantes	957	957	957	957	957	957	957	957	537	0

Qué significa: hasta 40 % no se pierde ningún visitante. La campaña puede prometer que **ningún mes con anuncio pasará del 40 % de su capacidad probada**, sin costo.

Cómo lo programamos: precios sombra y frontera de Pareto

Para leer los precios sombra se fijan las variables binarias en su valor óptimo y se vuelve a resolver como problema lineal; cada restricción guarda su precio en `.pi` y su holgura en `.slack`. La frontera de Pareto repite el modelo con un tope de ocupación distinto cada vez:

```
for v in y.values():
    v.lowBound = v.upBound = round(v.value())     # binarias fijas en el óptimo
    v.cat = pulp.LpContinuous
m.solve(pulp.PULP_CBC_CMD(msg=False))
```

```
sombra = {n: (c.pi, c.slack) for n, c in m.constraints.items()
          if n.startswith(("presupuesto", "equidad", "canal"))}   # presupuesto: 0.00159

for eps in (1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.45, 0.4, 0.35, 0.3):
    resolver(replace(Supuestos(), ocupacion_max=eps))            # 957 hasta 40 %; 537 con 35 %
```

Notebook 6, sección 4 · torre.campana.presupuesto.precios_sombra y pareto

8.7 Sensibilidad: ¿y si los supuestos están mal?

Si cambia…	Visitantes
Nada (caso base)	957
Conversión de Facebook de 3 %	542
Conversión de Facebook de 6.38 %	1,052
Clic de Facebook a 0.42 dólares	1,143
Golpe de tormenta de 25 % o 50 %	957
Presupuesto de 500 mil pesos al año	1,914
Presupuesto de 1 millón de pesos al año	3,827

Los visitantes son proporcionales al presupuesto porque el modelo es lineal y la capacidad no se alcanza. El reparto entre lugares y canales casi no cambia en ningún caso: **la recomendación es estable**.

¿Por qué un piso de 15 % y un tope de 70 %? Son reglas de política, no datos, así que las probamos con varios valores. Cualquier piso de equidad hasta un tercio cuesta cero visitantes; elegimos 15 %, la mitad de un reparto igual, para que ninguna comunidad quede fuera si cambian los datos. El tope por canal sí cuesta: con 50 % llegan 769 visitantes, con 70 % llegan 957 y sin tope, 1,239. Nos quedamos con 70 % porque deja a Facebook como canal principal sin que toda la campaña dependa de su conversión, que es el supuesto menos seguro del modelo.

Cómo lo programamos: el barrido del piso y del tope

Los supuestos están en una clase con valores por omisión; `replace` cambia uno a la vez. Con eso, la sensibilidad y el barrido del piso y del tope son un ciclo:

```
@dataclass(frozen=True)
class Supuestos:
    presupuesto_anual: float = 250_000.0
    cvr_facebook: float | None = None        # None = la de Google (5.75 %)
    piso_equidad: float = 0.15
    tope_canal: float = 0.70

for piso in (0.0, 0.15, 0.25, 1 / 3):
    resolver(replace(Supuestos(), piso_equidad=piso))   # 956.8 en los cuatro: no cuesta
for tope in (0.5, 0.6, 0.7, 0.8, 1.0):
    resolver(replace(Supuestos(), tope_canal=tope))     # 768.6, 862.7, 956.8, 1050.9, 1239.1
```

Notebook 6, sección 5 · torre.campana.presupuesto

8.8 La etapa 2 en la práctica: el motor de la Torre

Cada semana, para cada lugar, el motor decide: temporada alta (no se gasta), fuera del plan, pausado (tormenta cerca, clima raro o llegó más gente de la esperada) o encendido. El dinero pausado no se pierde: pasa a la siguiente semana permitida.

$$b = \frac{\text{pesos del mes}}{\text{lunes del mes}}, \qquad \text{gasto}_w = (b + A_{w-1}) \text{ si está encendido; } 0 \text{ si no}$$

donde A_w es el dinero acumulado: sube b cuando la semana se pausa y vuelve a 0 cuando se enciende (se gasta todo).

> **Paso a paso: el arrastre del dinero**
>
> Octubre tiene 4 lunes; con 4,000 pesos en el mes, $b = 1{,}000$. **Semana 1 pausada:** no se gasta y $A = 1{,}000$. **Semana 2 encendida:** se gastan $1{,}000 + 1{,}000 = 2{,}000$ y A vuelve a 0.

En las 239 semanas reales hubo 48 pausas (lugar-semana), todo el dinero pausado se gastó después y la opción “¿Ibas al norte?” se encendió 6 semanas.

Cómo lo programamos: una semana del motor

El motor es una función pura: recibe las señales de una semana, el plan del mes y la memoria del dinero pausado, y devuelve la decisión de cada lugar. Al no leer archivos ni usar Spark, se puede probar sola:

```
def decidir(fila, plan, memoria):
    for lugar in SUR:
        if (mes, lugar) in plan.alta:
            accion = "temporada alta"                       # no se gasta
        else:
            semanal = plan.pesos_mes[mes, lugar] / lunes_del_mes(semana)
            pausa = (fila["tormenta"] is True                 # tormenta a ≤ 200 km
                     or fila[f"{punto}_clima_raro"] is True   # Isolation Forest
                     or fila[f"{clave}_llegadas"] == "arriba")  # más gente de la esperada
            if pausa:
                memoria.arrastre[lugar] += semanal          # el dinero se guarda
                accion = "pausado"
            else:
                gasto = semanal + memoria.arrastre[lugar]   # se gasta lo de la semana
                memoria.arrastre[lugar] = 0.0               # y lo que estaba guardado
                accion = "encendido"
```

Notebook 6, sección 6 · torre.envivo.motor.decidir

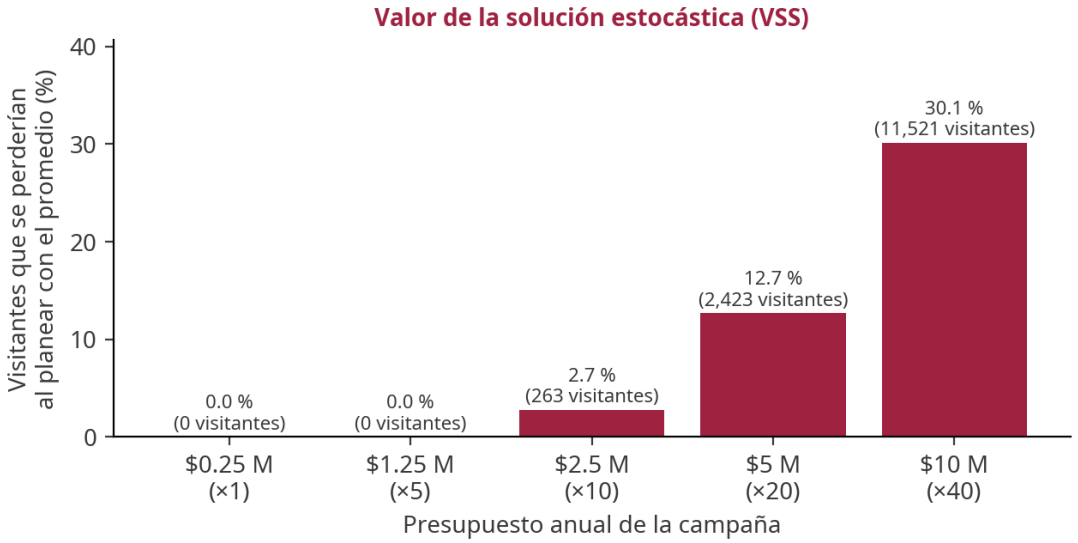
8.9 ¿Valió la pena el modelo estocástico? VSS y EVPI

Un modelo de dos etapas es más complicado que uno determinista. ¿Cambia la decisión? La programación estocástica (Birge y Louveaux, 2011) tiene dos medidas para responderlo:

Medida	Cómo se calcula	Qué pregunta responde
RP	El óptimo con los tres escenarios (nuestro modelo)	—
EEV	Resolver como si el escenario probable fuera seguro y evaluar	¿Qué pasa si planeo con el promedio?

Medida	Cómo se calcula	Qué pregunta responde
	ese plan con los tres escenarios y la etapa 2	
VSS = RP − EEV	Valor de la solución estocástica	¿Cuánto se gana por planear con la incertidumbre?
WS	El óptimo de cada escenario si se supiera que va a pasar, ponderado	El mejor caso imposible: saber el futuro
EVPI = WS − RP	Valor esperado de la información perfecta	¿Cuánto valdría un pronóstico perfecto?

Presupuesto anual	RP	EEV	VSS	VSS (%)	EVPI
$250,000 (×1)	957	957	0	0.0 %	0
$1,250,000 (×5)	4,784	4,784	0	0.0 %	0
$2,500,000 (×10)	9,568	9,305	263	2.7 %	0
$5,000,000 (×20)	19,136	16,712	2,423	12.7 %	0
$10,000,000 (×40)	38,271	26,750	11,521	30.1 %	0



Visitantes que se perderían al planear con el escenario probable, según el presupuesto.

Cómo se lee. Con el presupuesto actual ($250,000) y hasta 5 veces más, **VSS = 0**: la campaña es chica frente al espacio libre y ningún escenario se acerca a la capacidad donde se gasta. Es la misma razón por la que el precio sombra de la capacidad es 0. Con 10 veces el presupuesto, planear con el promedio perdería 263 visitantes (2.7 %); con 40 veces, 30 %, porque pondría dinero en meses que en el escenario bueno se llenan y habría que pausarlos. **EVPI = 0** siempre: el plan estocástico ya consigue lo

mismo que si se conociera el futuro, porque nunca gasta donde el escenario bueno rebasaría la capacidad.

Lo que respondemos. Con el presupuesto de hoy, el modelo estocástico no cambia el plan: es un seguro que no cuesta nada y que empieza a pagar cuando la campaña crece. Si el equipo consiguiera 10 veces más dinero, el mismo modelo ya evitaría perder visitantes sin reescribirlo.

```
def valor_estocastico(s=Supuestos()):
    rp = resolver(s)                                                  # tres escenarios
    det = resolver(s, {"probable": ("probable_p50_est", 1.0)})          # como si fuera seguro
    eev = evaluar_plan(det["plan"], rp["tabla"])     # ese plan, con la etapa 2 de los tres
    ws = sum(p * resolver(s, {e: (col, 1.0)})["visitantes_esperados"]
             for e, (col, p) in ESCENARIOS.items())                   # saber el futuro
    return rp["visitantes_esperados"] - eev, ws - rp["visitantes_esperados"]   # VSS, EVPI
```

Notebook 6, sección 5 · torre.campana.presupuesto.valor_estocastico

9. Mercadotecnia digital: la campaña

Materia: Mercadotecnia Digital · **Profesor:** Víctor Hugo Reyes Anselmo · **Criterio 7 de la rúbrica (8 %):** estrategia integral sustentada en datos; plan mercadológico, branding, buyer persona y medios digitales.

Conexión con la materia. El incidente crítico de esta UCA, *"Estrategias digitales para la redistribución del turismo"*, exige que "todas las decisiones sobre el buyer persona, los mensajes, los canales y el presupuesto" estén justificadas con los resultados de las otras cinco materias, y pregunta qué hacer "si la campaña genera una afluencia turística mayor a la capacidad del destino". Aplicamos el esquema clásico **STP** (segmentar, elegir y posicionar; Kotler y Keller, 2016), el **buyer persona**, la **propuesta de valor**, la selección de **medios**, el **embudo de conversión** con sus **KPI** y una regla anti-colapso programada. Cada rasgo de la campaña dice si es dato, cálculo o supuesto.

9.1 Segmentar: lo que dicen los datos

- **Quién visita el sur** (INAH 2025): Oxtankah, 94.8 % nacionales; la Ruta, 63.3 % nacionales y 36.7 % extranjeros.
- **Quién llega al estado en avión** (Unidad de Política Migratoria, 2025): 9,408,423 extranjeros a Cancún; Estados Unidos 56.3 % y Canadá 17.1 %; 53.4 % mujeres; más llegadas en marzo y menos en septiembre.
- **El hallazgo que define todo:** al aeropuerto de Chetumal llegaron **250 extranjeros** en 2025. El extranjero no llega al sur por avión: entra por Cancún y baja por carretera, en Tren Maya, desde Belice o en crucero.
- **Cómo se informan** (ENDUTIH 2025, INEGI): 92.1 % de la población de Quintana Roo usa internet (tercer lugar nacional); de quienes tienen teléfono inteligente, 90.6 % usa mensajería y 80.4 % redes sociales.
- **Qué valoran y qué los aleja** (capítulo 5): el ruido (2.76×), la suciedad (1.89×), el precio (1.66×) y las multitudes (1.52×) hunden una reseña; la calma (0.41×) y la cultura (0.48×) la protegen.

Cómo lo programamos: las cifras de la segmentación

Todas las cifras de la campaña salen de dos tablas de Silver, INAH y nacionalidades, y se guardan en `campana.json`, de donde las toma la página:

```
q = inah[inah.es_qroo & (inah.anio.astype(int) == 2025)]
v = q.groupby("region_campana").visitantes.sum()
veces_ruta = v["Tulum"] / v["Ruta arqueológica del sur"]              # 15.5
nac = (q[q.papel_campana == "promovida"]
       .groupby(["region_campana", "tipo_visitante"]).visitantes.sum().unstack())
pct_nac_bahia = nac.loc["Bahía Calderitas-Oxtankah", "nacional"] / nac.loc[
    "Bahía Calderitas-Oxtankah"].sum() * 100                           # 94.8 %
c = nacionalidad[(nacionalidad.lugar_campana == "Cancún") & (nacionalidad.anio == 2025)]
cancun = c.llegadas_extranjeros.sum()                                  # 9,408,423
pct_eeuu = c[c.pais == "Estados Unidos"].llegadas_extranjeros.sum() / cancun * 100
```

Notebook 7, sección 1 · torre.campana.marca.cifras

9.2 Elegir: dos viajeras ideales (buyer persona)

Elegimos dos personas: **“La que vuelve al sur”** (nacional) y **“La que baja del norte”** (Estados Unidos o Canadá, ya en Cancún o la Riviera). Descartamos solo la nacional (deja fuera al extranjero que ya está a 334 km) y solo la extranjera (la Bahía es 95 % nacional).

Persona	Rasgo	Valor y fuente	Tipo
La que vuelve al sur	Origen	Nacional: 94.8 % de Oxtankah y 63.3 % de la Ruta (INAH 2025)	Dato
	Cuándo viaja	De diciembre a abril; se le invita en los meses con espacio	Derivado
	Qué valora	Calma (0.41×) y cultura (0.48×) en las reseñas	Derivado
	Qué la aleja	Ruido, suciedad, precio y multitudes	Derivado
	Cómo se informa	Mensajería 90.6 %, redes 80.4 % (ENDUTIH 2025)	Dato
	Estado de origen, edad, ingreso	No hay dato oficial por destino	Hueco
La que baja del norte	Origen	EE. UU. 56.3 % y Canadá 17.1 % de 9,408,423 extranjeros en Cancún	Dato
	Sexo	53.4 % mujeres	Dato
	Por qué no llega sola	250 extranjeros en el aeropuerto de Chetumal	Dato
	Cuándo está	Más en marzo, menos en septiembre; Cancún en temporada alta de noviembre a abril	Dato
	Cómo se informa	Busca en Google y usa redes desde el destino	Supuesto
	Idioma	Inglés y español	Derivado

Cómo lo programamos: cada rasgo con su etiqueta

Cada persona es una lista de rasgos, y cada rasgo es una tupla de cuatro partes: el nombre, el valor, el tipo (dato, derivado, supuesto o hueco) y la fuente. Así la etiqueta no se puede olvidar: el texto de la página y de este documento se arma de esa misma lista:

```
{"clave": "vuelve", "nombre": "La que vuelve al sur",
 "atributos": [
   ("Origen", f"Nacional: {k['pct_nac_bahia']} % de Oxtankah, {k['pct_nac_ruta']} % de la Ruta",
    "dato", "INAH 2025"),
   ("Estado de origen", "No se publica de qué estado vienen", "hueco", "—"),
   ("Qué valora", f"Calma ({q['calma']:.2f}×) y cultura ({q['cultura']:.2f}×)",
    "derivado", "Rest-Mex 2025"),
```

```
  ("Cómo se informa", "Mensajería (90.6 %) y redes sociales (80.4 %)",
   "dato", "ENDUTIH 2025, pág. 17")]}
```

Notebook 7, sección 2 · torre.campana.marca.personas

9.3 Posicionar: la marca

- **Nombre:** “El sur tiene espacio” (*The south has room*). Dice el dato (capacidad ociosa) y la promesa (sin multitudes) a la vez.
- **Lema:** “Pirámides, bahía y sabor del sur, con espacio para disfrutarlos”.
- **Personalidad:** cercana y tranquila; orgullosa de lo maya y lo mexicano; con espíritu de aventura.
- **Propuesta de valor:** el Caribe mexicano que todavía tiene espacio: zonas mayas con 15.5 veces menos visitantes que Tulum (INAH 2025), una bahía tranquila y la comida del sur.
- **Declaración de posicionamiento:** para quien quiere el Caribe mexicano sin multitudes, el sur de Quintana Roo es la opción cultural y tranquila; a diferencia de Tulum o Cancún, aquí hay espacio, y la campaña lo cuida.
- **Identidad visual:** el sistema “Sur mexicano”: rosa, amarillo, turquesa y añil en bloques; tipografías Bricolage y Figtree; greca maya; fotos reales con coordenada comprobada.

9.4 Los mensajes: cada promesa con su dato

Reglas: cada anuncio lleva el dato que lo respalda; no se mencionan precios ni horas de viaje (no hay dato oficial); el lenguaje sale de las reseñas de 5 estrellas; y el largo se revisa contra los límites de cada plataforma (Google: título de hasta 30 caracteres y descripción de hasta 90; Meta: título de hasta 40 y texto de hasta 125).

Persona	Canal	Mensaje	Dato que lo respalda
Vuelve	Google (es)	“Pirámides mayas con espacio”	Tulum 1,031,443 visitantes; Ruta 66,628 (15.5 veces menos)
Vuelve	Facebook (es)	“La bahía que te esperaba”	Oxtankah: 11,017 visitantes en 2025, 94.8 % mexicanos
Vuelve	WhatsApp	“¿Y si este puente nos vamos al sur?”	90.6 % usa mensajería; sin costo
Baja	Google (en)	“Mayan pyramids, with room”	El mismo 15.5; Cancún en temporada alta
Baja	Facebook (en)	“Your Caribbean trip, one stop south”	Tren Maya con estación en Chetumal; solo se muestra si la Torre marca el norte lleno

Cómo lo programamos: cada anuncio cabe en su plataforma

Antes de guardar la campaña, el código revisa el largo de cada anuncio contra los límites de Google y de Meta; si uno se pasa, se detiene:

```
LIMITES = {"google_titulo": 30, "google_descripcion": 90, "meta_titulo": 40, "meta_texto": 125}
for m in mensajes:
    if m["canal"].startswith("Google"):
        assert all(len(t) <= LIMITES["google_titulo"] for t in m["titulos"])
        assert len(m["descripcion"]) <= LIMITES["google_descripcion"]
    elif m["canal"].startswith("Facebook"):
        assert len(m["titulo"]) <= LIMITES["meta_titulo"]
        assert len(m["texto"]) <= LIMITES["meta_texto"]
```

Notebook 7, sección 4 · torre.campana.marca.revisar_largos

9.5 Los medios

Medio	Peso	Justificación con datos
Facebook / Instagram	70 %	6.61 visitantes por cada mil pesos; 80.4 % usa redes. Tope de 70 % para no depender de una plataforma (capítulo 8)
Google búsqueda	30 %	Llega a quien ya busca viajar; conversión de 5.75 % en viajes
WhatsApp (orgánico)	Sin costo	90.6 % usa mensajería; el botón de compartir viaja entre familia y amigos
La página	Destino	Planeador por lugar y mes en 10 idiomas, sin llamadas a otros sitios

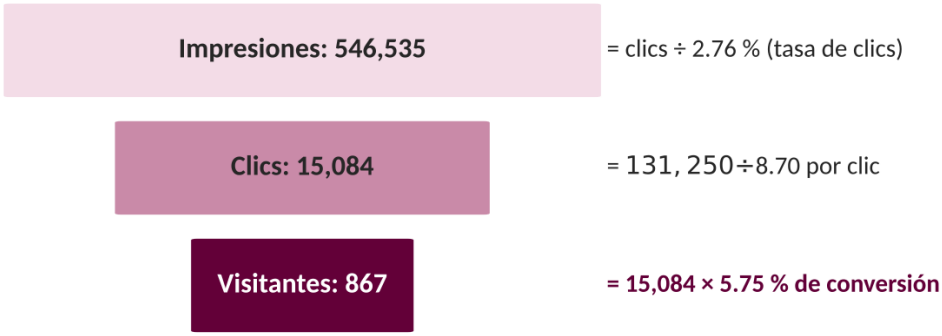
Calendario: los 9 meses del plan (octubre de 2026 a junio de 2027) con el dinero del capítulo 8. “La que baja del norte” se activa solo en los meses en que Cancún está en temporada alta y el sur tiene espacio; además, la Torre la enciende por semana cuando el norte se satura.

9.6 Cómo se mide: el embudo y los indicadores

> **La idea en palabras**
>
> El embudo se recorre al revés de como se paga: con el dinero se compran clics; para conseguir esos clics, cierta cantidad de personas tuvo que ver el anuncio; y de los clics, una parte se convierte en visitantes.

$$\text{clics} = \frac{\text{pesos}}{\text{costo por clic}}, \qquad \text{impresiones} = \frac{\text{clics}}{\text{CTR}}, \qquad \text{visitantes} = \text{clics} \times \text{tasa de conversión}$$

Indicador	Fórmula	Qué mide
CTR (tasa de clics)	clics ÷ impresiones	Qué tan atractivo es el anuncio
Tasa de conversión	planes de viaje ÷ clics	Qué tan convincente es la página
CPA (costo por visitante)	pesos gastados ÷ conversiones	Qué tan eficiente es el gasto



El embudo de Facebook con el reparto del capítulo 8.

Paso a paso con nuestros datos: el embudo esperado

Facebook. Clics: 131,250 ÷ 8.70 = 15,084. Con una tasa de clics de 2.76 %, eso supone 15,084 ÷ 0.0276 = 546,535 impresiones. Visitantes: 15,084 × 0.0575 = 867.

Google. Clics: 56,250 ÷ 36.17 = 1,555. Con una tasa de clics de 8.73 %, 1,555 ÷ 0.0873 = 17,814 impresiones. Visitantes: 1,555 × 0.0575 = 89.

Costo por visitante: 187,500 ÷ 956 = 196 pesos.

Son estimaciones con promedios de EE. UU.: las metas reales se fijan con el primer mes de campaña.

Indicador	Fórmula	Meta
Alcance	Personas únicas que vieron un anuncio	Se fija con el primer mes real
Interacción (CTR)	clics ÷ impresiones	Google ≥ 8.73 %; Facebook ≥ 2.76 %
Conversión	planes de viaje ÷ clics	≥ 5.75 %
Costo por visitante	pesos gastados ÷ conversiones	≤ 196 pesos
Afluencia	visitantes reales − escenario probable	+957 en 9 meses
Presupuesto	pesos gastados ÷ pesos planeados	100 % (lo pausado se gasta después)
Económico	ocupación hotelera de Chetumal	Hueco: no se publica desde 2025
Social	visitantes por cada mil habitantes (Radar)	Nunca “saturado”
Ambiental	semanas con anuncio en temporada alta o con mal clima	0
Capacidad	(esperados + campaña) ÷ capacidad probada	≤ 40 % en meses con anuncio

Cómo lo programamos: el embudo por canal

El embudo usa el dinero del plan de cada canal y los promedios de WordStream. La suma de visitantes de los dos canales es exactamente la del modelo de presupuesto (956.8), porque es el mismo cálculo visto por canal:

```
for canal in ("Facebook", "Google"):
    cpc = t[pagina[canal], "Average CPC"] * TC              # costo por clic en pesos
    ctr = t[pagina[canal], "Average CTR"] / 100             # tasa de clics
    clics = plan_por_canal[canal] / cpc                     # Facebook: 15,084
    impresiones = clics / ctr                               # Facebook: 546,535
    visitantes = clics * cvr                                # Facebook: 867
costo_por_visitante = 187_500 / visitantes_totales          # 196 pesos
```

Notebook 7, sección 5

9.7 La regla anti-colapso

El incidente pregunta qué hacer si la afluencia rebasa la capacidad. La campaña tiene la respuesta programada, no solo escrita:

1. Nunca anuncia un lugar en su temporada alta (restricción R2 del presupuesto).
2. No permite que los visitantes esperados más los de la campaña pasen de la capacidad probada (R3).
3. La Torre pausa el anuncio si hay tormenta, clima raro o llegó más gente de la esperada.
4. A quien iba al norte lleno le ofrece el lugar del sur con más espacio, nunca otro lugar lleno.

Cómo lo programamos: comprobar la regla con las 239 semanas

La regla anti-colapso no se afirma: se comprueba con las decisiones reales de la reproducción. En ninguna semana de temporada alta se gastó un peso, y cada vez que se encendió «¿Ibas al norte?» el lugar del sur recomendado estaba encendido esa semana:

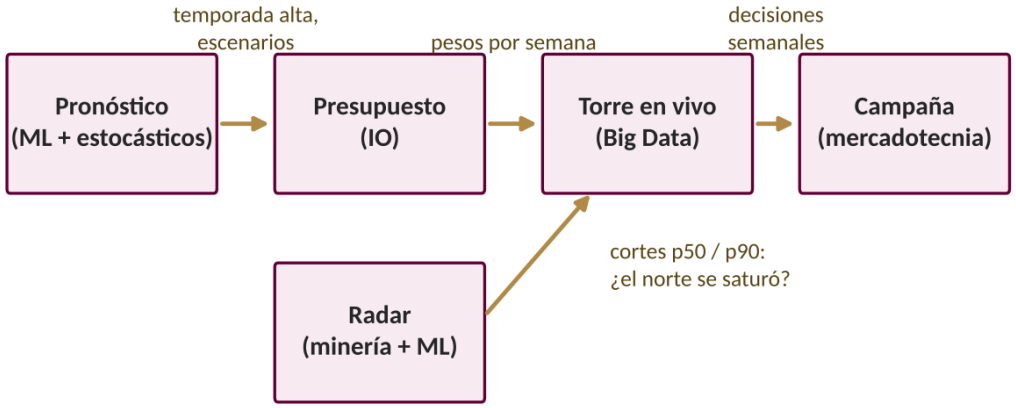
```
dec = pd.read_parquet("datos/gold/envivo_decisiones.parquet")
norte = pd.read_parquet("datos/gold/envivo_norte.parquet")
assert dec[dec.accion == "temporada alta"].pesos_gastados.sum() == 0      # 0 pesos
encendido = set(zip(dec[dec.accion == "encendido"].semana,
                    dec[dec.accion == "encendido"].lugar))
assert all((r.semana, r.destino) in encendido
           for r in norte[norte.ibas_al_norte].itertuples())             # 6 semanas
```

Notebook 7, sección 7

10. Integración y propuesta final

Criterios 8 y 9 de la rúbrica (10 % y 8 %): articula los seis componentes; compara escenarios y propone una alternativa viable y equilibrada entre beneficio económico, distribución turística e impacto ambiental.

10.1 La estafeta: la salida de una materia es la entrada de la siguiente



Cómo se pasan los resultados entre los módulos del sistema.

Paso	Sale de (materia)	Archivo	Lo usa
1	Forma del año y Monte Carlo (Aprendizaje de Máquina y Estocásticos)	`pronostico_calendario`, `pronostico_escenarios`	El presupuesto: qué meses son temporada alta y cuánta capacidad queda
2	Presupuesto (Investigación de Operaciones)	`presupuesto_plan`	La Torre: cuánto gastar cada semana
3	Radar (Minería y Aprendizaje de Máquina)	Cortes p50/p90 de la ocupación semanal	La Torre: cuándo el norte está saturado
4	Torre en vivo (Grandes Volúmenes)	`envivo_decisiones`	La campaña: indicador ambiental y “¿Ibas al norte?”
5	Campaña (Mercadotecnia Digital)	`campana.json`	La página y el coloquio

Cómo cuidamos cada cifra

Dos mecanismos impiden que una cifra del documento se separe del cálculo. El primero son las **pruebas automáticas** del proyecto (316, con pytest): unas revisan la forma de los datos y otras recalculan una cifra conocida y la comparan con la que está escrita. Si alguien cambia un dato o un modelo, la prueba falla:

```
def test_comparacion_contra_persistencia(r):
    m = r["metricas"].set_index("modelo")
    assert (m.loc["Persistencia (línea base)", "aciertos"],
```

```
            m.loc["Random Forest", "aciertos"]) == (126, 130)
    assert r["mejor"] == "Random Forest" and r["le_gana_a_persistencia"]

def test_f1_macro_a_mano():                        # el ejemplo resuelto del capítulo 6.1
    def f1(vp, fp, fn):
        p, rr = vp / (vp + fp), vp / (vp + fn)
        return 2 * p * rr / (p + rr)
    assert round((f1(75, 9, 12) + f1(50, 14, 12) + f1(5, 3, 2)) / 3, 3) == 0.779
```

tests/test_radar_prediccion.py

El segundo es la función `coincide`, que cierra cada paso de los siete notebooks. Compara la cifra que el notebook acaba de calcular a la vista con la que guardó el proyecto; si la diferencia pasa la tolerancia, el notebook se detiene y la entrega no se arma:

```
def coincide(nombre, nuestro, proyecto, tolerancia=1e-6):
    a = np.asarray(nuestro, dtype=float).ravel()
    b = np.asarray(proyecto, dtype=float).ravel()
    dif = float(np.nanmax(np.abs(a - b)))
    print(f"{'✓' if dif <= tolerancia else '✗'} {nombre}: coincide con el proyecto")
    assert dif <= tolerancia, f"{nombre} no coincide con el proyecto"

coincide("λ por mes", p["lambda"], proyecto["lambda"])   # ✓ λ por mes: coincide
```

Los siete notebooks (torre.documento.uca.comun)

10.2 Comparación de escenarios y propuesta

Escenario	Qué pasa	Decisión de la campaña
Año malo (percentil 10)	Llegan menos visitantes de lo esperado	La campaña trae su +957; no hay riesgo de capacidad
Año probable (percentil 50)	Lo pronosticado	Se gasta el plan completo; ningún mes con anuncio pasa del 40 % de su capacidad
Año bueno (percentil 90)	Llegan más visitantes	La Torre pausa el anuncio en los meses que se llenarían y el dinero pasa a semanas con espacio
Tormenta (agosto a octubre)	Hasta −50 % en el mes	Pausa semanal; el año cambia solo 1–2 %

Propuesta final: una campaña de 187,500 pesos en nueve meses (70 % Facebook/Instagram y 30 % Google, más WhatsApp orgánico), repartida Ruta 41.5 %, Bahía 36.9 % y Chetumal 21.6 %, sin anuncio en diciembre, que trae en el escenario esperado unos **957 visitantes adicionales a 196 pesos cada uno**, sin que ningún mes con anuncio pase del 40 % de su capacidad probada. Equilibra los tres objetivos del Problema Prototípico: más actividad económica en el sur, menos presión relativa en el norte y cero anuncios en temporada alta o con mal clima.

10.3 La página y el servidor

La página (https://catoxp.github.io/torre-del-caribe-unrc-2026/) es HTML, CSS y JavaScript sin framework y funciona sin internet. Todas sus cifras salen de un solo archivo generado desde Silver y Gold. La parte del viajero (planeador, qué hacer, lugares) está en 10 idiomas; la parte de los datos

(Radar, presupuesto, Torre, campaña) va en español. El servidor FastAPI tiene 8 servicios; entre ellos, `/api/optimizar` resuelve en vivo el modelo del capítulo 8 en 0.3 segundos con los supuestos que manda la página, y `/api/stream` transmite la Torre semana a semana. La auditoría de accesibilidad con axe-core (normas WCAG 2.1, niveles A y AA) pasó de 67 fallas a 0 en cuatro vistas. No hicimos prueba con personas, y lo declaramos.

El clima que ve el viajero. La página muestra la lluvia y la temperatura máxima normales 1991–2020 medidas en las estaciones del Servicio Meteorológico Nacional (Chetumal, Nicolás Bravo y Cancún; en la Riviera Maya no hay estación con normales y se usa ERA5). Los modelos siguen con el reanálisis ERA5, porque necesitan la lluvia de cada mes de cada año. Comparados en 1991–2020, ERA5 reproduce la forma de la lluvia (correlación de 0.96 a 0.99, de 3 % a 9 % menos al año), pero marca de 2 a 3 °C menos de temperatura máxima, y eso para un viajero sí importa.

La entrega. Además de este documento entregamos siete notebooks, uno por materia, ya ejecutados. Cada uno rehace los cálculos de su capítulo a la vista, explica cada paso, justifica cada hiperparámetro con las alternativas que descartamos y compara cada cifra con la del proyecto: si una no coincidiera, el notebook se detendría. Los cálculos de Poisson (7.3) y del método gráfico (8.3) también están en Excel, con fórmulas.

Cómo lo programamos: de los resultados a la página

La página no calcula modelos: lee un solo archivo, `datos/pagina.js`, que arma el módulo `datos_pagina` con las tablas de Gold. Así cada cifra que ve el viajero sale del mismo resultado que este documento. El precio de «Cotiza tu viaje» lo calcula `cotizar.js` en el navegador, nunca la inteligencia artificial: suma las entradas oficiales del INAH y el tarifario del equipo, y aplica el margen según qué tan lleno está el mes (8 % en un mes tranquilo, 18 % en uno normal y 30 % en temporada alta, siempre entre 5 % y 30 %):

```python
def generar():
    datos = {"generado": date.today().isoformat(), "regiones": fichas_regiones(),
             "llegar": llegar_pagina(mov, viajero), "tarifas": tarifas_pagina(),
             "historia": historia_pagina(), "fauna": fauna_pagina()}     # una clave por sección
    datos["radar"] = radar()                        # índice, estados y predicción (Gold)
    datos["pronostico"] = planeador_pagina()        # calendario de 12 meses y escenarios
    datos["presupuesto"] = presupuesto_pagina()     # reparto del capítulo 8
    datos["campana"] = campana_pagina()             # personas, mensajes e indicadores
    SALIDA.write_text("window.TORRE_DATOS = " + json.dumps(datos, ensure_ascii=False) + ";",
                      encoding="utf-8")
```

torre.api.datos_pagina

```javascript
function cotizar(pedido, tarifas) {
  for (const zona of pedido.zonas) {                       // 1. entradas del INAH (oficial)
    const t = tarifas.oficiales.find((x) => x.zona === zona);
    const unitario = pedido.nacional && t.nacional != null ? t.nacional : t.general;
    conceptos.push({ concepto: "entrada", detalle: zona, monto: unitario * personas });
  }
  // 2. hospedaje, comida, transporte y tren: solo si el equipo ya los cotizó; si no, "faltan"
  const m = tarifas.margen;                                // 3. margen según el mes
  const margen = Math.min(m.maximo, Math.max(m.minimo, m.niveles[pedido.nivel]));
  const costo = conceptos.reduce((a, c) => a + c.monto, 0);
  return { costo, margen, precio: Math.round(costo * (1 + margen)), faltan };
}
```

frontend/cotizar.js (se prueba con Node desde tests/test_viaje.py)

Cómo lo programamos: la inteligencia artificial solo elige del catálogo

En «Cotiza tu viaje», Gemini recibe el pedido del viajero y un **catálogo cerrado**: los lugares, las zonas arqueológicas y los negocios reales del DENUE, cada uno con un identificador. Su instrucción le prohíbe inventar lugares, horarios o precios. Al regresar, la página revisa la respuesta: si trae un solo identificador que no está en el catálogo, se descarta toda y el plan lo arma un respaldo por reglas con los mismos datos. La frase que escribe la IA se borra si trae cifras, porque las cifras salen de nuestros datos:

```
function validarIA(x, p, cat) {
  const L = cat.lugares.find((l) => l.clave === x.lugar);
  if (!L || x.dias.length !== p.dias) return null;              // forma equivocada: fuera
  for (const d of x.dias)
    for (const m of d.momentos)
      if (m.ids.some((id) => !cat.ids.has(id))) return null;  // un id inventado: fuera
  let mes = x.mes;                                            // el mes: el que pidió el viajero
  if (Number.isInteger(p.mes)) mes = p.mes;                     // o, si no, uno tranquilo
  else if (L.meses[mes].nivel !== "tranquila")
    mes = L.meses.findIndex((m) => m.nivel === "tranquila");
  const encaja = !/\d/.test(x.encaja) ? x.encaja : "";          // sin cifras inventadas
  return { lugar: x.lugar, mes, dias: x.dias, encaja, fuente: "ia" };
}
```

frontend/app.js (validarIA; si la IA falla o tarda más de 25 s, entra armarPorReglas)

11. Lo que no se pudo medir y lo que se supuso

11.1 Supuestos

Supuesto	Dónde	Cómo se protegió
Costos por clic y conversiones de EE. UU.	Presupuesto	Se etiquetan como supuesto y se prueba la sensibilidad
Cada conversión es un visitante	Presupuesto	Es una cota alta: protege la capacidad
Golpe de una tormenta (0, 25 o 50 %)	Monte Carlo	Se prueban los tres valores; el año cambia 1–2 %
Capacidad = el mes más alto ya recibido	Monte Carlo y presupuesto	Se aclara que no es la capacidad física oficial
El futuro se parece al pasado	Rango conformal	Se mide la cobertura real; en la Ruta 2023 falló y se dice
La misma matriz de Markov para 7 centros	Radar semanal	Se declara como simplificación
El dato mensual se conoce al mes siguiente	Torre	En la realidad tarda 1–2 meses; se declara
Pesos iguales en el índice	Radar	Sensibilidad: cambia el estado del 0.6–6.0 % de los casos

11.2 Lo que los modelos no pueden afirmar

- Que el clasificador anticipe saturaciones del sur: en los cinco lugares casi no hubo cambios con qué probarlo.
- Que el clima cause cambios en los visitantes: las sensibilidades son asociaciones.
- Que la tasa de tormentas no esté subiendo: en 1996–2025 hubo el doble que en 1966–1995; con tan pocos casos no es significativo ($p = 0.071$), pero podría serlo con más años.
- El rendimiento real de la campaña: los indicadores tienen metas, pero solo se medirán cuando corra.

Que el clasificador le gane a la persistencia con significancia estadística: la ventaja es de 4 aciertos de 156 (McNemar, $p = 0.19$). Su valor está en anticipar cambios y en sus probabilidades.

Que anticipe las saturaciones nuevas: en el año de prueba hubo 2 lugares que pasaron a saturado desde otro estado y no las anticipó; por eso la Torre en vivo vigila cada semana.

Que el norte «no tenga memoria»: la prueba de Anderson y Goodman rechaza el orden 1 ($p < 0.001$); usamos el orden 1 porque el orden 2 solo mejora el Brier 2 a 3 %.

Que la campaña traiga los 957 visitantes del plan: con la incertidumbre de las fuentes, lo más probable son 788 (entre 579 y 1,016).

Los huecos de datos están en la sección 2.3.

Referencias

Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. En *Proceedings of the 20th International Conference on Very Large Data Bases* (pp. 487–499). Morgan Kaufmann.

Álvarez-Carmona, M. Á., et al. (2025). *Rest-Mex 2025* [Conjunto de datos]. IberLEF 2025. https://portal.odesia.uned.es/en/dataset/rest-mex-2025

Anderson, T. W., & Goodman, L. A. (1957). Statistical inference about Markov chains. The Annals of Mathematical Statistics, 28(1), 89–110. https://doi.org/10.1214/aoms/1177707039

Angelopoulos, A. N., & Bates, S. (2023). Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning, 16*(4), 494–591. https://doi.org/10.1561/2200000101

Bates, J. M., & Granger, C. W. J. (1969). The combination of forecasts. Operational Research Quarterly, 20(4), 451–468. https://doi.org/10.2307/3008764

Birge, J. R., & Louveaux, F. (2011). Introduction to stochastic programming (2.ª ed.). Springer. https://doi.org/10.1007/978-1-4614-0237-4

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3.

Canca Ortiz, J. D., & Villa Caro, G. (2021). *Introducción a la investigación de operaciones*. Editorial Universidad de Sevilla.

Chang, W. L., Boyd, D., & Levin, O. (2019). *NIST Big Data Interoperability Framework: Volume 6, Big Data Reference Architecture* (Version 2) (NIST Special Publication 1500-6r2). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.1500-6r2

Cohen, J. (1960). A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1), 37–46. https://doi.org/10.1177/001316446002000104

Efron, B., & Tibshirani, R. J. (1993). An introduction to the bootstrap. Chapman & Hall/CRC.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Gretzel, U., Sigala, M., Xiang, Z., & Koo, C. (2015). Smart tourism: Foundations and developments. *Electronic Markets, 25*(3), 179–188. https://doi.org/10.1007/s12525-015-0196-8

Hersbach, H., et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society, 146*(730), 1999–2049. https://doi.org/10.1002/qj.3803

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.ª ed.). OTexts. https://otexts.com/fpp3/

Instituto Nacional de Estadística y Geografía. (2021). *Censo de Población y Vivienda 2020: Principales resultados por localidad (ITER)*. INEGI. https://www.inegi.org.mx/programas/ccpv/2020/

Instituto Nacional de Estadística y Geografía. (2025). *Cuenta satélite del turismo de México 2024*. INEGI. https://www.inegi.org.mx/temas/turismosat/

Instituto Nacional de Estadística y Geografía. (2026). *Directorio Estadístico Nacional de Unidades Económicas (DENUE)*. INEGI. https://www.inegi.org.mx/app/mapa/denue/

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3.ª ed.). Wiley.

Kotler, P., & Keller, K. L. (2016). *Marketing management* (15.ª ed.). Pearson.

Landsea, C. W., & Franklin, J. L. (2013). Atlantic hurricane database uncertainty and presentation of a new database format. *Monthly Weather Review, 141*(10), 3576–3592. https://doi.org/10.1175/MWR-D-12-00254.1

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. En *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Massey, F. J., Jr. (1951). The Kolmogorov-Smirnov test for goodness of fit. Journal of the American Statistical Association, 46(253), 68–78. https://doi.org/10.1080/01621459.1951.10500769

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2), 153–157. https://doi.org/10.1007/BF02295996

Monroe, B. L., Colaresi, M. P., & Quinn, K. M. (2008). Fightin' words: Lexical feature selection and evaluation for identifying the content of political conflict. *Political Analysis, 16*(4), 372–403. https://doi.org/10.1093/pan/mpn018

Organización Mundial del Turismo. (2005). *Indicadores de desarrollo sostenible para los destinos turísticos: Guía práctica*. OMT.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Ross, S. M. (2019). *Introduction to probability models* (12.ª ed.). Academic Press.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Secretaría de Turismo. (2025). *Compendio estadístico del turismo en México 2024*. Gobierno de México. https://www.datatur.sectur.gob.mx/SitePages/CompendioEstadistico.aspx

Secretaría de Turismo. (2026). *DataTur: Sistema Nacional de la Información Estadística del Sector Turismo de México*. Gobierno de México. https://www.datatur.sectur.gob.mx/

Secretaría de Turismo del Estado de Quintana Roo. (2026). *SITUR-Q: Sistema de Información Turística de Quintana Roo*. Gobierno de Quintana Roo.

Taha, H. A. (2024). *Investigación de operaciones* (11.ª ed.). Pearson Educación.

Universidad Nacional Rosario Castellanos. (2026). *Problema Prototípico e Incidentes Críticos, semestre 2026-2: Turismo inteligente sustentable para México*. Licenciatura en Ciencias de Datos para Negocios, 5.º semestre.

Wang, X., Smith, K., & Hyndman, R. (2006). Characteristic-based clustering for time series data. *Data Mining and Knowledge Discovery, 13*(3), 335–364. https://doi.org/10.1007/s10618-005-0039-x

Ward, J. H., Jr. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association, 58*(301), 236–244. https://doi.org/10.1080/01621459.1963.10500845

WordStream by LocaliQ. (2025). *Google Ads benchmarks 2025*. https://www.wordstream.com/blog/2025-google-ads-benchmarks

World Tourism Organization. (2018). *'Overtourism'? Understanding and managing urban tourism growth beyond perceptions*. UNWTO. https://doi.org/10.18111/9789284420070

Anexo A. Dónde está cada cálculo

Cada sección de este documento tiene su notebook en la entrega y su módulo en el paquete de código del proyecto (`torre`). Los notebooks rehacen el cálculo a la vista y lo comparan con el resultado que guardó el módulo; los módulos son los que alimentan la página.

Sección	Notebook de la entrega	Módulo del proyecto
3.1 Arquitectura y manifiesto	1 Grandes volúmenes, sección 1	torre.base.manifiesto
3.2 Spark (HURDAT2, DENUE)	1 Grandes volúmenes, secciones 2–4	torre.base.entorno, silver_huracanes, silver_denue
3.3–3.4 Limpieza y conciliación	1 Grandes volúmenes, sección 5	torre.base.silver_siturq, silver_cruceros
3.5 DuckDB y estrella	1 Grandes volúmenes, sección 6	torre.base.almacen, diccionario
3.6 Streaming	1 Grandes volúmenes, sección 7	torre.envivo.torre, senales
4.1 Concentración (HHI)	2 Planteamiento, sección 3	torre.radar.planteamiento
4.2 Criterios	2 Planteamiento, sección 4	torre.radar.criterios
Clima del viajero (SMN y ERA5)	2 Planteamiento, sección 6	torre.base.silver_clima, torre.pronostico.calendario
5.1 Índice de presión	3 Minería, sección 1	torre.radar.panel, indice
5.2 Ward	3 Minería, sección 2	torre.radar.clustering
5.3 Forma del año	3 Minería, sección 3	torre.pronostico.forma
5.4 Isolation Forest	3 Minería, sección 4	torre.envivo.senales
5.5 Minería de texto	3 Minería, sección 5	torre.campana.texto
5.6 Léxico del DENUE	3 Minería, sección 6	torre.campana.lugares
6.1 Clasificador	4 Aprendizaje de máquina, secciones 1–2	torre.radar.prediccion
6.2 Pronóstico y rango	4 Aprendizaje de máquina, secciones 3–5	torre.pronostico.series, modelos, intervalos, seleccion
7.1 Markov	5 Estocásticos, sección 3	torre.radar.markov
7.2–7.3 Tormentas y Poisson	5 Estocásticos, secciones 1–2 y Excel	torre.base.silver_huracanes, torre.pronostico.escenarios
7.4–7.5 Monte Carlo y sensibilidad	5 Estocásticos, secciones 4–5	torre.pronostico.escenarios

Sección	Notebook de la entrega	Módulo del proyecto
8.1–8.7 Presupuesto	6 Investigación de Operaciones, secciones 1–5 y Excel	torre.campana.presupuesto
8.8 Motor semanal	6 Investigación de Operaciones, sección 6	torre.envivo.motor
9 Campaña	7 Mercadotecnia, secciones 1–8	torre.campana.marca
10.3 Página	—	torre.api.datos_pagina, frontend/
3.8 Las columnas que inferimos	1 Grandes volúmenes, sección 8	docs/documento/edicion_94/capturas.py
6.1 Todas las métricas e impacto	4 Aprendizaje de máquina, secciones 2.1 a 2.4	torre.radar.prediccion.metricas_detalle
6.2 Combinar modelos	—	torre.pronostico.seleccion.combinaciones
7.6 El año como proceso de Poisson	5 Modelos estocásticos, sección 2	torre.pronostico.escenarios.temporada_tormentas
7.7 Markov a largo plazo y prueba de orden	5 Modelos estocásticos, sección 3	torre.radar.markov (largo_plazo, prueba_orden, comparar_orden2)
7.8 Monte Carlo de la campaña	5 Modelos estocásticos, sección 4	torre.campana.presupuesto.montecarlo_campana
8.9 VSS y EVPI	6 Investigación de operaciones, sección 5	torre.campana.presupuesto.valor_estocastico

Anexo B. Glosario

Los términos técnicos que aparecen en el documento y en los fragmentos de código, en orden alfabético, explicados en una o dos frases.

Término	Qué quiere decir
Apriori y lift	Apriori busca combinaciones de cosas que aparecen juntas (temas de una reseña). El lift dice cuántas veces más probable es la consecuencia cuando aparece la combinación: lift 2 es el doble de lo normal.
Bootstrap	Volver a sacar, con reemplazo, muestras del mismo tamaño de los casos que ya se tienen, miles de veces, para ver cuánto podría variar una métrica. Así sale el intervalo del 90 % de la exactitud.
Bronze, Silver y Gold	Las tres capas de datos: el archivo tal como se descargó, la tabla limpia y con tipos, y los resultados de los modelos.
Brier	Puntaje que mide qué tan buenas son unas probabilidades: suma de las diferencias al cuadrado entre lo que se predijo y lo que pasó. 0 es perfecto.
Buyer persona	Retrato de un cliente ideal armado con datos; en la campaña, cada rasgo dice si es dato, derivado, supuesto o hueco.
Cadena de Markov	Modelo en el que el estado de la semana siguiente depende solo del estado de esta semana, con probabilidades que se leen en la matriz de transición.
CBC y PuLP	PuLP es la librería de Python con la que escribimos el modelo de optimización; CBC es el programa libre que lo resuelve.
Cobertura	Porcentaje de veces que el valor real cayó dentro del rango pronosticado. Se mide, no se supone.
coincide	Función de los notebooks que compara una cifra recién calculada con la que guardó el proyecto y detiene el notebook si no son iguales.
CTR, CPC y conversión	Tasa de clics (clics ÷ personas que vieron el anuncio), costo por clic y fracción de clics que terminan en un plan de viaje.
DataFrame	Una tabla en memoria, de pandas o de Spark, con renglones y columnas con nombre.
DuckDB	Base de datos SQL que vive en un archivo y lee los Parquet sin copiarlos; sirve para consultar en milisegundos.
Evaluación perezosa	Spark no calcula al escribir una transformación: arma el plan completo y lo ejecuta cuando se pide un resultado.
Exactitud balanceada	Promedio de la sensibilidad de cada clase: no premia acertar solo la clase más común.
F1-macro	Promedio del F1 (mezcla de precisión y sensibilidad) de los tres

Término	Qué quiere decir
	estados, con el mismo peso aunque «saturado» sea raro.
Firestore	Base de datos NoSQL de documentos de Firebase (Google). La página guarda ahí los corazones y las solicitudes de cotización.
Frontera de Pareto	Conjunto de soluciones en el que no se puede mejorar un objetivo (visitantes) sin empeorar el otro (presión sobre la capacidad).
Función de ventana	Cálculo que mira renglones vecinos: el último encabezado de la tormenta, o las cuatro semanas anteriores de un centro.
Hiperparámetro	Valor que se fija antes de entrenar un modelo (número de árboles, umbral, radio); cada uno está justificado en su notebook.
Importancia por permutación	Cuánto baja la exactitud si se revuelve al azar una columna; si baja mucho, el modelo depende de ella.
Índice de Herfindahl-Hirschman (HHI)	Suma de las cuotas al cuadrado; mide qué tan concentrado está un reparto. Normalizado va de 0 (parejo) a 1 (todo en una unidad).
Intervalo conformal	Rango del 90 % calculado con los errores reales que el modelo ya cometió, sin suponer que siguen una curva normal.
Isolation Forest	Detector de rarezas: parte los datos con cortes al azar; lo raro queda aislado en pocos cortes.
Kappa de Cohen	Acierto corregido por el que se lograría al azar con las mismas frecuencias. 0 es azar y 1 es perfecto.
Kolmogórov-Smirnov	Prueba que compara una distribución observada con una teórica por la mayor distancia entre sus curvas acumuladas.
Log-loss	Menos el promedio del logaritmo de la probabilidad que el modelo le dio a lo que sí pasó; castiga mucho decir «casi imposible» y fallar.
MAE y MAPE	Error absoluto medio (en las unidades de la serie) y error porcentual medio (para comparar series distintas).
McNemar	Prueba que compara dos modelos en los mismos casos, contando solo aquellos donde uno acierta y el otro no.
Monte Carlo	Simular muchos futuros posibles (aquí, 10,000) para ver qué tan malo o bueno puede salir un año.
Origen móvil	Probar un modelo como si cada mes del pasado fuera hoy: entrenar solo con lo anterior y comparar con lo que pasó.
Parquet	Formato de archivo por columnas, comprimido, que guarda el tipo de cada dato (fecha, número, texto).
Partición	Subcarpeta de una tabla por el valor de una columna (año, estado); permite leer solo lo necesario.
Percentil	Valor debajo del cual queda cierto porcentaje de los datos: el percentil 90 deja abajo al 90 %.

Término	Qué quiere decir
Precio sombra	Cuánto mejora el objetivo si una restricción se afloja una unidad. Una restricción que no aprieta tiene precio cero.
Proceso de Poisson	Modelo para contar eventos raros e independientes en un periodo; solo necesita la tasa promedio λ.
Programación lineal y binaria	Optimizar una función con restricciones lineales; binaria cuando una variable solo puede valer 0 o 1 (encendido o pausado).
Propiedad de Markov	Que el futuro dependa solo del estado de hoy y no de cómo se llegó a él. Se puede probar con los datos (Anderson y Goodman).
Semilla	Número que fija el azar de una simulación para que cualquiera obtenga exactamente los mismos resultados (aquí, 2026).
SHA-256	Huella digital de 64 caracteres de un archivo; si cambia un solo byte, la huella cambia por completo.
Silueta	Medida de qué tan bien quedó cada elemento en su grupo, de −1 a 1; sirve para elegir cuántos grupos hacer.
Spark y PySpark	Spark reparte el trabajo de una tabla grande entre los núcleos (o entre máquinas); PySpark es su versión para Python.
Structured Streaming	La forma de Spark de procesar datos que llegan por partes; aquí, una semana por lote.
Validación y prueba	El periodo con el que se elige el modelo y el periodo, distinto, con el que solo se mide.
VSS y EVPI	En un modelo estocástico, cuánto se gana por planear con la incertidumbre (VSS) y cuánto valdría conocer el futuro (EVPI).
Ward	Agrupamiento jerárquico que en cada paso une los dos grupos que menos aumentan la variación dentro de los grupos.